

Credit_Risk_Analysis

Updating and Importing Libraries

In [1]: `!pip install imblearn`

```
Requirement already satisfied: imblearn in c:\users\home\anaconda3\lib\site-packages (0.0)
Requirement already satisfied: imbalanced-learn in c:\users\home\anaconda3\lib\site-packages (from imblearn) (0.12.4)
Requirement already satisfied: numpy>=1.17.3 in c:\users\home\anaconda3\lib\site-packages (from imbalanced-learn->imblearn) (2.3.5)
Requirement already satisfied: scipy>=1.5.0 in c:\users\home\anaconda3\lib\site-packages (from imbalanced-learn->imblearn) (1.16.3)
Requirement already satisfied: scikit-learn>=1.0.2 in c:\users\home\anaconda3\lib\site-packages (from imbalanced-learn->imblearn) (1.5.2)
Requirement already satisfied: joblib>=1.1.1 in c:\users\home\anaconda3\lib\site-packages (from imbalanced-learn->imblearn) (1.5.2)
Requirement already satisfied: threadpoolctl>=2.0.0 in c:\users\home\anaconda3\lib\site-packages (from imbalanced-learn->imblearn) (3.5.0)
```

In [1]: `import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

import statsmodels`

In [3]: `## to suppress any Pandas warning

pd.set_option('mode.chained_assignment', None)`

In [4]: `from sklearn.preprocessing import MinMaxScaler, LabelEncoder, OneHotEncoder`

In [5]: `from sklearn.linear_model import LogisticRegression
from sklearn import metrics
from sklearn.metrics import confusion_matrix, accuracy_score, classification_report
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GridSearchCV`

In [6]: `from sklearn.model_selection import train_test_split`

In [7]: `from sklearn.tree import DecisionTreeClassifier`

In [8]: `!pip install xgboost`

Requirement already satisfied: xgboost in c:\users\home\anaconda3\lib\site-packages (3.2.0)
 Requirement already satisfied: numpy in c:\users\home\anaconda3\lib\site-packages (from xgboost) (2.3.5)
 Requirement already satisfied: scipy in c:\users\home\anaconda3\lib\site-packages (from xgboost) (1.16.3)

In [9]: `from xgboost import XGBClassifier`

In [10]: `!pip install catboost`

Requirement already satisfied: catboost in c:\users\home\anaconda3\lib\site-packages (1.2.10)
 Requirement already satisfied: graphviz in c:\users\home\anaconda3\lib\site-packages (from catboost) (0.21)
 Requirement already satisfied: matplotlib in c:\users\home\anaconda3\lib\site-packages (from catboost) (3.10.9)
 Requirement already satisfied: numpy<3.0,>=1.16.0 in c:\users\home\anaconda3\lib\site-packages (from catboost) (2.3.5)
 Requirement already satisfied: pandas<4.0,>=0.24 in c:\users\home\anaconda3\lib\site-packages (from catboost) (2.3.3)
 Requirement already satisfied: scipy in c:\users\home\anaconda3\lib\site-packages (from catboost) (1.16.3)
 Requirement already satisfied: plotly in c:\users\home\anaconda3\lib\site-packages (from catboost) (6.3.0)
 Requirement already satisfied: six in c:\users\home\anaconda3\lib\site-packages (from catboost) (1.17.0)
 Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\home\anaconda3\lib\site-packages (from pandas<4.0,>=0.24->catboost) (2.9.0.post0)
 Requirement already satisfied: pytz>=2020.1 in c:\users\home\anaconda3\lib\site-packages (from pandas<4.0,>=0.24->catboost) (2025.2)
 Requirement already satisfied: tzdata>=2022.7 in c:\users\home\anaconda3\lib\site-packages (from pandas<4.0,>=0.24->catboost) (2025.2)
 Requirement already satisfied: contourpy>=1.0.1 in c:\users\home\anaconda3\lib\site-packages (from matplotlib->catboost) (1.3.3)
 Requirement already satisfied: cycler>=0.10 in c:\users\home\anaconda3\lib\site-packages (from matplotlib->catboost) (0.11.0)
 Requirement already satisfied: fonttools>=4.22.0 in c:\users\home\anaconda3\lib\site-packages (from matplotlib->catboost) (4.60.1)
 Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\home\anaconda3\lib\site-packages (from matplotlib->catboost) (1.4.9)
 Requirement already satisfied: packaging>=20.0 in c:\users\home\anaconda3\lib\site-packages (from matplotlib->catboost) (25.0)
 Requirement already satisfied: pillow>=8 in c:\users\home\anaconda3\lib\site-packages (from matplotlib->catboost) (12.0.0)
 Requirement already satisfied: pyparsing>=3 in c:\users\home\anaconda3\lib\site-packages (from matplotlib->catboost) (3.2.5)
 Requirement already satisfied: narwhals>=1.15.1 in c:\users\home\anaconda3\lib\site-packages (from plotly->catboost) (2.7.0)

In [11]: `from catboost import CatBoostClassifier`

In [12]: `!pip install lightgbm`

Requirement already satisfied: lightgbm in c:\users\home\anaconda3\lib\site-packages (4.6.0)

Requirement already satisfied: numpy>=1.17.0 in c:\users\home\anaconda3\lib\site-packages (from lightgbm) (2.3.5)

Requirement already satisfied: scipy in c:\users\home\anaconda3\lib\site-packages (from lightgbm) (1.16.3)

```
In [13]: import lightgbm as lgb
        from lightgbm import LGBMClassifier
        from sklearn.metrics import mean_squared_error
```

```
In [14]: from sklearn.metrics import roc_auc_score

        import pandas.core.algorithms as algos
        import scipy.stats.stats as stats
        import traceback
        import string
```

```
In [15]: !pip install scorecardpy
```

Requirement already satisfied: scorecardpy in c:\users\home\anaconda3\lib\site-packages (0.1.9.7)

Requirement already satisfied: numpy in c:\users\home\anaconda3\lib\site-packages (from scorecardpy) (2.3.5)

Requirement already satisfied: pandas>=1.0.0 in c:\users\home\anaconda3\lib\site-packages (from scorecardpy) (2.3.3)

Requirement already satisfied: matplotlib in c:\users\home\anaconda3\lib\site-packages (from scorecardpy) (3.10.9)

Requirement already satisfied: scikit-learn>=0.19.1 in c:\users\home\anaconda3\lib\site-packages (from scorecardpy) (1.5.2)

Requirement already satisfied: statsmodels in c:\users\home\anaconda3\lib\site-packages (from scorecardpy) (0.14.5)

Requirement already satisfied: patsy in c:\users\home\anaconda3\lib\site-packages (from scorecardpy) (1.0.1)

Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\home\anaconda3\lib\site-packages (from pandas>=1.0.0->scorecardpy) (2.9.0.post0)

Requirement already satisfied: pytz>=2020.1 in c:\users\home\anaconda3\lib\site-packages (from pandas>=1.0.0->scorecardpy) (2025.2)

Requirement already satisfied: tzdata>=2022.7 in c:\users\home\anaconda3\lib\site-packages (from pandas>=1.0.0->scorecardpy) (2025.2)

Requirement already satisfied: six>=1.5 in c:\users\home\anaconda3\lib\site-packages (from python-dateutil>=2.8.2->pandas>=1.0.0->scorecardpy) (1.17.0)

Requirement already satisfied: scipy>=1.6.0 in c:\users\home\anaconda3\lib\site-packages (from scikit-learn>=0.19.1->scorecardpy) (1.16.3)

Requirement already satisfied: joblib>=1.2.0 in c:\users\home\anaconda3\lib\site-packages (from scikit-learn>=0.19.1->scorecardpy) (1.5.2)

Requirement already satisfied: threadpoolctl>=3.1.0 in c:\users\home\anaconda3\lib\site-packages (from scikit-learn>=0.19.1->scorecardpy) (3.5.0)

Requirement already satisfied: contourpy>=1.0.1 in c:\users\home\anaconda3\lib\site-packages (from matplotlib->scorecardpy) (1.3.3)

Requirement already satisfied: cycycler>=0.10 in c:\users\home\anaconda3\lib\site-packages (from matplotlib->scorecardpy) (0.11.0)

Requirement already satisfied: fonttools>=4.22.0 in c:\users\home\anaconda3\lib\site-packages (from matplotlib->scorecardpy) (4.60.1)

Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\home\anaconda3\lib\site-packages (from matplotlib->scorecardpy) (1.4.9)

Requirement already satisfied: packaging>=20.0 in c:\users\home\anaconda3\lib\site-packages (from matplotlib->scorecardpy) (25.0)

Requirement already satisfied: pillow>=8 in c:\users\home\anaconda3\lib\site-packages (from matplotlib->scorecardpy) (12.0.0)

Requirement already satisfied: pyparsing>=3 in c:\users\home\anaconda3\lib\site-packages (from matplotlib->scorecardpy) (3.2.5)

In [16]: `import scorecardpy as sc`

C:\Users\Home\anaconda3\Lib\site-packages\scorecardpy\germancredit.py:5: UserWarning: pkg_resources is deprecated as an API. See https://setuptools.pypa.io/en/latest/pkg_resources.html. The pkg_resources package is slated for removal as early as 2025-11-30. Refrain from using this package or pin to Setuptools<81.

`import pkg_resources`

In [17]: `!pip install mlxtend --break-system-packages`

Requirement already satisfied: mlxtend in c:\users\home\anaconda3\lib\site-packages (0.23.1)

Requirement already satisfied: scipy>=1.2.1 in c:\users\home\anaconda3\lib\site-packages (from mlxtend) (1.16.3)

Requirement already satisfied: numpy>=1.16.2 in c:\users\home\anaconda3\lib\site-packages (from mlxtend) (2.3.5)

Requirement already satisfied: pandas>=0.24.2 in c:\users\home\anaconda3\lib\site-packages (from mlxtend) (2.3.3)

Requirement already satisfied: scikit-learn>=1.0.2 in c:\users\home\anaconda3\lib\site-packages (from mlxtend) (1.5.2)

Requirement already satisfied: matplotlib>=3.0.0 in c:\users\home\anaconda3\lib\site-packages (from mlxtend) (3.10.9)

Requirement already satisfied: joblib>=0.13.2 in c:\users\home\anaconda3\lib\site-packages (from mlxtend) (1.5.2)

Requirement already satisfied: contourpy>=1.0.1 in c:\users\home\anaconda3\lib\site-packages (from matplotlib>=3.0.0->mlxtend) (1.3.3)

Requirement already satisfied: cyclor>=0.10 in c:\users\home\anaconda3\lib\site-packages (from matplotlib>=3.0.0->mlxtend) (0.11.0)

Requirement already satisfied: fonttools>=4.22.0 in c:\users\home\anaconda3\lib\site-packages (from matplotlib>=3.0.0->mlxtend) (4.60.1)

Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\home\anaconda3\lib\site-packages (from matplotlib>=3.0.0->mlxtend) (1.4.9)

Requirement already satisfied: packaging>=20.0 in c:\users\home\anaconda3\lib\site-packages (from matplotlib>=3.0.0->mlxtend) (25.0)

Requirement already satisfied: pillow>=8 in c:\users\home\anaconda3\lib\site-packages (from matplotlib>=3.0.0->mlxtend) (12.0.0)

Requirement already satisfied: pyparsing>=3 in c:\users\home\anaconda3\lib\site-packages (from matplotlib>=3.0.0->mlxtend) (3.2.5)

Requirement already satisfied: python-dateutil>=2.7 in c:\users\home\anaconda3\lib\site-packages (from matplotlib>=3.0.0->mlxtend) (2.9.0.post0)

Requirement already satisfied: pytz>=2020.1 in c:\users\home\anaconda3\lib\site-packages (from pandas>=0.24.2->mlxtend) (2025.2)

Requirement already satisfied: tzdata>=2022.7 in c:\users\home\anaconda3\lib\site-packages (from pandas>=0.24.2->mlxtend) (2025.2)

Requirement already satisfied: six>=1.5 in c:\users\home\anaconda3\lib\site-packages (from python-dateutil>=2.7->matplotlib>=3.0.0->mlxtend) (1.17.0)

Requirement already satisfied: threadpoolctl>=3.1.0 in c:\users\home\anaconda3\lib\site-packages (from scikit-learn>=1.0.2->mlxtend) (3.5.0)

```
In [18]: import mlxtend
```

```
In [19]: # Show every column without cutting off data

pd.set_option('display.max_columns', None)
```

```
In [20]: # Set a specific maximum column width for text values

pd.set_option('display.max_colwidth', None)
```

```
In [ ]:
```

Uploading Dataset

```
In [21]: train = pd.read_csv('train.csv')
train.head()
```

```
Out[21]:
```

	customer_id	name	age	gender	owns_car	owns_house	no_of_children	net_yearly_i
0	CST_115179	ita Bose	46	F	N	Y	0.0	107
1	CST_121920	Alper Jonathan	29	M	N	Y	0.0	109
2	CST_109330	Umesh Desai	37	M	N	Y	0.0	230
3	CST_128288	Rie	39	F	N	Y	0.0	122
4	CST_151355	McCool	46	M	Y	Y	0.0	387

◀  ▶

```
In [22]: train.shape
```

```
Out[22]: (45528, 19)
```

```
In [23]: train["credit_card_default"].value_counts(normalize=True)
```

```
Out[23]: credit_card_default
0    0.918797
1    0.081203
Name: proportion, dtype: float64
```

```
In [24]: # reshaping(downsizing) the dataset

df = train.sample(n = 30000, random_state = 42, replace = False)
df
```

Out[24]:

	customer_id	name	age	gender	owns_car	owns_house	no_of_children	net_y
34126	CST_163708	Wachtel	50	F	N	Y	0.0	
44609	CST_131305	Deepa	29	F	N	Y	0.0	
44264	CST_131350	Jason	39	F	N	Y	0.0	
33556	CST_106655	Gilbert Kreijger	35	F	Y	Y	1.0	
28050	CST_154013	Slater	53	M	Y	Y	0.0	
...	...	...	...	...	...	...	...	...
37523	CST_113336	Leika Kihara	26	F	N	Y	0.0	
36228	CST_151817	Dixon	31	F	N	N	0.0	
32772	CST_107659	P.J. Huffstutter	50	F	N	N	0.0	
9762	CST_141352	an	25	M	N	Y	0.0	
33325	CST_129867	Hyunjoo	50	F	N	N	0.0	

30000 rows × 19 columns



In []:

Exploratory Data Analysis

In [25]:

df.shape

Out[25]: (30000, 19)

In [26]:

df.info()

```
<class 'pandas.core.frame.DataFrame'>
Index: 30000 entries, 34126 to 33325
Data columns (total 19 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   customer_id                          30000 non-null  object
1   name                                  30000 non-null  object
2   age                                   30000 non-null  int64
3   gender                               30000 non-null  object
4   owns_car                             29631 non-null  object
5   owns_house                           30000 non-null  object
6   no_of_children                       29498 non-null  float64
7   net_yearly_income                   30000 non-null  float64
8   no_of_days_employed                 29685 non-null  float64
9   occupation_type                     30000 non-null  object
10  total_family_members                 29944 non-null  float64
11  migrant_worker                      29947 non-null  float64
12  yearly_debt_payments                 29938 non-null  float64
13  credit_limit                         30000 non-null  float64
14  credit_limit_used(%)                30000 non-null  int64
15  credit_score                         29994 non-null  float64
16  prev_defaults                       30000 non-null  int64
17  default_in_last_6months             30000 non-null  int64
18  credit_card_default                 30000 non-null  int64
dtypes: float64(8), int64(5), object(6)
memory usage: 4.6+ MB
```

In []:

```
In [27]: cat_col=[col for col in df.columns if df[col].dtypes == 'object']
cat_col
```

Out[27]: ['customer_id', 'name', 'gender', 'owns_car', 'owns_house', 'occupation_type']

```
In [28]: num_col=[col for col in df.columns if df[col].dtypes != 'object']
num_col
```

Out[28]: ['age',
'no_of_children',
'net_yearly_income',
'no_of_days_employed',
'total_family_members',
'migrant_worker',
'yearly_debt_payments',
'credit_limit',
'credit_limit_used(%)',
'credit_score',
'prev_defaults',
'default_in_last_6months',
'credit_card_default']

```
In [29]: print('No cat_col:', len(cat_col))
print('No num_col:', len(num_col))
```

No cat_col: 6

No num_col: 13


```
In [30]: df.duplicated().sum()
```

```
Out[30]: np.int64(0)
```

```
In [ ]:
```

Identifying missing values

```
In [31]: df.isnull().sum().sort_values(ascending=False)
```

```
Out[31]: no_of_children      502
         owns_car          369
         no_of_days_employed 315
         yearly_debt_payments 62
         total_family_members 56
         migrant_worker      53
         credit_score         6
         customer_id          0
         default_in_last_6months 0
         prev_defaults         0
         credit_limit_used(%)  0
         credit_limit          0
         occupation_type      0
         name                  0
         net_yearly_income     0
         owns_house            0
         gender                0
         age                   0
         credit_card_default    0
         dtype: int64
```

```
In [32]: df['credit_card_default'].value_counts()
```

```
Out[32]: credit_card_default
         0    27537
         1     2463
         Name: count, dtype: int64
```

```
In [33]: df['credit_card_default'].value_counts(normalize=True)
```

```
Out[33]: credit_card_default
         0    0.9179
         1    0.0821
         Name: proportion, dtype: float64
```

After reshaping, the value_count for credit_card_default remains almost the same. 0(non-defaulters) is 91.79% and 1(defaulter) 8.21%

```
In [ ]:
```

Handling missing values(using median,mode)

Fill NAN for Categorical features

```
In [34]: df[cat_col].isnull().sum()
```

```
Out[34]: customer_id      0  
         name            0  
         gender          0  
         owns_car        369  
         owns_house      0  
         occupation_type  0  
         dtype: int64
```

```
In [35]: for i in cat_col:  
         print("Column Name:", i)  
         print(df[i].value_counts(normalize=True))  
         print("\n")
```

Column Name: customer_id

customer_id

CST_163708 0.000033

CST_131964 0.000033

CST_162494 0.000033

CST_138114 0.000033

CST_102185 0.000033

...

CST_148239 0.000033

CST_130171 0.000033

CST_132131 0.000033

CST_112807 0.000033

CST_129867 0.000033

Name: proportion, Length: 30000, dtype: float64

Column Name: name

name

Jonathan 0.004367

Jonathan Stempel 0.004133

Jessica 0.003600

Sarah 0.003533

David 0.003467

...

Baker Jonathan 0.000033

Joshua Franklin 0.000033

Fathiya Dahrul 0.000033

Becker 0.000033

Olga Dzyubenko 0.000033

Name: proportion, Length: 3514, dtype: float64

Column Name: gender

gender

F 0.657700

M 0.342267

XNA 0.000033

Name: proportion, dtype: float64

Column Name: owns_car

owns_car

N 0.660862

Y 0.339138

Name: proportion, dtype: float64

Column Name: owns_house

owns_house

Y 0.6962

N 0.3038

Name: proportion, dtype: float64

Column Name: occupation_type

occupation_type

```

Unknown          0.314500
Laborers         0.178200
Sales staff      0.103000
Core staff       0.089900
Managers         0.069033
Drivers          0.060767
High skill tech staff 0.037567
Accountants      0.031433
Medicine staff   0.028067
Security staff   0.022100
Cooking staff    0.019667
Cleaning staff   0.015000
Private service staff 0.008467
Low-skill Laborers 0.007633
Secretaries      0.004700
Waiters/barmen staff 0.004600
Realty agents    0.002000
HR staff         0.001700
IT staff         0.001667
Name: proportion, dtype: float64

```

As there are missing values mostly less than 2%, so, let us impute all missing values with statistical way. FillNA by MODE - categorical variable and FillNA by MEDIAN - numerical variable

In []:

FillNA by MODE - categorical variable

Gender

In [36]: `df['gender'].value_counts()`

```

Out[36]: gender
F      19731
M      10268
XNA         1
Name: count, dtype: int64

```

In [37]: `## Replacing XNA with M`

```

df['gender'] = df['gender'].apply(lambda x: 'M' if x=='XNA' else x)
df['gender'].value_counts()

```

```

Out[37]: gender
F      19731
M      10269
Name: count, dtype: int64

```

Owns_Car (cat_col)

In [38]: `df.fillna({'owns_car':df['owns_car'].mode()[0]},inplace=True)`
`df['owns_car'].value_counts(normalize=True)`

```
Out[38]: owns_car
N    0.665033
Y    0.334967
Name: proportion, dtype: float64
```

```
In [39]: df[cat_col].isnull().sum()
```

```
Out[39]: customer_id    0
name                  0
gender                0
owns_car              0
owns_house            0
occupation_type       0
dtype: int64
```

```
In [ ]:
```

FillNA by MEDIAN - Numerical Column

```
In [40]: df[num_col].isnull().sum().sort_values(ascending=False).head(10)
```

```
Out[40]: no_of_children    502
no_of_days_employed    315
yearly_debt_payments    62
total_family_members    56
migrant_worker          53
credit_score             6
age                      0
net_yearly_income        0
credit_limit             0
credit_limit_used(%)     0
dtype: int64
```

no_of_children(num_col)

```
In [41]: df.fillna({'no_of_children': df['no_of_children'].mode()[0]}, inplace=True)
int(df['no_of_children'].isna().sum())
```

```
Out[41]: 0
```

no_of_days_employed (num_col)

```
In [42]: # impute 'no_of_days_employed' with median based on 'occupation_type'

occ_p_days_employed = df.groupby(by='occupation_type')['no_of_days_employed'].median

for i in occ_p_days_employed.index:
    df['no_of_days_employed'] = np.where((df['no_of_days_employed'].isna()) & (df['
int(df.no_of_days_employed.isna().sum())
```

```
Out[42]: 0
```

yearly_debt_payments(num_col)

```
In [43]: ### impute 'yearly_debt_payments' with median based on 'credit_card_default'

yearDebt_credit = df.groupby(by='credit_card_default')['yearly_debt_payments'].medi
for i in yearDebt_credit.index:
    df['yearly_debt_payments'] = np.where((df['yearly_debt_payments'].isna()) & (df

int(df.yearly_debt_payments.isna().sum())
```

Out[43]: 0

total_family_members(num_col)

```
In [44]: # imputing with mode of the data

df.fillna({'total_family_members': df['total_family_members'].mode()[0]}, inplace=T

#check null values count
int(df['total_family_members'].isna().sum())
```

Out[44]: 0

migrant_worker(num_col)

```
In [45]: # imputing with mode of the data
df.fillna({'migrant_worker': df['migrant_worker'].mode()[0]}, inplace=True)

# check null values count
int(df['migrant_worker'].isna().sum())
```

Out[45]: 0

credit_score(num_col)

```
In [46]: # check medians of credit_score wrt credit_card_default
cred_score = df.groupby(by='credit_card_default')['credit_score'].median()

for i in cred_score.index:
    df['credit_score'] = np.where((df['credit_score'].isna()) & (df['credit_card_de

# check for null values in 'credit_score'
int(df['credit_score'].isna().sum())
```

Out[46]: 0

```
In [47]: df[num_col].isnull().sum().sort_values(ascending = False)
```

```
Out[47]: age                0
no_of_children            0
net_yearly_income         0
no_of_days_employed       0
total_family_members       0
migrant_worker            0
yearly_debt_payments       0
credit_limit              0
credit_limit_used(%)       0
credit_score              0
prev_defaults             0
default_in_last_6months    0
credit_card_default        0
dtype: int64
```

```
In [48]: # cat and num col
```

```
df.isnull().sum()
```

```
Out[48]: customer_id        0
name                      0
age                      0
gender                   0
owns_car                 0
owns_house              0
no_of_children           0
net_yearly_income        0
no_of_days_employed      0
occupation_type          0
total_family_members     0
migrant_worker           0
yearly_debt_payments     0
credit_limit             0
credit_limit_used(%)     0
credit_score             0
prev_defaults            0
default_in_last_6months  0
credit_card_default       0
dtype: int64
```

```
In [ ]:
```

Concentrated EDA

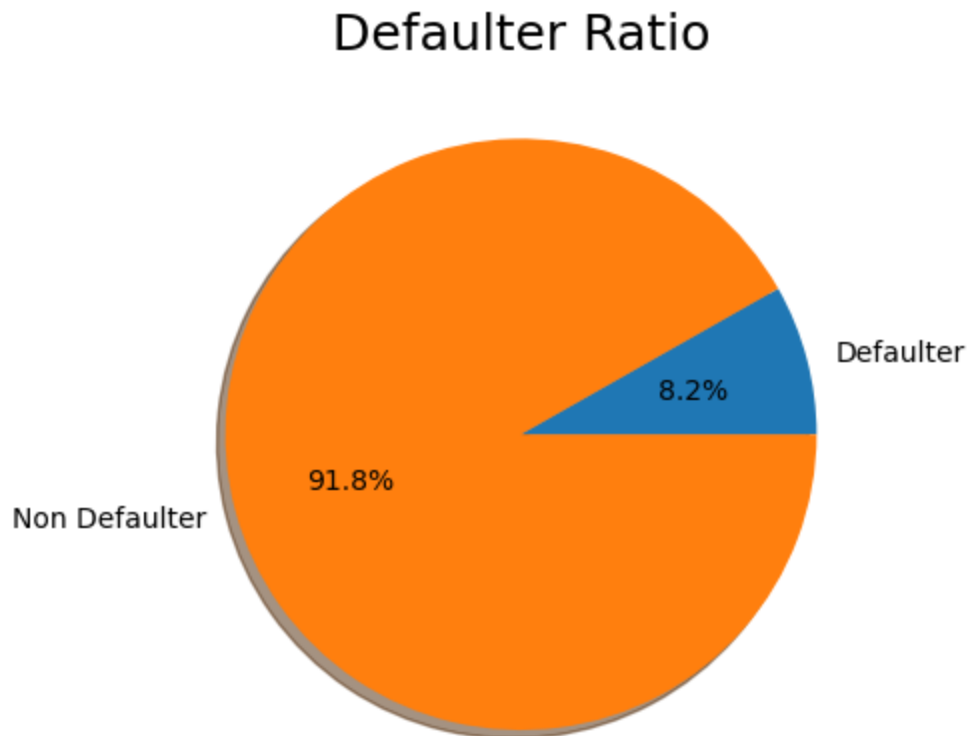
(Key features - credit_card_default, age, credit_score, credit limit percentage, occupation, gender)

```
In [49]: # Plotting Credit_default ratio
```

```
total_default_values = df.credit_card_default.value_counts(ascending=True).values

plt.figure()
plt.pie(total_default_values, labels=['Defaulter', 'Non Defaulter'], autopct='%1.1f
```

```
plt.title('Defaulter Ratio', fontsize=18)  
plt.show()
```

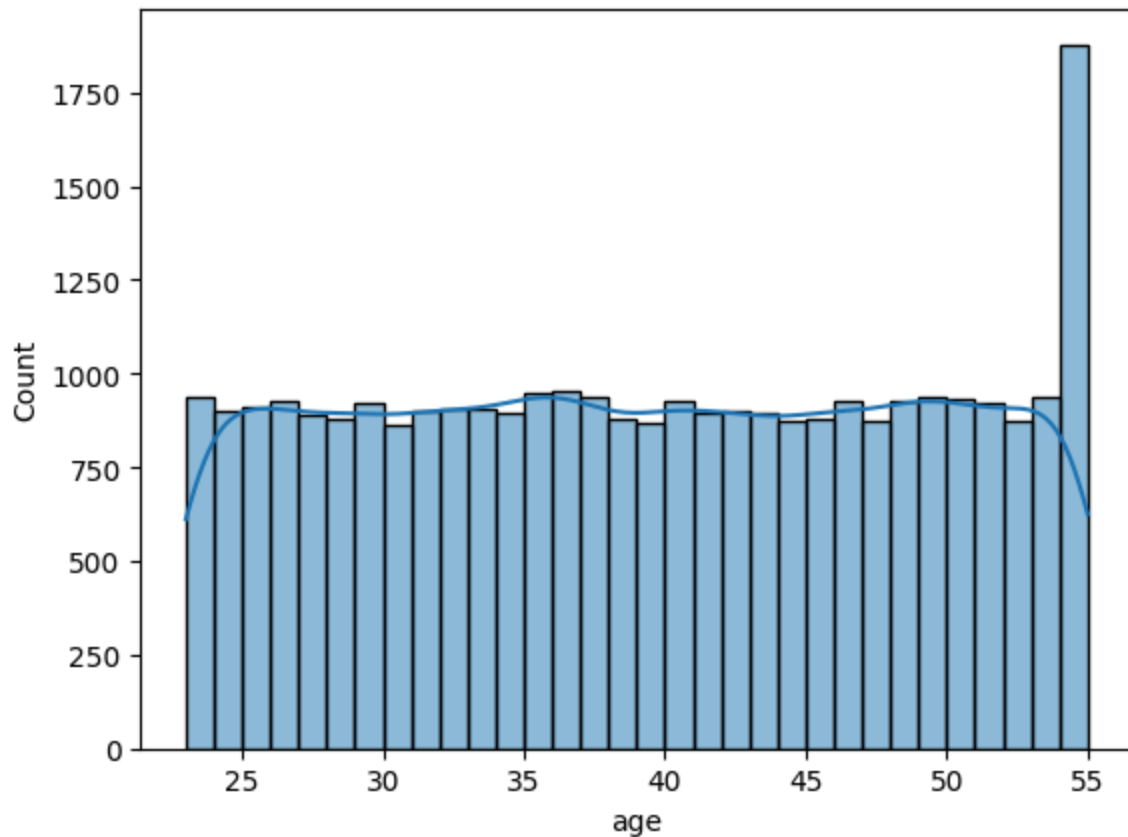


The pie chart shows the distribution of the target variable - 'credit_card_default' Non Defaulter: 91.8% (Majority customers)
Defaulter: 8.2% (Very few customers) Reflecting a highly imbalanced dataset.

In []:

In [50]: *# visualize AGE feature - histplot*

```
plt.figure()  
sns.histplot(df['age'], kde=True)  
plt.show()
```

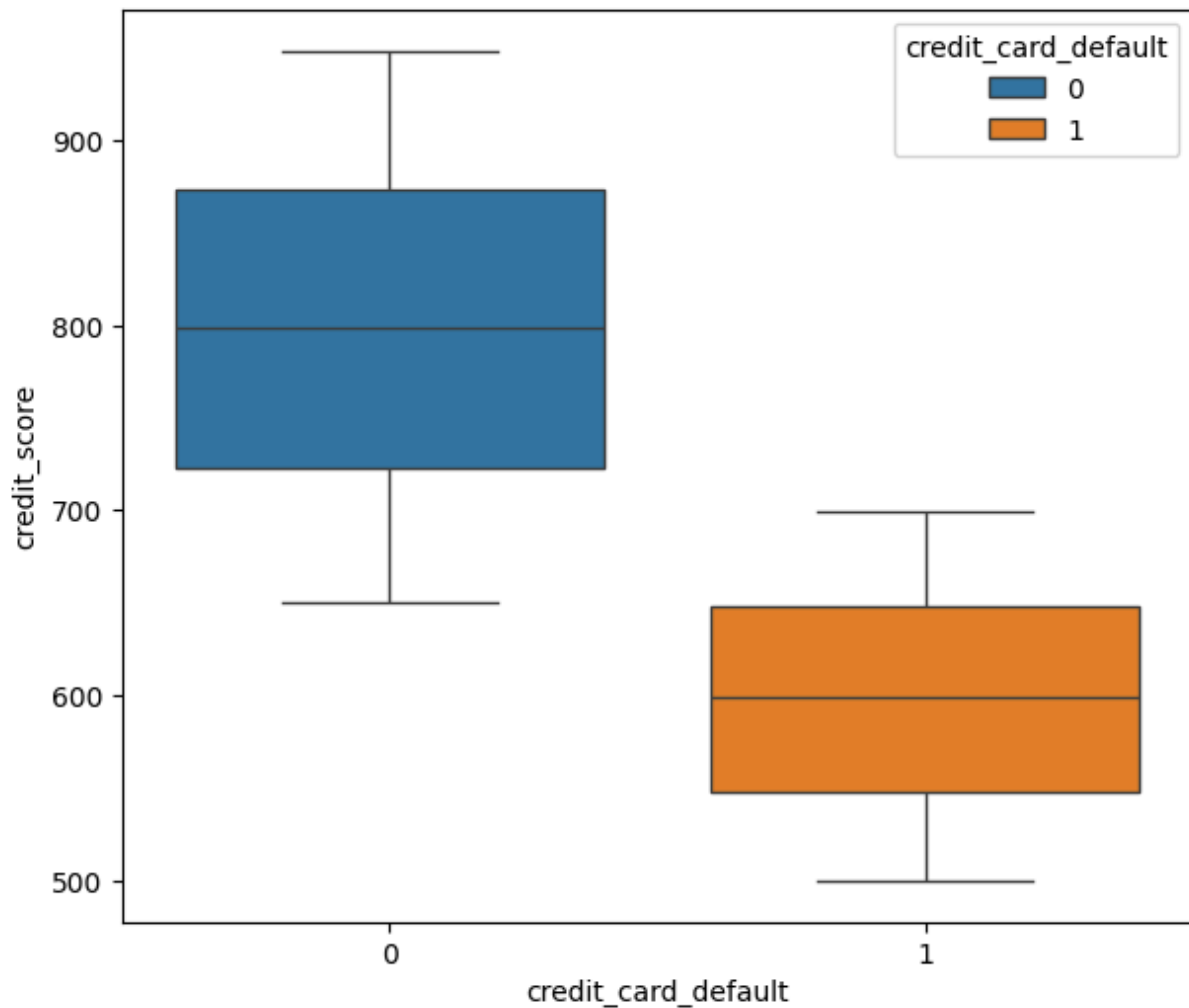



-Most ages lie between 23 to 55. -The distribution looks - uniformly spread,no strong skewness,no extreme outliers visible -There is only 1 noticeable spike near 55,as many customers fall in that age high might be because of age capping during preprocessing, synthetic/generated data rounded values.

In []:

In [51]: *# boxplot of credit_defaulter-credit_score*

```
plt.figure(figsize=[7,6])
sns.boxplot(x=df['credit_card_default'], y=df['credit_score'],hue=df['credit_card_d
plt.show()
```

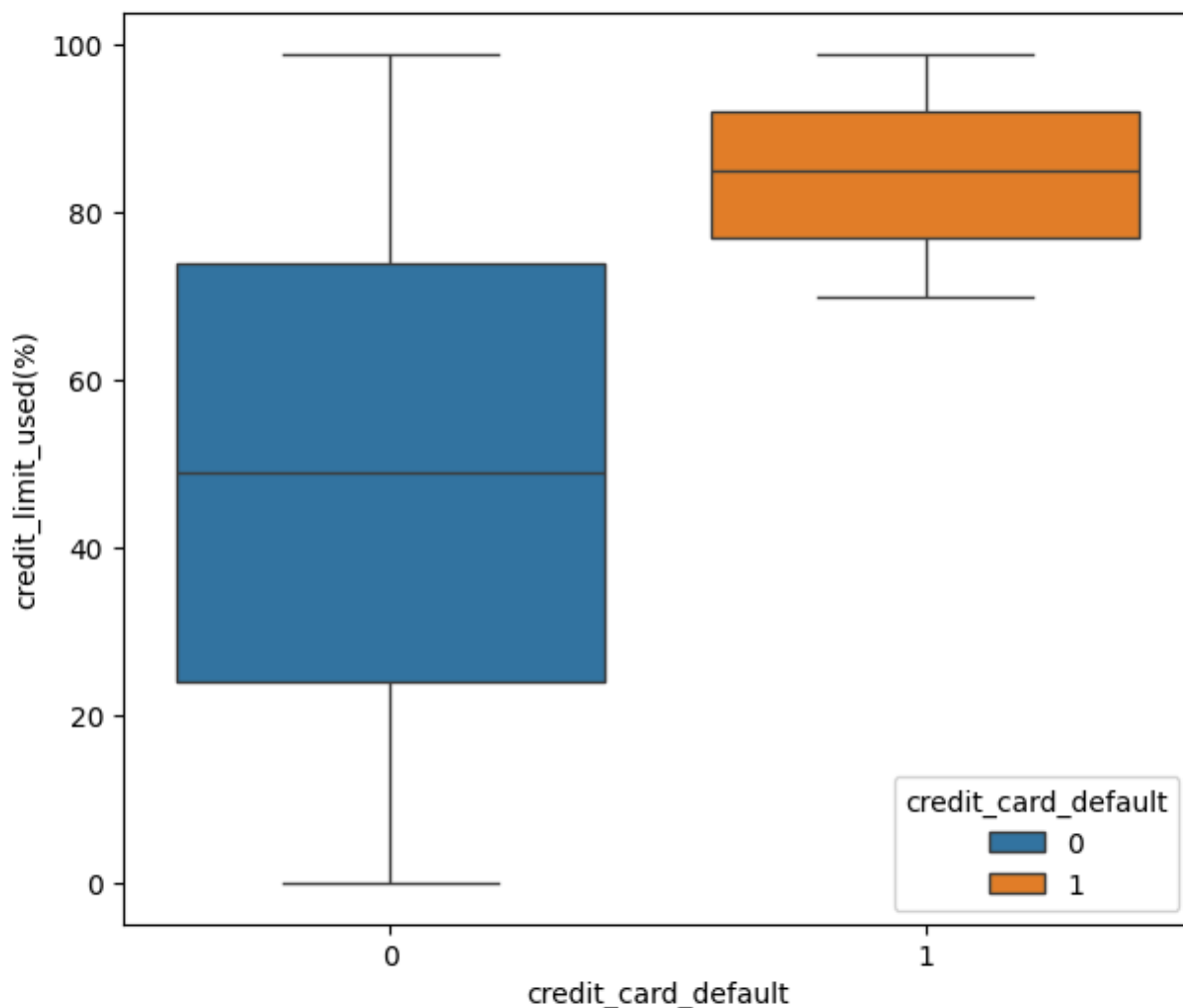


Non-defaulters (class 0) have higher credit scores with a Median credit score ≈ 800 Scores mostly range between 720 – 880 indicating financially reliable customers. Defaulters (Class 1) have lower credit scores with a Median credit score ≈ 600 Scores mostly range between 550 – 650 suggesting a higher financial risk. Therefore Lower credit score $\rightarrow$ Higher default probability
Higher credit score $\rightarrow$ Lower default probability

In []:

In [52]: *# boxplot of credit_defaulter-credit_limit_utilization*

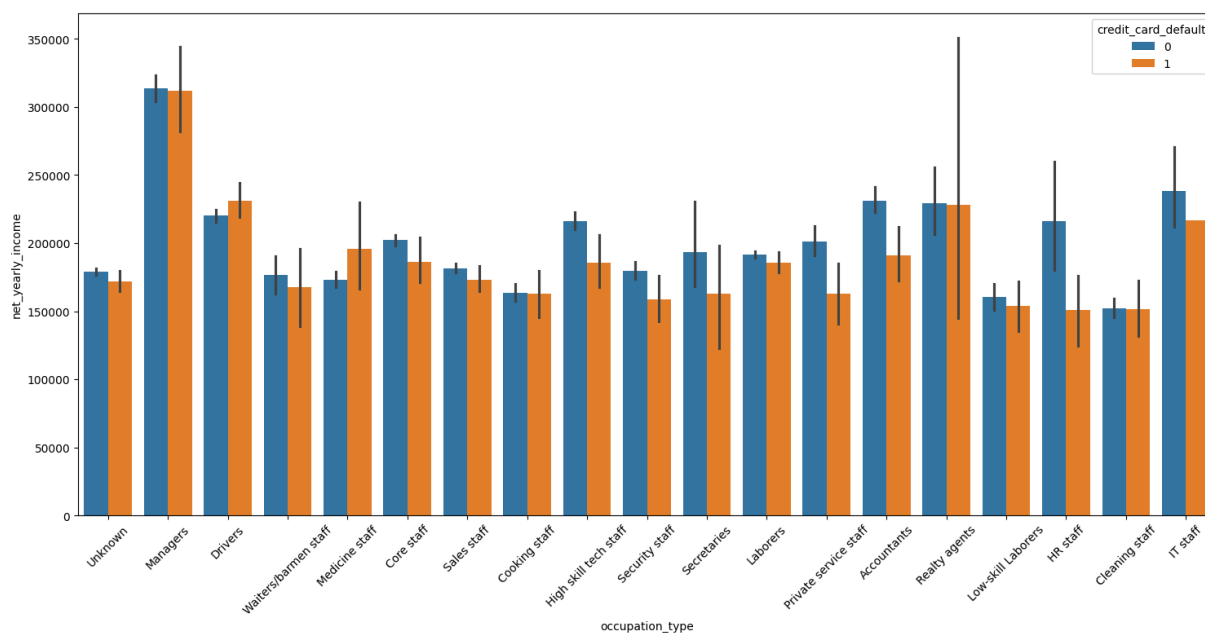
```
plt.figure(figsize=[7,6])
sns.boxplot(x=df['credit_card_default'],y=df['credit_limit_used(%)'],hue=df['credit
plt.show()
```



-Defaulters box (class 1) is positioned significantly higher on the y-axis compared to Non-defaulters(class 0) indicating that the individuals who default uses higher percentage of their credit limit. -Defaulters show median of 85% which is quite high compared to the median of Non-defaulters 50%. -The box plot reflects a strong positive correlation between credit limit utilization and default risk. -Higher credit limit utilization is one of the key factors for predicting the credit card defaults.

In []:

```
In [53]: # clustered horizontal barplots for occupation types wrt 'net_yearly_income' and 'c
plt.figure(figsize=[18,8])
sns.barplot(x=df['occupation_type'], y=df['net_yearly_income'], hue=df['credit_card
plt.xticks(rotation=45)
plt.show()
```



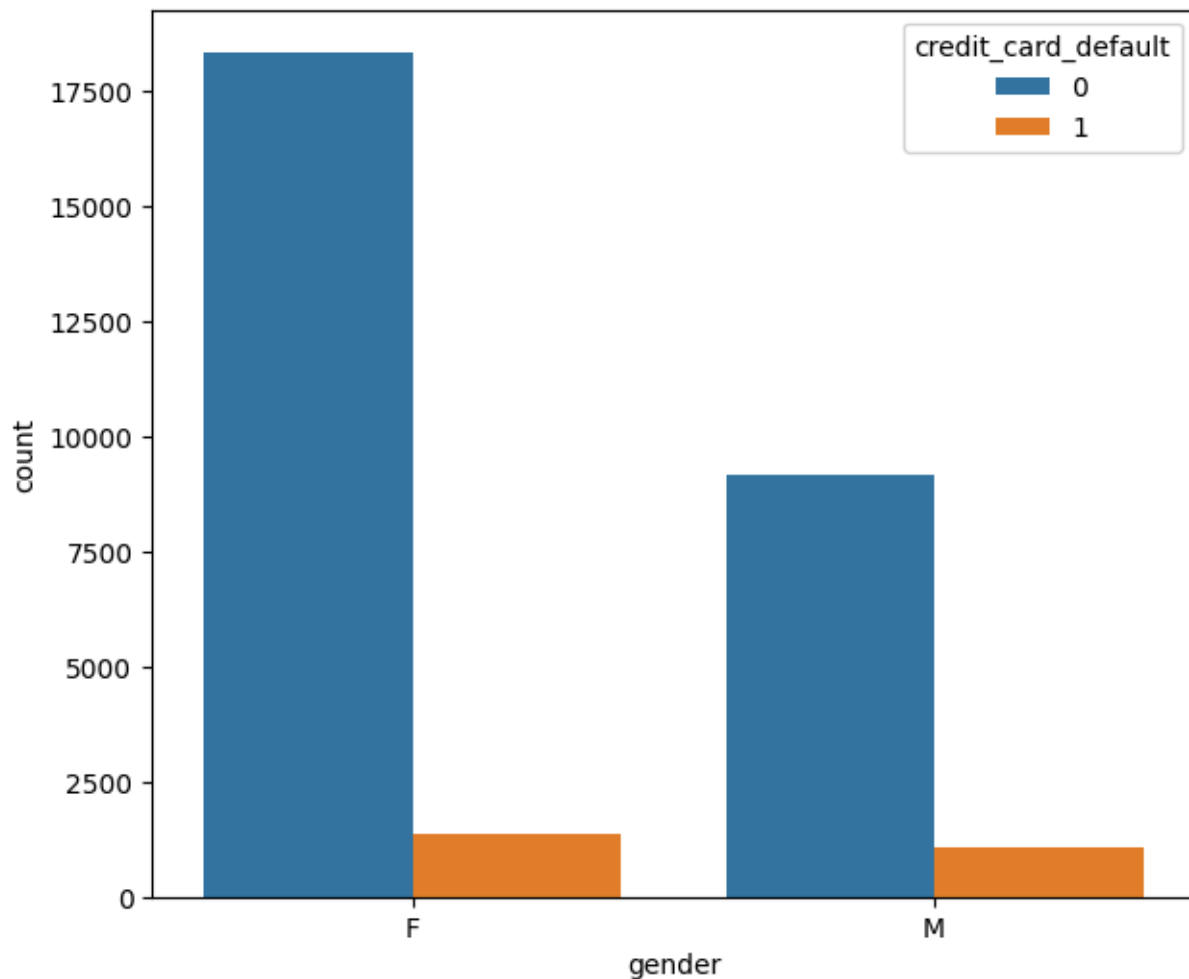
Income Variation - People with Occupations like Managers, RealtyAgents, IT staff, Drivers show a high income ranging from Rs 2,00,000 to Rs 3,25,000. While Cleaning staff, Hr staff, cooking staff reflect a low income people. With occupation such as Managers, RealtyAgents, IT staff and Drivers, the average income for those who defaulted is slightly less from those didn't. High earned annual income reflects less default rate of EMI's. Error bars (thin black lines at the end of each bar) - long error bars indicate high variability/small sample size/avg value less reliable short error bars - avg income is more concentrated and consistent. Therefore, borrower's (low) income level and occupation_type make them a high risk candidate for a credit card.

In []:

In [54]:

```
# male or female defaulters

plt.figure(figsize=[7,6])
sns.countplot(x=df['gender'], hue=df['credit_card_default'])
plt.show()
```



The above barchart illustrates a class imbalance comparing the count of individual who are non defaulters (class 0-blue) and defaulters(class 1-orange) across the gender category - male and female.

In []:

In [55]: *# boxplot of all continuous variables*

```
plt.figure(figsize=[16, 12])
plt.subplot(2,3,1)
df['net_yearly_income'].plot.box()
plt.title('net_yearly_income Box Plot')

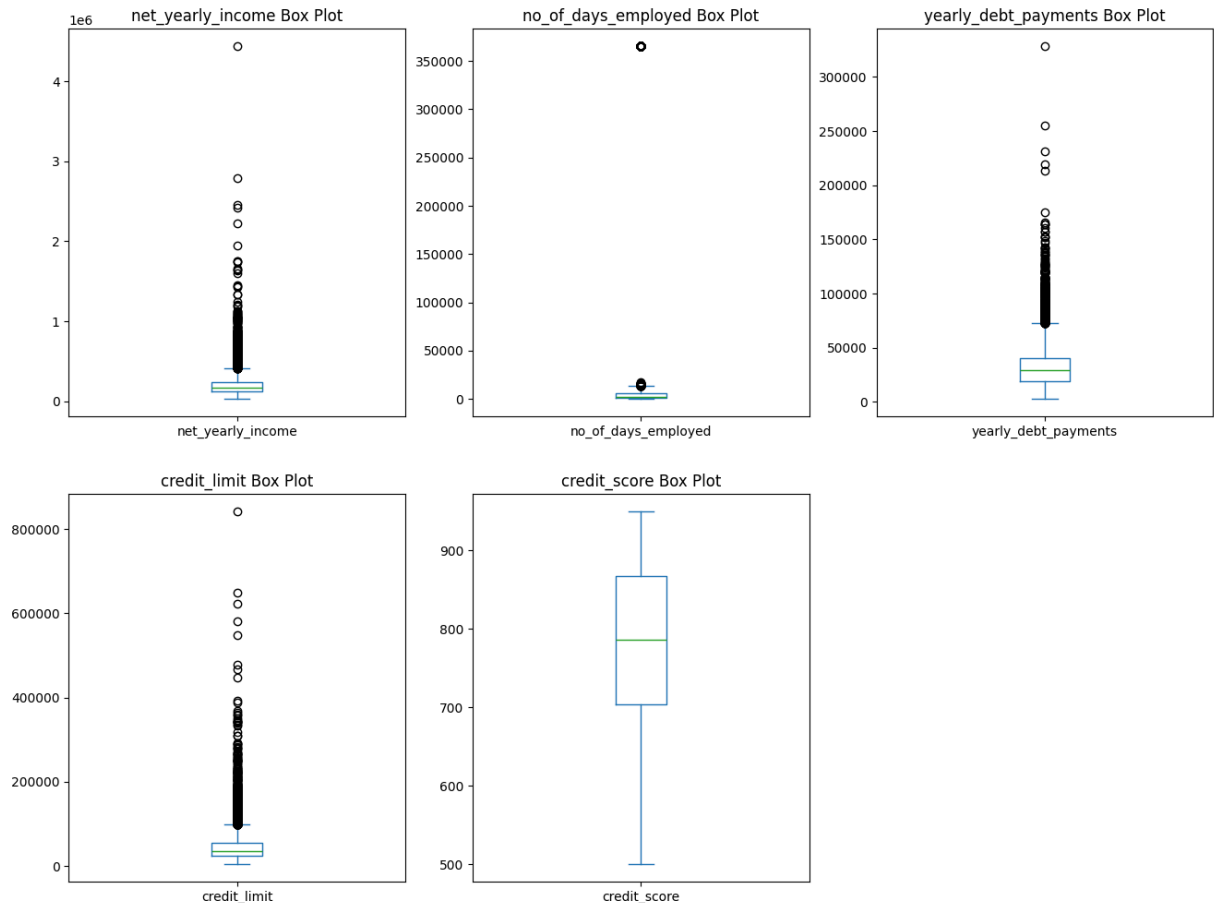
plt.subplot(2,3,2)
df['no_of_days_employed'].plot.box()
plt.title('no_of_days_employed Box Plot')

plt.subplot(2,3,3)
df['yearly_debt_payments'].plot.box()
plt.title('yearly_debt_payments Box Plot')

plt.subplot(2,3,4)
df['credit_limit'].plot.box()
plt.title('credit_limit Box Plot')

plt.subplot(2,3,5)
df['credit_score'].plot.box()
```

```
plt.title('credit_score Box Plot')
plt.show()
```



Net_yearly_incomeBox Plot - most of the data is concentrated at the lower end of the scale(0). There exists a significant amount of outliers. Data is highly right skewed. no_of_days - most of the data is concentrated between 0 to 50000 days. There is a massive outlier near (350,000 days roughly - 958 years). yearly_debt_payments - Bulk debt payments fall below 50,000. There are several outliers ranging from 100,000 to 300,000. credit_limitbox - most of the data is concentrated at lower end (under 100,000). There are numerous outliers extending upto 800,000 credit_score_boxplot -this is more normally distributed and balanced with a median around 750. No outliers exist. The above variables are heavily skewed by outliers. Scaling or capping of these outliers is necessary before building the ML Model

In []:

Data Processings

WOE Binning (grouping categorical variables into discrete categories and replacing them with a calculated WOE score)

```
In [56]: selected_vars = ['customer_id', 'name', 'age', 'gender', 'owns_car', 'owns_house',
                          'no_of_children', 'net_yearly_income', 'no_of_days_employed', 'oc
                          'total_family_members', 'migrant_worker', 'yearly_debt_payments',
                          'credit_limit_used(%)', 'credit_score', 'prev_defaults',
                          'default_in_last_6months', 'credit_card_default']
df_woe = df[selected_vars]
```

```
In [57]: print(df_woe.dtypes)
```

```

customer_id      object
name             object
age             int64
gender          object
owns_car        object
owns_house      object
no_of_children   float64
net_yearly_income float64
no_of_days_employed float64
occupation_type  object
total_family_members float64
migrant_worker   float64
yearly_debt_payments float64
credit_limit     float64
credit_limit_used(%) int64
credit_score     float64
prev_defaults    int64
default_in_last_6months int64
credit_card_default int64
dtype: object

```

```
In [58]: df_woe = df[selected_vars].copy()
```

```
In [59]: # convert categorical columns explicitly

df_woe['occupation_type'] = df_woe['occupation_type'].astype('category')
df_woe['gender'] = df_woe['gender'].astype('category')
```

```
In [60]: df = df_woe.drop(columns=['customer_id', 'name'])
```

```
In [61]: import warnings
warnings.filterwarnings("ignore", category=FutureWarning)
```

```
In [62]: bins = sc.woebin(df, y='credit_card_default')
```

```
[INFO] creating woe binning ...
```

```
C:\Users\Home\anaconda3\Lib\site-packages\scorecardpy\condition_fun.py:40: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consistent and as-expected, please specify a format.
```

```
datetime_cols = dat.apply(pd.to_numeric,errors='ignore').select_dtypes(object).apply(pd.to_datetime,errors='ignore').select_dtypes('datetime64').columns.tolist()
```

```
C:\Users\Home\anaconda3\Lib\site-packages\scorecardpy\condition_fun.py:40: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consistent and as-expected, please specify a format.
```

```
datetime_cols = dat.apply(pd.to_numeric,errors='ignore').select_dtypes(object).apply(pd.to_datetime,errors='ignore').select_dtypes('datetime64').columns.tolist()
```

```
Binning on 30000 rows and 17 columns in 00:00:12
```

```
In [63]: print(type(bins))
```

```
<class 'dict'>
```

```
In [64]: print(len(bins))
```

16

```
In [65]: for var in bins.keys():  
         print("\n", "="*50)  
         print(var)  
         print(bins[var])
```



```

=====
migrant_worker
      variable      bin  count  count_distr  good  bad  badprob  \
0  migrant_worker  [-inf,1.0)  24648      0.8216  22725  1923  0.078019
1  migrant_worker  [1.0,inf)   5352      0.1784   4812   540  0.100897

      woe    bin_iv  total_iv  breaks  is_special_values
0 -0.055429  0.002466  0.012561    1.0             False
1  0.226851  0.010094  0.012561    inf             False

=====
no_of_days_employed
      variable      bin  count  count_distr  good  bad  \
0  no_of_days_employed  [-inf,1000.0)  7943      0.264767  7027  916
1  no_of_days_employed  [1000.0,3000.0)  9933      0.331100  9036  897
2  no_of_days_employed  [3000.0,4000.0)  2442      0.081400  2273  169
3  no_of_days_employed  [4000.0,inf)   9682      0.322733  9201  481

      badprob    woe    bin_iv  total_iv  breaks  is_special_values
0  0.115322  0.376652  0.043963  0.12486  1000.0             False
1  0.090305  0.104234  0.003758  0.12486  3000.0             False
2  0.069206 -0.184807  0.002574  0.12486  4000.0             False
3  0.049680 -0.537050  0.074565  0.12486    inf             False

=====
owns_house
      variable  bin  count  count_distr  good  bad  badprob    woe  \
0  owns_house   Y  20886      0.6962  19180  1706  0.081682 -0.005566
1  owns_house   N   9114      0.3038   8357   757  0.083059  0.012659

      bin_iv  total_iv  breaks  is_special_values
0  0.000022  0.00007    Y             False
1  0.000049  0.00007    N             False

=====
gender
      variable  bin  count  count_distr  good  bad  badprob    woe  bin_iv  \
0  gender      F  19731      0.6577  18353  1378  0.069839 -0.175009  0.018727
1  gender      M  10269      0.3423   9184  1085  0.105658  0.278268  0.029776

      total_iv  breaks  is_special_values
0  0.048503    F             False
1  0.048503    M             False

=====
yearly_debt_payments
      variable      bin  count  count_distr  good  bad  \
0  yearly_debt_payments  [-inf,18000.0)  6686      0.222867  6227  459
1  yearly_debt_payments  [18000.0,46000.0)  18067      0.602233  16396  1671
2  yearly_debt_payments  [46000.0,56000.0)  2703      0.090100  2518  185
3  yearly_debt_payments  [56000.0,inf)   2544      0.084800  2396  148

      badprob    woe    bin_iv  total_iv  breaks  is_special_values
0  0.068651 -0.193449  0.007694  0.03171  18000.0             False
1  0.092489  0.130535  0.010838  0.03171  46000.0             False
2  0.068442 -0.196714  0.003212  0.03171  56000.0             False

```

3 0.058176 -0.370193 0.009966 0.03171 inf False

=====

net_yearly_income

	variable	bin	count	count_distr	good	bad	\
0	net_yearly_income	[-inf,80000.0)	1728	0.057600	1602	126	
1	net_yearly_income	[80000.0,200000.0)	16777	0.559233	15282	1495	
2	net_yearly_income	[200000.0,240000.0)	3913	0.130433	3587	326	
3	net_yearly_income	[240000.0,320000.0)	4459	0.148633	4135	324	
4	net_yearly_income	[320000.0,390000.0)	1574	0.052467	1493	81	
5	net_yearly_income	[390000.0,inf)	1549	0.051633	1438	111	

	badprob	woe	bin_iv	total_iv	breaks	is_special_values
0	0.072917	-0.128576	0.000902	0.019779	80000.0	False
1	0.089110	0.089601	0.004661	0.019779	200000.0	False
2	0.083312	0.015976	0.000034	0.019779	240000.0	False
3	0.072662	-0.132349	0.002464	0.019779	320000.0	False
4	0.051461	-0.499943	0.010664	0.019779	390000.0	False
5	0.071659	-0.147328	0.001054	0.019779	inf	False

=====

credit_limit_used(%)

	variable	bin	count	count_distr	good	bad	\
0	credit_limit_used(%)	[-inf,72.0)	19993	0.666433	19850	143	
1	credit_limit_used(%)	[72.0,92.0)	7192	0.239733	5551	1641	
2	credit_limit_used(%)	[92.0,inf)	2815	0.093833	2136	679	

	badprob	woe	bin_iv	total_iv	breaks	is_special_values
0	0.007153	-2.518964	1.669542	2.476275	72.0	False
1	0.228170	1.195478	0.555512	2.476275	92.0	False
2	0.241208	1.268081	0.251222	2.476275	inf	False

=====

credit_limit

	variable	bin	count	count_distr	good	bad	badprob	\
0	credit_limit	[-inf,20000.0)	4989	0.166300	4557	432	0.086590	
1	credit_limit	[20000.0,50000.0)	16474	0.549133	15097	1377	0.083586	
2	credit_limit	[50000.0,58000.0)	2344	0.078133	2124	220	0.093857	
3	credit_limit	[58000.0,68000.0)	2067	0.068900	1919	148	0.071601	
4	credit_limit	[68000.0,86000.0)	2089	0.069633	1963	126	0.060316	
5	credit_limit	[86000.0,inf)	2037	0.067900	1877	160	0.078547	

	woe	bin_iv	total_iv	breaks	is_special_values
0	0.058156	0.000576	0.010832	20000.0	False
1	0.019562	0.000212	0.010832	50000.0	False
2	0.146722	0.001788	0.010832	58000.0	False
3	-0.148197	0.001423	0.010832	68000.0	False
4	-0.331797	0.006679	0.010832	86000.0	False
5	-0.048106	0.000154	0.010832	inf	False

=====

credit_score

	variable	bin	count	count_distr	good	bad	badprob	\
0	credit_score	[-inf,660.0)	2896	0.096533	894	2002	0.691298	
1	credit_score	[660.0,690.0)	3176	0.105867	2838	338	0.106423	
2	credit_score	[690.0,inf)	23928	0.797600	23805	123	0.005140	

	woe	bin_iv	total_iv	breaks	is_special_values
0	3.220347	2.513044	4.845323	660.0	False
1	0.286341	0.009784	4.845323	690.0	False
2	-2.851316	2.322494	4.845323	inf	False

```
=====
default_in_last_6months
      variable      bin count count_distr good bad \
0 default_in_last_6months [-inf,inf) 30000      1.0 27537 2463
```

	badprob	woe	bin_iv	total_iv	breaks	is_special_values
0	0.0821	0.0	0.0	0.0	inf	False

```
=====
total_family_members
      variable      bin count count_distr good bad \
0 total_family_members [-inf,2.0) 6544      0.218133 5976 568
1 total_family_members [2.0,3.0) 15467      0.515567 14278 1189
2 total_family_members [3.0,4.0) 5170      0.172333 4717 453
3 total_family_members [4.0,inf) 2819      0.093967 2566 253
```

	badprob	woe	bin_iv	total_iv	breaks	is_special_values
0	0.086797	0.060765	0.000826	0.005208	2.0	False
1	0.076873	-0.071457	0.002555	0.005208	3.0	False
2	0.087621	0.071114	0.000898	0.005208	4.0	False
3	0.089748	0.097436	0.000929	0.005208	inf	False

```
=====
age
      variable      bin count count_distr good bad badprob woe \
0 age [-inf,25.0) 1838      0.061267 1672 166 0.090316 0.104362
1 age [25.0,32.0) 6292      0.209733 5787 505 0.080261 -0.024660
2 age [32.0,38.0) 5548      0.184933 5070 478 0.086157 0.052665
3 age [38.0,40.0) 1748      0.058267 1629 119 0.068078 -0.202448
4 age [40.0,42.0) 1824      0.060800 1652 172 0.094298 0.151903
5 age [42.0,46.0) 3548      0.118267 3288 260 0.073281 -0.123203
6 age [46.0,53.0) 6385      0.212833 5863 522 0.081754 -0.004599
7 age [53.0,inf) 2817      0.093900 2576 241 0.085552 0.044954
```

	bin_iv	total_iv	breaks	is_special_values
0	0.000697	0.006941	25.0	False
1	0.000126	0.006941	32.0	False
2	0.000524	0.006941	38.0	False
3	0.002195	0.006941	40.0	False
4	0.001495	0.006941	42.0	False
5	0.001705	0.006941	46.0	False
6	0.000004	0.006941	53.0	False
7	0.000193	0.006941	inf	False

```
=====
occupation_type
      variable \
0 occupation_type
1 occupation_type
2 occupation_type
```

```

3  occupation_type
4  occupation_type
5  occupation_type

```

```

bin \
0          Accountants%,%Cleaning staff%,%Cooking s
taff
1          Core s
taff
2  Drivers%,%HR staff%,%High skill tech staff%,%IT staff%,%Laborers%,%Low-skill Labo
rers
3          Managers%,%Medicine s
taff
4  Private service staff%,%Realty agents%,%Sales staff%,%Secretaries%,%Security s
taff
5          Unknown%,%Waiters/barmen s
taff

```

	count	count_distr	good	bad	badprob	woe	bin_iv	total_iv \
0	1983	0.066100	1809	174	0.087746	0.072676	0.000360	0.067108
1	2697	0.089900	2540	157	0.058213	-0.369523	0.010530	0.067108
2	8626	0.287533	7703	923	0.107002	0.292415	0.027783	0.067108
3	2913	0.097100	2708	205	0.070374	-0.166805	0.002520	0.067108
4	4208	0.140267	3810	398	0.094582	0.155218	0.003606	0.067108
5	9573	0.319100	8967	606	0.063303	-0.280276	0.022308	0.067108

br

```

eaks \
0          Accountants%,%Cleaning staff%,%Cooking s
taff
1          Core s
taff
2  Drivers%,%HR staff%,%High skill tech staff%,%IT staff%,%Laborers%,%Low-skill Labo
rers
3          Managers%,%Medicine s
taff
4  Private service staff%,%Realty agents%,%Sales staff%,%Secretaries%,%Security s
taff
5          Unknown%,%Waiters/barmen s
taff

```

```

is_special_values
0          False
1          False
2          False
3          False
4          False
5          False

```

```

=====
owns_car
variable bin  count  count_distr  good  bad  badprob  woe \
0  owns_car  Y  10049    0.334967  9295  754  0.075032 -0.097689
1  owns_car  N  19951    0.665033  18242 1709  0.085660  0.046332

```

	bin_iv	total_iv	breaks	is_special_values
0	0.003069	0.004524	Y	False
1	0.001456	0.004524	N	False

```
=====
```

no_of_children								
	variable	bin	count	count_distr	good	bad	badprob	\
0	no_of_children	[-inf,1.0)	21044	0.701467	19402	1642	0.078027	
1	no_of_children	[1.0,2.0)	5966	0.198867	5412	554	0.092860	
2	no_of_children	[2.0,inf)	2990	0.099667	2723	267	0.089298	

	woe	bin_iv	total_iv	breaks	is_special_values
0	-0.055311	0.002097	0.006803	1.0	False
1	0.134941	0.003831	0.006803	2.0	False
2	0.091910	0.000875	0.006803	inf	False

```
=====
```

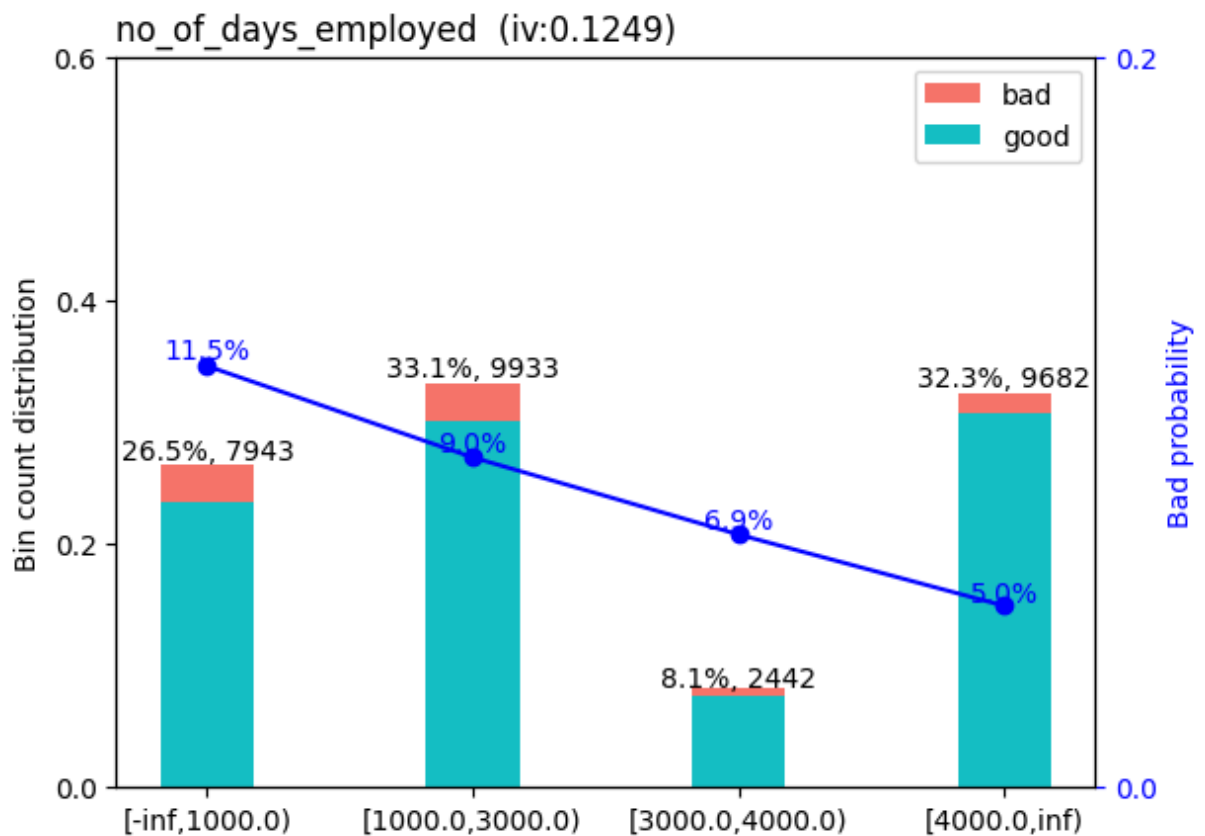
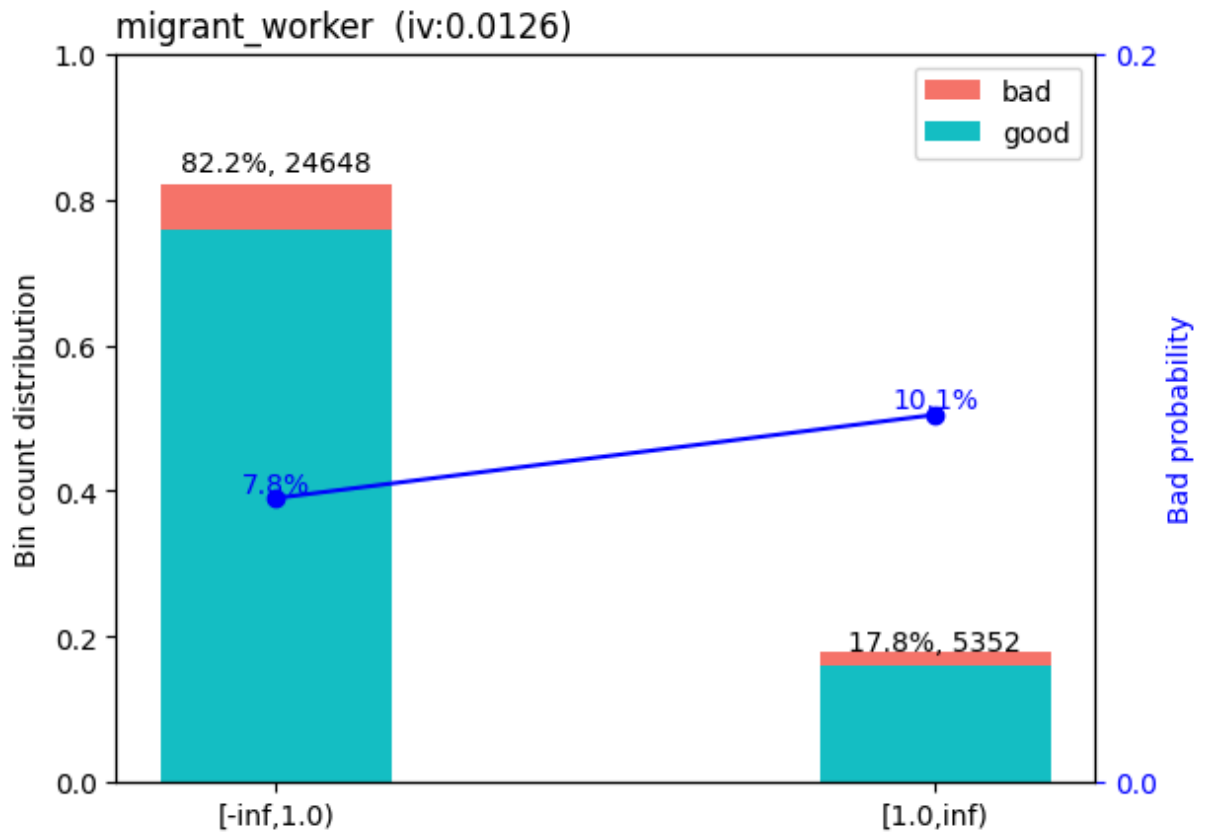
prev_defaults								
	variable	bin	count	count_distr	good	bad	badprob	woe \
0	prev_defaults	[-inf,inf)	30000	1.0	27537	2463	0.0821	0.0

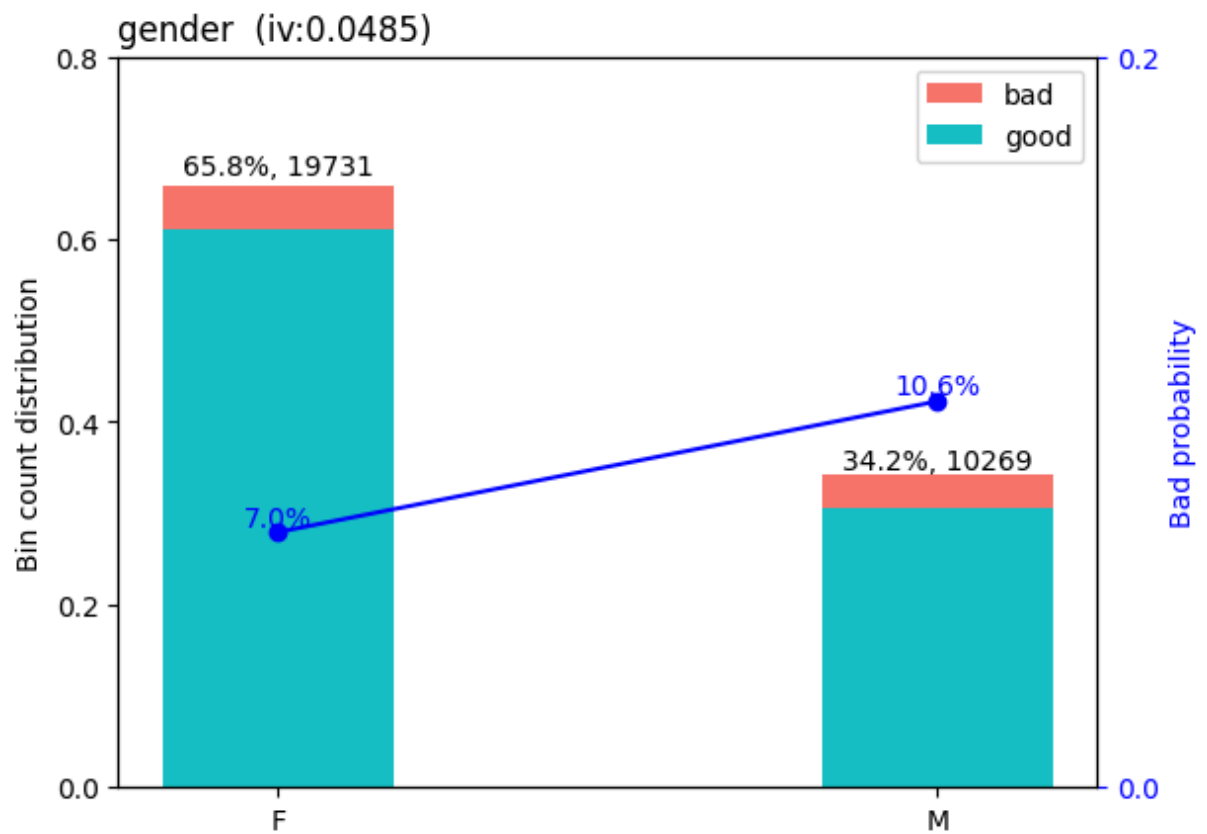
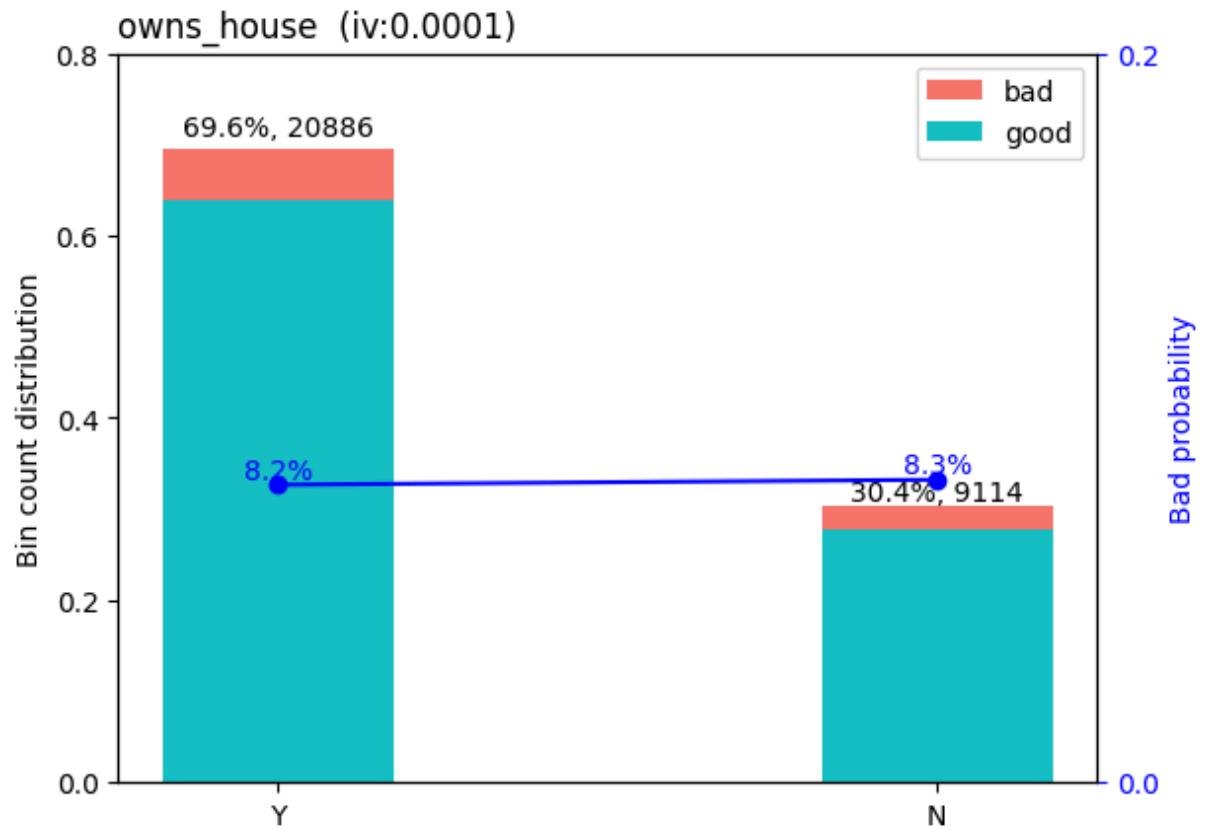
	bin_iv	total_iv	breaks	is_special_values
0	0.0	0.0	inf	False

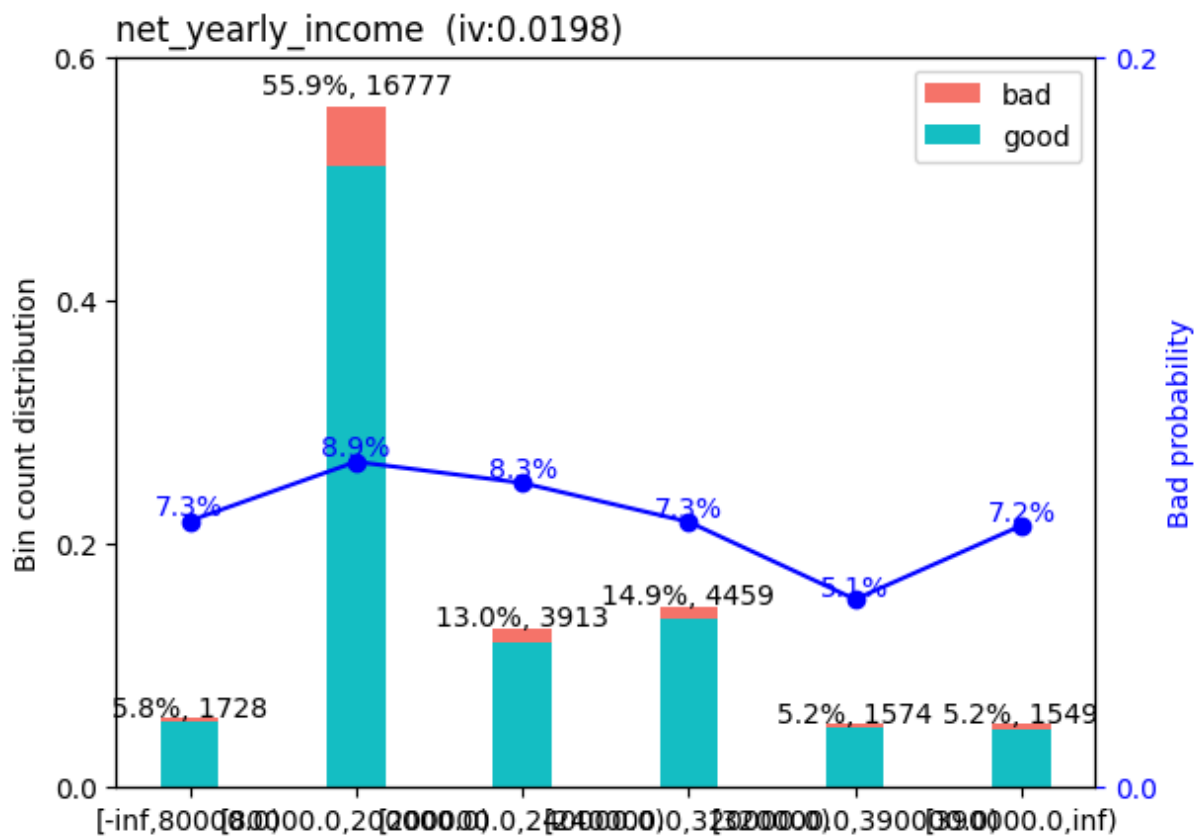
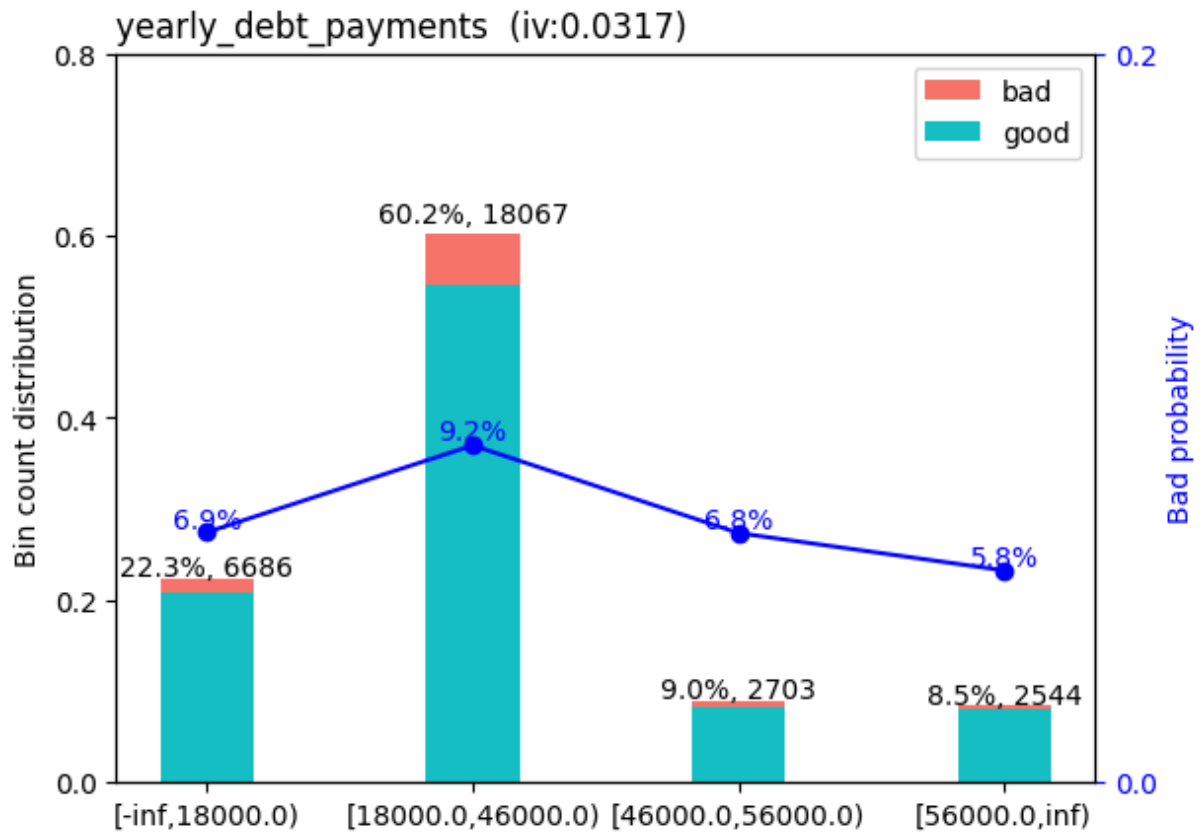
In [66]: *# visualizing the Weight of Evidence (WOE) bins created by woebin().*

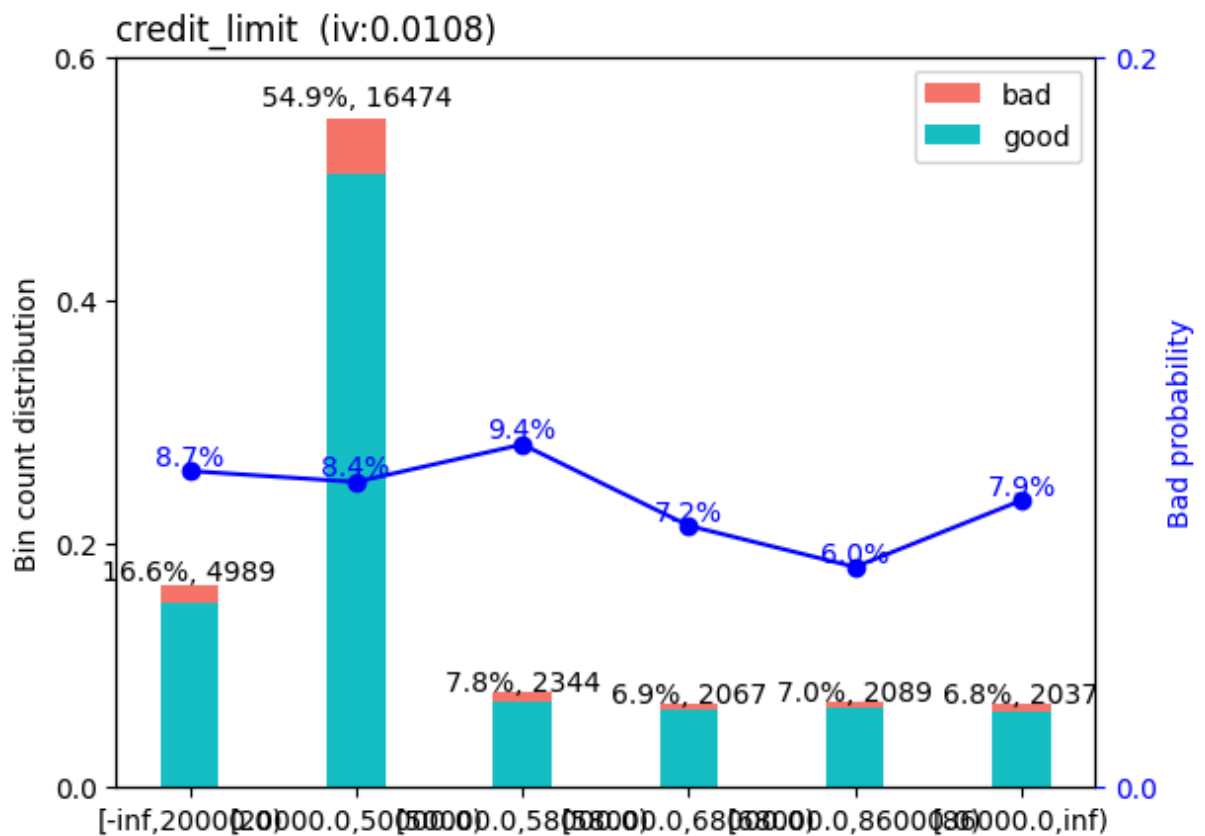
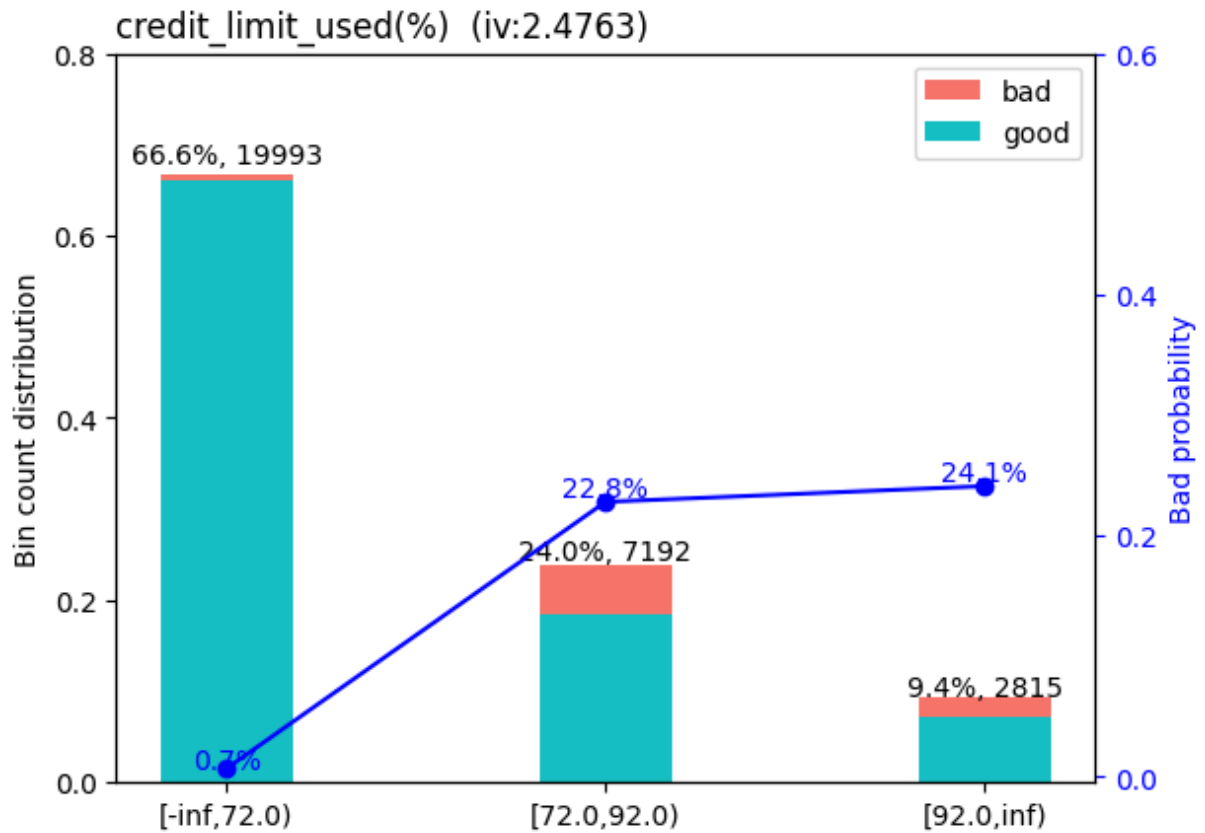
```
sc.woebin_plot(bins)
```

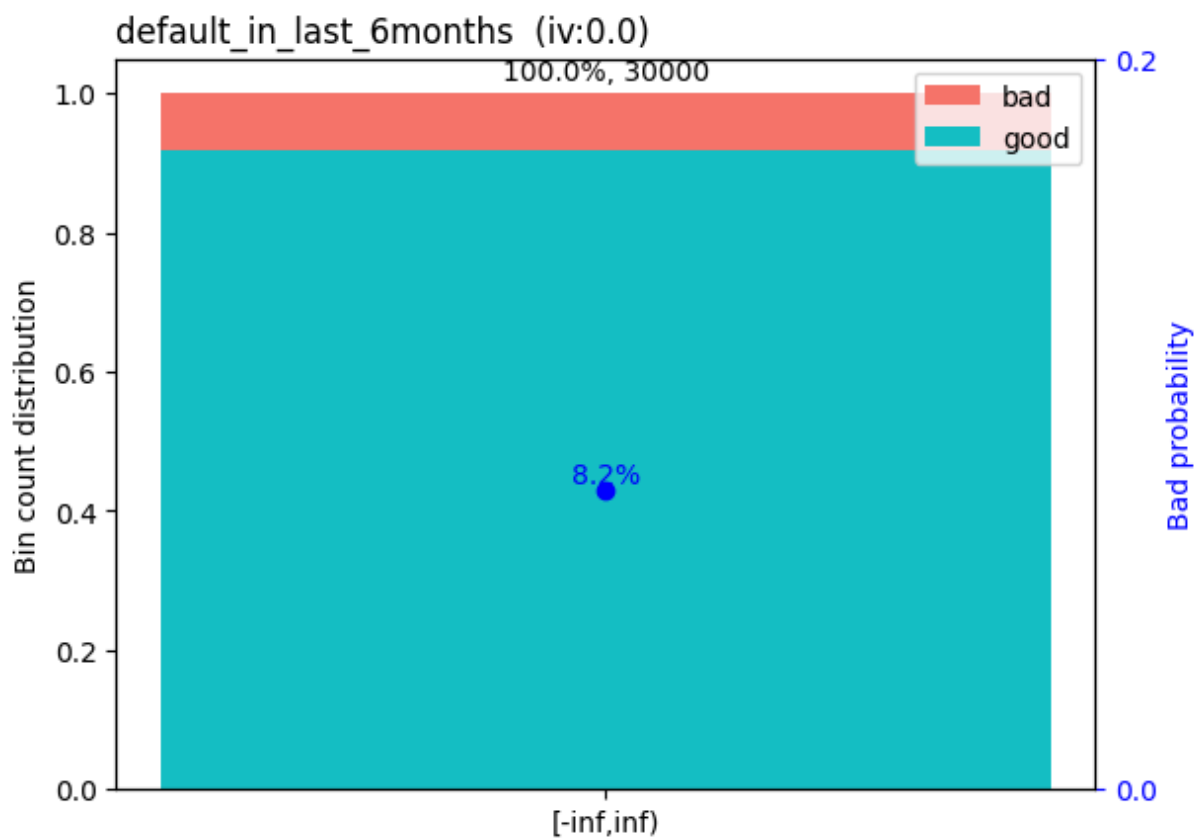
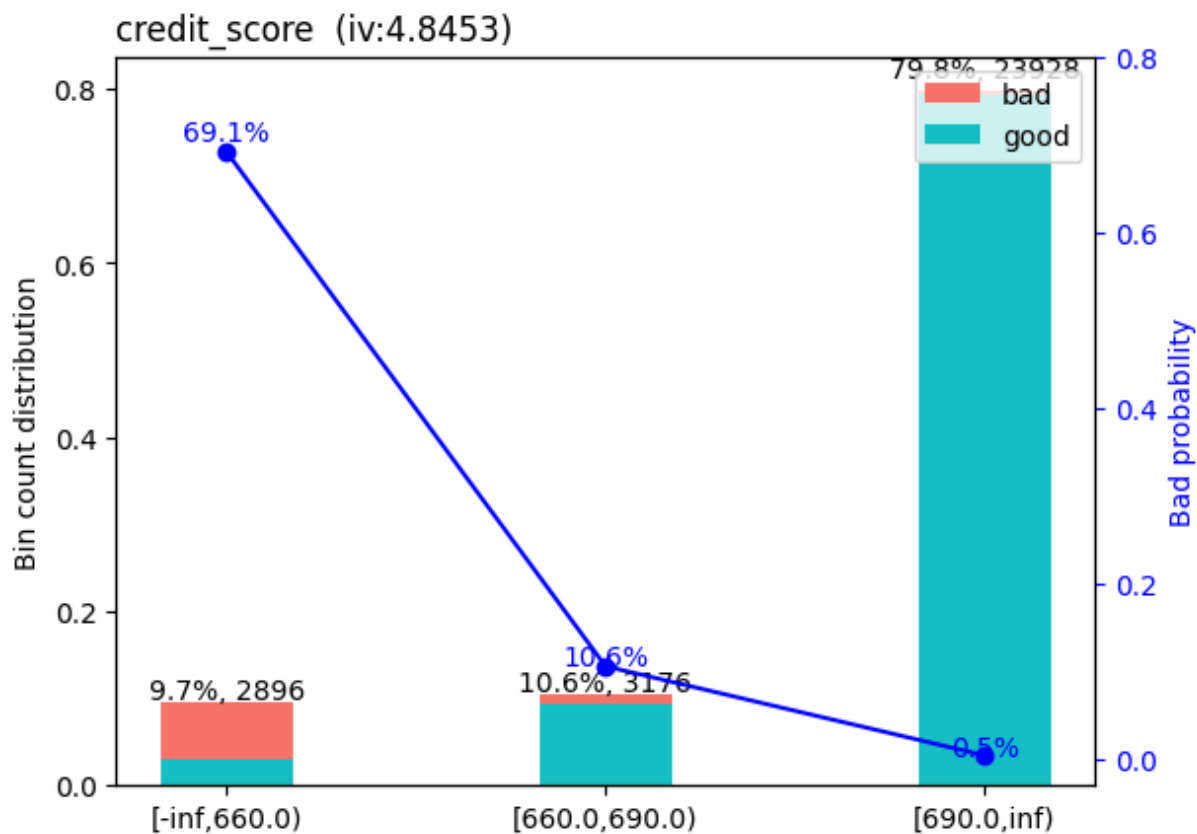
Out[66]: {'migrant_worker': <Figure size 640x480 with 2 Axes>,
 'no_of_days_employed': <Figure size 640x480 with 2 Axes>,
 'owns_house': <Figure size 640x480 with 2 Axes>,
 'gender': <Figure size 640x480 with 2 Axes>,
 'yearly_debt_payments': <Figure size 640x480 with 2 Axes>,
 'net_yearly_income': <Figure size 640x480 with 2 Axes>,
 'credit_limit_used(%)': <Figure size 640x480 with 2 Axes>,
 'credit_limit': <Figure size 640x480 with 2 Axes>,
 'credit_score': <Figure size 640x480 with 2 Axes>,
 'default_in_last_6months': <Figure size 640x480 with 2 Axes>,
 'total_family_members': <Figure size 640x480 with 2 Axes>,
 'age': <Figure size 640x480 with 2 Axes>,
 'occupation_type': <Figure size 640x480 with 2 Axes>,
 'owns_car': <Figure size 640x480 with 2 Axes>,
 'no_of_children': <Figure size 640x480 with 2 Axes>,
 'prev_defaults': <Figure size 640x480 with 2 Axes>}

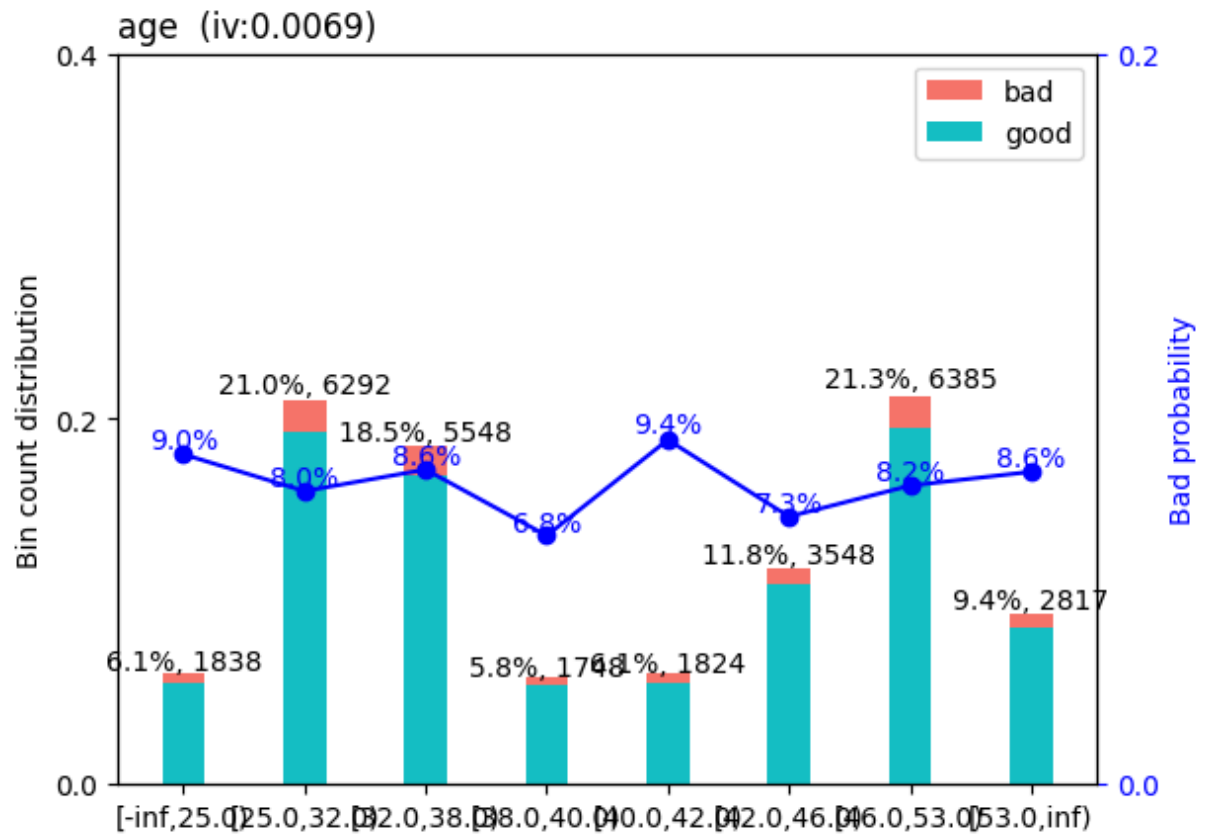
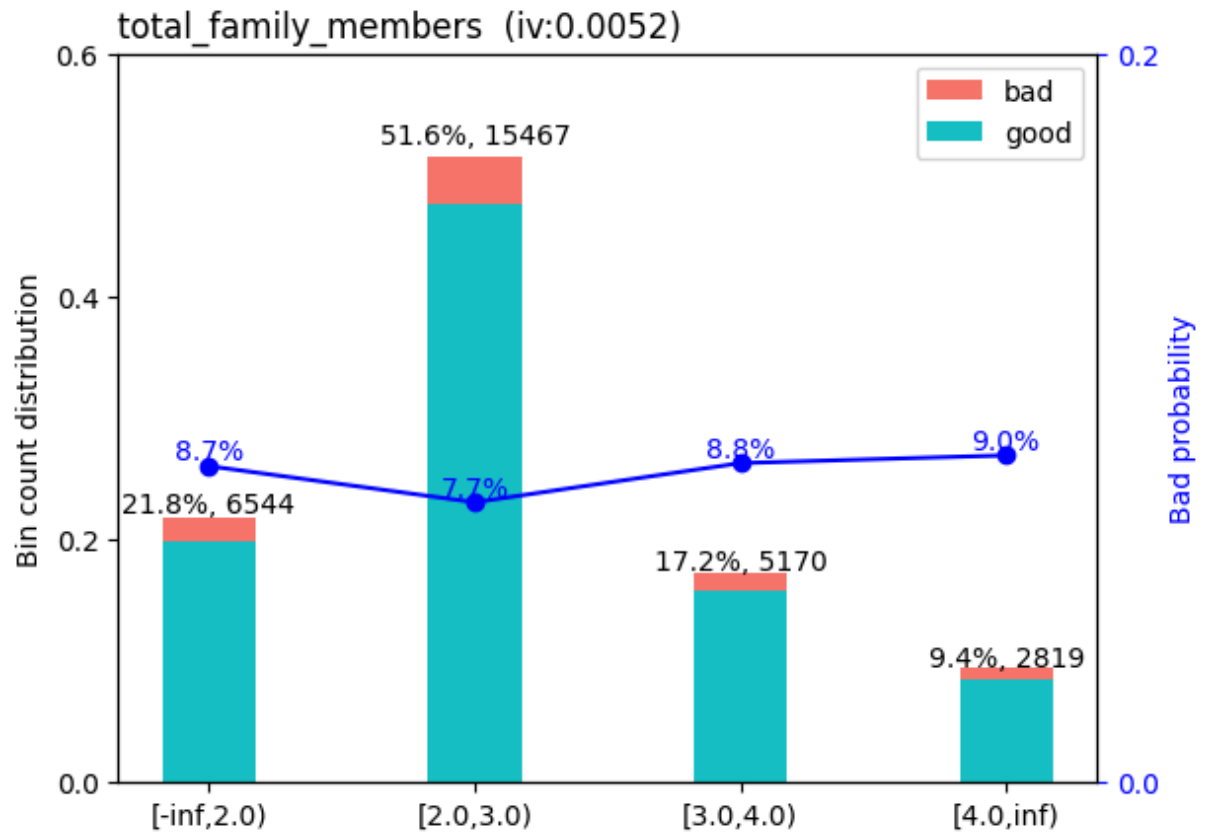


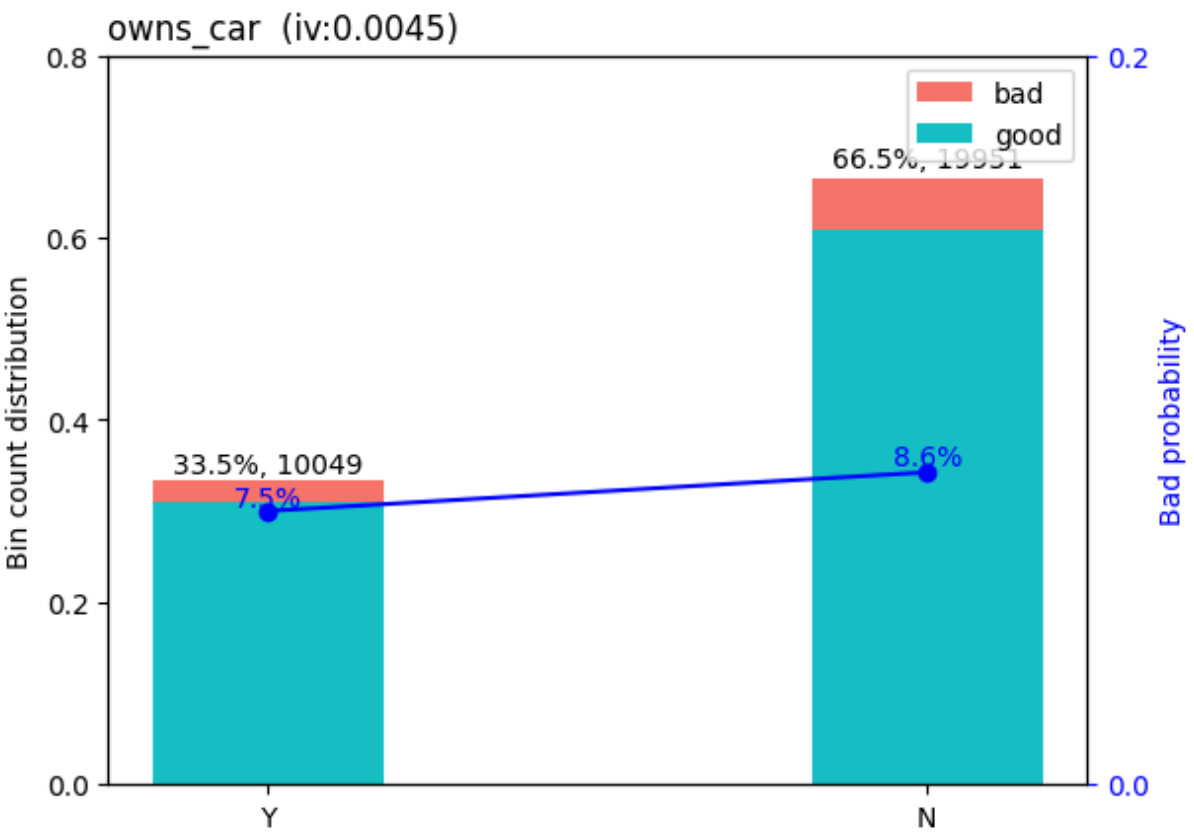
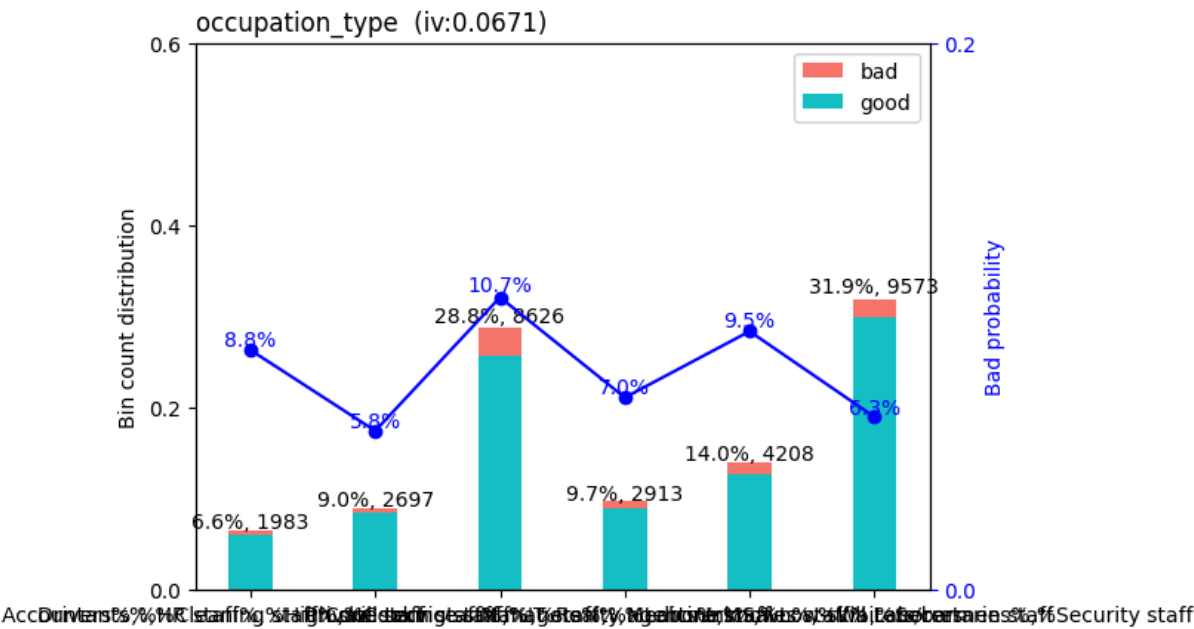


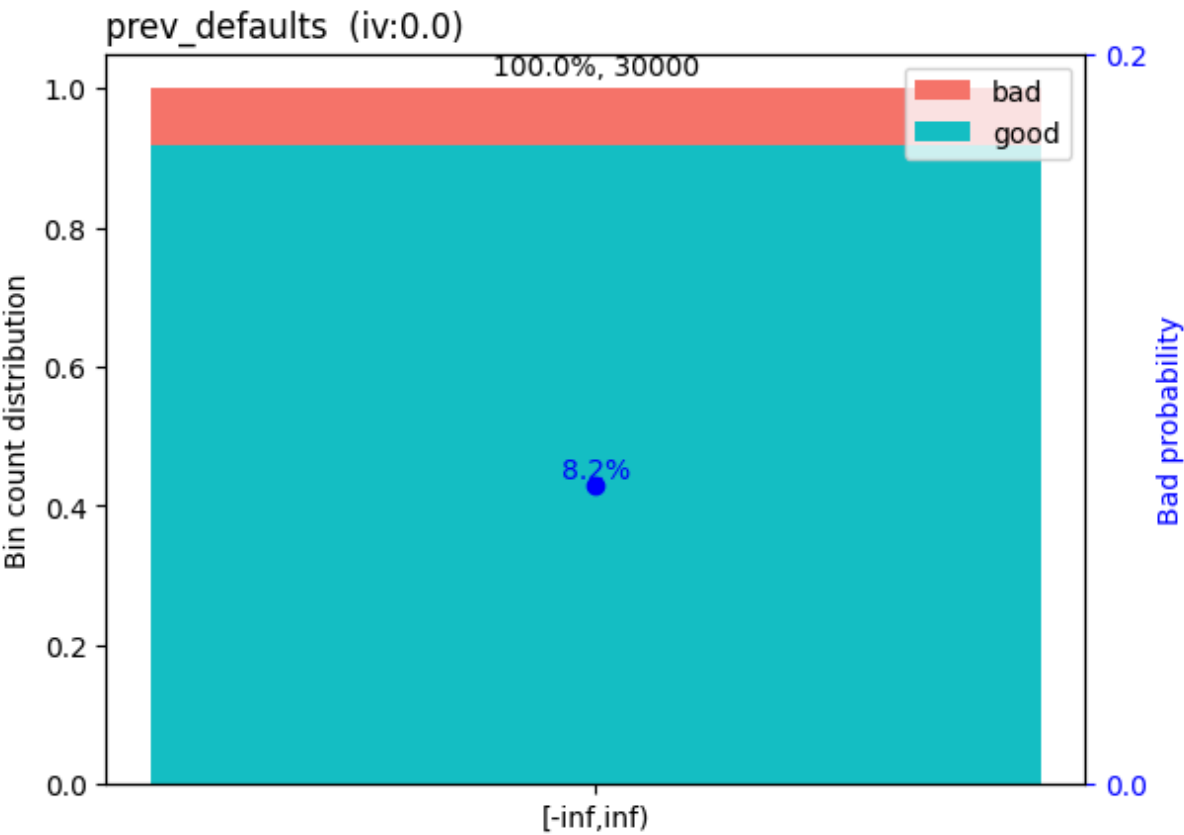
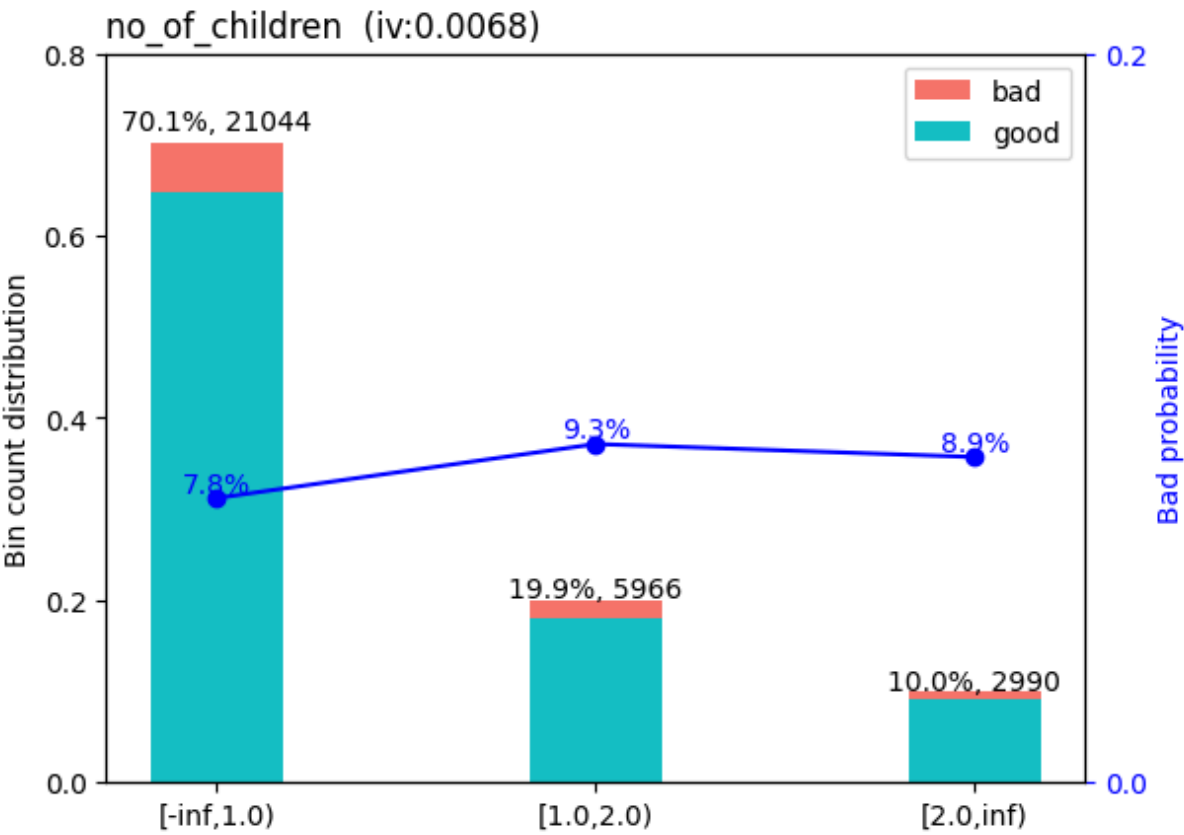












```
In [ ]:
```

Calculating Info_value

In [67]: `import scorecardpy as sc`

In [68]: `# Calculate IV using credit_card_default as target
iv_df = sc.iv(df,y='credit_card_default')

Sort IV values from highest to lowest
iv_df = iv_df.sort_values(by='info_value',ascending=False)

print(iv_df)`

	variable	info_value
15	prev_defaults	6.972746
9	default_in_last_6months	6.712061
8	credit_score	5.166422
6	credit_limit_used(%)	3.079678
1	no_of_days_employed	0.762549
12	occupation_type	0.096320
3	gender	0.048503
11	age	0.015057
4	yearly_debt_payments	0.013054
0	migrant_worker	0.012561
14	no_of_children	0.010090
10	total_family_members	0.008589
7	credit_limit	0.004771
13	owns_car	0.004524
5	net_yearly_income	0.003568
2	owns_house	0.000070

In [69]: `low_iv_cols = iv_df.loc[iv_df['info_value'] < 0.02,'variable'].tolist()

print(low_iv_cols)`

`['age', 'yearly_debt_payments', 'migrant_worker', 'no_of_children', 'total_family_members', 'credit_limit', 'owns_car', 'net_yearly_income', 'owns_house']`

In [70]: `# Dropping Variable with Low(<0.02)info_value

iv_df = iv_df[iv_df['info_value']>=0.02]
print(iv_df)`

	variable	info_value
15	prev_defaults	6.972746
9	default_in_last_6months	6.712061
8	credit_score	5.166422
6	credit_limit_used(%)	3.079678
1	no_of_days_employed	0.762549
12	occupation_type	0.096320
3	gender	0.048503

In []:

In [71]: `## Transforming the original variables into their corresponding WOE (Weight of Evid`

```
train_woe = sc.woebin_ply(df, bins)
train_woe.head(10)
```

[INFO] converting into woe values ...

Out[71]:

	credit_card_default	credit_limit_used(%)_woe	credit_limit_woe	credit_score_woe	mi
34126	0	-2.518964	0.019562	-2.851316	
44609	0	-2.518964	0.019562	-2.851316	
44264	1	1.195478	0.019562	0.286341	
33556	0	-2.518964	0.019562	-2.851316	
28050	0	-2.518964	-0.148197	-2.851316	
4448	0	-2.518964	0.058156	-2.851316	
15054	0	-2.518964	-0.148197	-2.851316	
26095	0	-2.518964	0.058156	-2.851316	
2593	0	-2.518964	0.058156	-2.851316	
297	0	-2.518964	0.146722	-2.851316	

In [72]:

```
# dropping text columns that aren't necessary like 'customer_id' and 'name'

old_df = df.copy()

#df.drop(['customer_id', 'name'], axis=1, inplace=True)
df.drop(low_iv_cols, axis=1, inplace=True)
df.head()
```

Out[72]:

	gender	no_of_days_employed	occupation_type	credit_limit_used(%)	credit_score	mi
34126	F	4181.0	Unknown	23	807.0	
44609	F	365250.0	Unknown	60	858.0	
44264	F	365241.0	Unknown	81	667.0	
33556	F	1882.0	Managers	32	690.0	
28050	M	365244.0	Unknown	65	721.0	

In [73]:

```
df["gender"] = df["gender"].apply(lambda x : 1 if x == "F" else 0)
```

Creating dummies of occupation_type

In [74]:

```
occupation_type_encoded_df = pd.get_dummies(df["occupation_type"], prefix="occupat
df = pd.concat([df, occupation_type_encoded_df], axis=1)
df.drop(columns=["occupation_type"], inplace=True)
df.head()
```

Out[74]:

	gender	no_of_days_employed	credit_limit_used(%)	credit_score	prev_defaults	defi
34126	1	4181.0	23	807.0	0	
44609	1	365250.0	60	858.0	0	
44264	1	365241.0	81	667.0	0	
33556	1	1882.0	32	690.0	0	
28050	0	365244.0	65	721.0	0	

In []:

Splitting Data

In [75]:

```
X = df.drop(['credit_card_default'], axis=1)
y = df['credit_card_default']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, stratify=y

print("X=",X_train.shape, X_test.shape)
print("y=",y_train.shape, y_test.shape)
```

X= (21000, 25) (9000, 25)
y= (21000,) (9000,)

In [76]:

X_train.head()

Out[76]:

	gender	no_of_days_employed	credit_limit_used(%)	credit_score	prev_defaults	defi
26658	1	365246.0	96	922.0	0	
5389	1	1206.0	16	711.0	0	
8704	1	4040.0	18	761.0	0	
37747	0	96.0	51	854.0	0	
27346	0	5326.0	92	700.0	0	

In [77]:

y_train


```
Out[77]: 26658    0
          5389    0
          8704    0
          37747   0
          27346   0
          ..
          34093   0
          823     0
          32715   0
          14006   0
          21152   0
          Name: credit_card_default, Length: 21000, dtype: int64
```

```
In [ ]:
```

```
In [78]: # WOE data

X_woe = train_woe.drop(['credit_card_default'], axis=1)
y_woe = train_woe['credit_card_default']

X_train_woe, X_test_woe, y_train_woe, y_test_woe = train_test_split(X_woe, y_woe, t

print("X=",X_train_woe.shape, X_test_woe.shape)
print("y=",y_train_woe.shape, y_test_woe.shape)

X= (21000, 16) (9000, 16)
y= (21000,) (9000,)
```

```
In [79]: X_train_woe.head()
```

Out[79]:

	credit_limit_used(%)_woe	credit_limit_woe	credit_score_woe	migrant_worker_woe	r
26658	1.268081	0.146722	-2.851316	-0.055429	
5389	-2.518964	0.058156	-2.851316	0.226851	
8704	-2.518964	0.019562	-2.851316	-0.055429	
37747	-2.518964	0.019562	-2.851316	-0.055429	
27346	1.268081	0.146722	-2.851316	-0.055429	

```
In [80]: y_woe
```

```
Out[80]: 34126    0
         44609    0
         44264    1
         33556    0
         28050    0
         ..
         37523    0
         36228    0
         32772    0
         9762     0
         33325    0
Name: credit_card_default, Length: 30000, dtype: int64
```

In []:

Scaling data

```
In [81]: # scaling the features to standardize
scaler = MinMaxScaler()

cols = X_train.columns
X_train[cols] = scaler.fit_transform(X_train[cols])
X_test[cols] = scaler.transform(X_test[cols])
```

In []:

Data Imbalancing Handling

```
In [82]: from imblearn.over_sampling import SMOTE
```

```
In [83]: # initialize SMOTE method

sm = SMOTE(random_state=42)
X_train,y_train = sm.fit_resample(X_train,y_train)
```

```
In [84]: print("Dimension of X_train_sm Shape:", X_train.shape)
print("Dimension of y_train_sm Shape:", y_train.shape)
```

```
Dimension of X_train_sm Shape: (38552, 25)
Dimension of y_train_sm Shape: (38552,)
```

```
In [85]: X_train = pd.DataFrame(data=X_train, columns=cols)
print("X_train.shape:",X_train.shape)
y_train = pd.Series(y_train)
print("y_train.shape:",y_train.shape)
X_train.head()
```

```
X_train.shape: (38552, 25)
y_train.shape: (38552,)
```

Out[85]:

	gender	no_of_days_employed	credit_limit_used(%)	credit_score	prev_defaults	default_i
0	1.0	0.999984	0.969697	0.939866	0.0	
1	1.0	0.003296	0.161616	0.469933	0.0	
2	1.0	0.011055	0.181818	0.581292	0.0	
3	0.0	0.000257	0.515152	0.788419	0.0	
4	0.0	0.014576	0.929293	0.445434	0.0	

In [86]: `y_train.value_counts(normalize=True)`

Out[86]: credit_card_default
 0 0.5
 1 0.5
 Name: proportion, dtype: float64

In []:

Logistic Regression

In [87]: `from statsmodels.discrete.discrete_model import Logit`

```
logit = Logit(y_train, X_train)
logit_fit = logit.fit()
```

Warning: Maximum number of iterations has been exceeded.
 Current function value: 0.347706
 Iterations: 35

C:\Users\Home\anaconda3\Lib\site-packages\statsmodels\base\model.py:607: Convergence
 Warning: Maximum Likelihood optimization failed to converge. Check mle_retvals
 warnings.warn("Maximum Likelihood optimization failed to "

In [88]: `# Detailed statistical breakdown of your logistic regression model`

```
logit_fit.summary()
```

Out[88]:

Logit Regression Results

Dep. Variable:		credit_card_default	No. Observations:		38552		
Model:		Logit	Df Residuals:		38527		
Method:		MLE	Df Model:		24		
Date:		Tue, 16 Jun 2026	Pseudo R-squ.:		0.4984		
Time:		17:39:37	Log-Likelihood:		-13405.		
converged:		False	LL-Null:		-26722.		
Covariance Type:		nonrobust	LLR p-value:		0.000		
		coef	std err	z	P> z 	[0.025	0.975]
	gender	-0.1968	0.072	-2.749	0.006	-0.337	-0.056
	no_of_days_employed	-0.3081	0.115	-2.672	0.008	-0.534	-0.082
	credit_limit_used(%)	9.2943	0.219	42.503	0.000	8.866	9.723
	credit_score	-26.3145	0.615	-42.753	0.000	-27.521	-25.108
	prev_defaults	263.2647	59.808	4.402	0.000	146.043	380.486
	default_in_last_6months	-23.0260	2.813	-8.187	0.000	-28.539	-17.513
	occupation_Accountants	3.8655	0.315	12.278	0.000	3.248	4.483
	occupation_Cleaning staff	4.3373	0.347	12.492	0.000	3.657	5.018
	occupation_Cooking staff	4.5636	0.331	13.781	0.000	3.915	5.213
	occupation_Core staff	4.1199	0.286	14.407	0.000	3.559	4.680
	occupation_Drivers	4.1163	0.283	14.557	0.000	3.562	4.671
	occupation_HR staff	1.1363	1.221	0.930	0.352	-1.258	3.530
	occupation_High skill tech staff	2.7927	0.322	8.664	0.000	2.161	3.424
	occupation_IT staff	-15.9695	2905.906	-0.005	0.996	-5711.441	5679.502
	occupation_Laborers	3.9471	0.271	14.541	0.000	3.415	4.479
	occupation_Low-skill Laborers	3.9467	0.409	9.657	0.000	3.146	4.748
	occupation_Managers	3.5840	0.289	12.381	0.000	3.017	4.151
	occupation_Medicine staff	3.4782	0.332	10.482	0.000	2.828	4.129
	occupation_Private service staff	2.4475	0.430	5.694	0.000	1.605	3.290
	occupation_Realty agents	2.3664	0.824	2.872	0.004	0.752	3.981
	occupation_Sales staff	3.9178	0.280	13.999	0.000	3.369	4.466
	occupation_Secretaries	2.2059	0.592	3.724	0.000	1.045	3.367

occupation_Security staff	3.1293	0.345	9.082	0.000	2.454	3.805
occupation_Unknown	3.5746	0.276	12.948	0.000	3.033	4.116
occupation_Waiters/barmen staff	5.0792	0.436	11.649	0.000	4.225	5.934

Possibly complete quasi-separation: A fraction 0.60 of observations can be perfectly predicted. This might indicate that there is complete quasi-separation. In this case some parameters will not be identified.

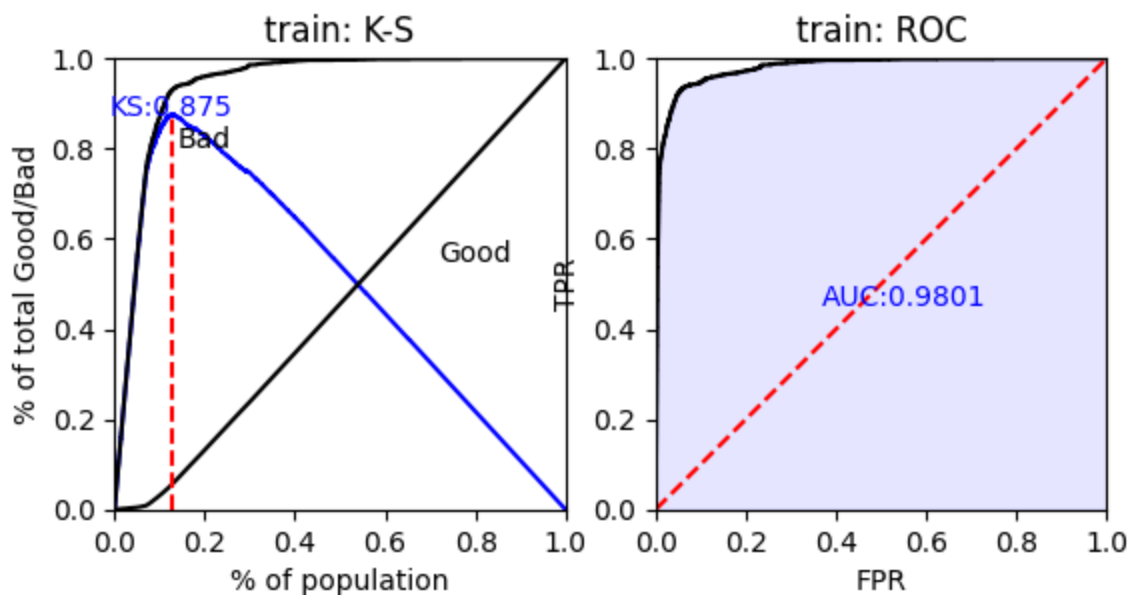
```
In [89]: lr = LogisticRegression(penalty='l1', C=0.9, solver='saga', n_jobs=-1)
lr.fit(X_train_woe, y_train_woe)
```

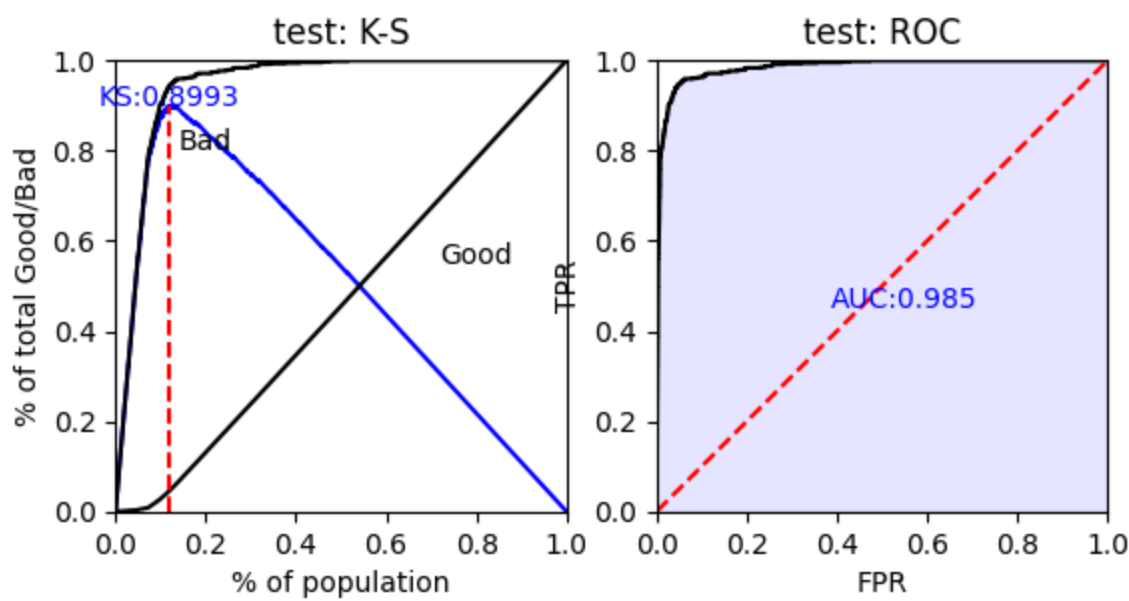
```
Out[89]: LogisticRegression
LogisticRegression(C=0.9, n_jobs=-1, penalty='l1', solver='saga')
```

```
In [90]: # predicted probability

train_pred = lr.predict_proba(X_train_woe)[:,-1]
test_pred = lr.predict_proba(X_test_woe)[:,-1]
```

```
In [91]: # performance ks & roc -----
train_perf = sc.perf_eva(y_train_woe, train_pred, title = "train")
test_perf = sc.perf_eva(y_test_woe, test_pred, title = "test")
```





```
In [92]: # score -----
card = sc.scorecard(bins, lr, X_train_woe.columns)

old_df = old_df.copy()

# credit score
score = sc.scorecard_ply(old_df, card, only_total_score=False, print_step=0, replac
```

```
In [93]: score
```

Out[93]:

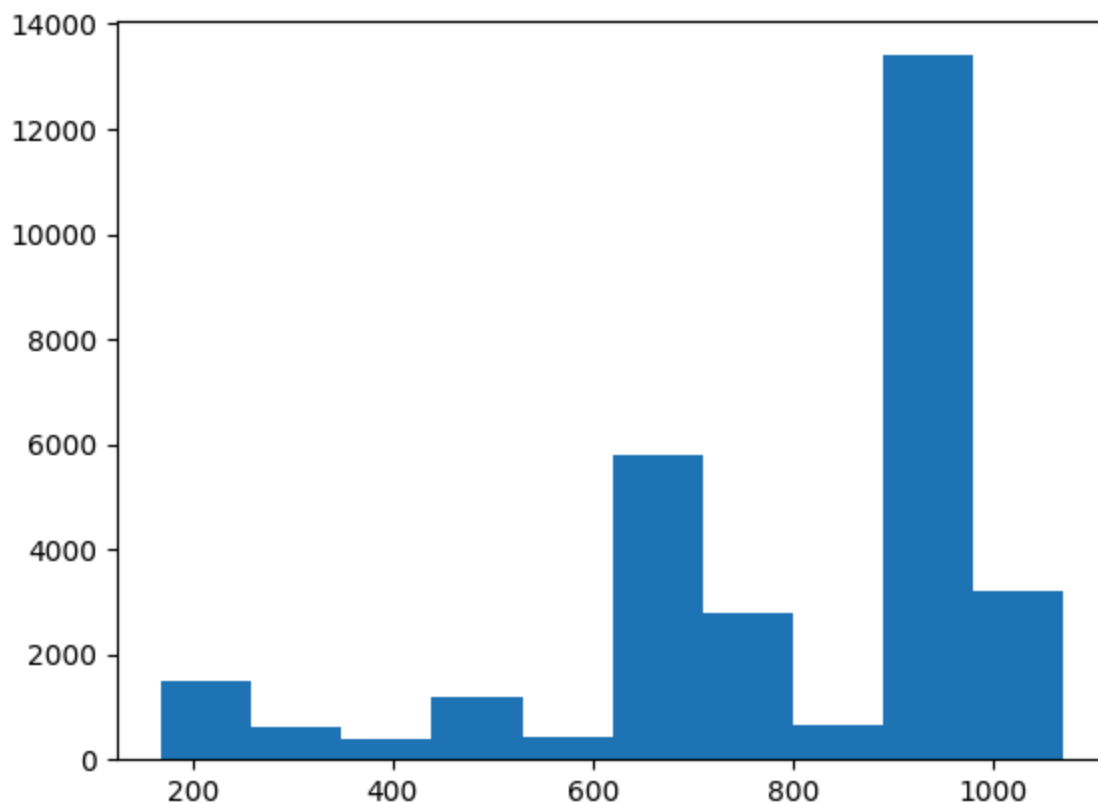
	credit_limit_used(%)_points	credit_limit_points	credit_score_points	migrant_worker_p
34126	177.0	-2.0	201.0	
44609	177.0	-2.0	201.0	
44264	-84.0	-2.0	-20.0	
33556	177.0	-2.0	201.0	
28050	177.0	12.0	201.0	
...	...	...	...	
37523	177.0	-2.0	201.0	
36228	177.0	-5.0	201.0	
32772	177.0	-2.0	201.0	
9762	177.0	-2.0	201.0	
33325	177.0	-2.0	201.0	

30000 rows × 12 columns



```
In [94]: plt.hist(score["score"])
```

```
Out[94]: (array([ 1503.,   629.,   407.,  1204.,   437.,  5800.,  2788.,   645.,
        13398.,  3189.]),
 array([ 169.,  259.,  349.,  439.,  529.,  619.,  709.,  799.,  889.,
        979., 1069.]),
 <BarContainer object of 10 artists>)
```



- The data is heavily skewed towards the higher end of the scale. There are 2 major clusters - one between 600-750 and another much larger dominant spike around 900s, which typically correlate with lower credit risk. - This suggests that the majority of the individuals in this dataset have high scores, which typically correlates with lower credit risk. - There is a much smaller concentration of data point in the lower range(200 to 500), representing a minority of high-risk entries.

```
In [ ]:
```

Evaluation matrix

```
In [95]: # evaluation function
```

```
def evaluation(model):
    y_train_pred = model.predict(X_train)
    y_test_pred = model.predict(X_test)
    print("="*50)

    print(confusion_matrix(y_train, y_train_pred))
    print(classification_report(y_train, y_train_pred))
    print("Accuracy of TRAIN data:", 100*accuracy_score(y_train, y_train_pred))
    print("F1_Score of TRAIN data:", 100*f1_score(y_train, y_train_pred, average="m
    print("="*50)
```

```

print(confusion_matrix(y_test, y_test_pred))
print(classification_report(y_test, y_test_pred))
print("Accuracy of TEST data:", 100*accuracy_score(y_test, y_test_pred))
print("F1_Score of TEST data:", 100*f1_score(y_test, y_test_pred, average="macro"))
print("RMSE: ", rmse)

# calculate the RMSE
rmse = np.sqrt(mean_squared_error(y_test, y_test_pred))
print("RMSE: ", rmse)

# calculate ROC AUC
roc_auc = roc_auc_score(y_test, y_test_pred, average=None)
print("ROC AUC score: ", roc_auc)

```

In [96]: `y_test`

```

Out[96]: 11858    0
         22651    0
         38925    0
         2917    0
         18012    0
         ..
         37539    0
         17978    1
         5227     0
         3994     0
         8471     0
Name: credit_card_default, Length: 9000, dtype: int64

```

In []:

Logistic Regression

In [97]: `# initialize LogisticRegression and fit the model`

```

logreg = LogisticRegression()
logreg.fit(X_train, y_train)

```

Out[97]:

LogisticRegression

In [98]: `# prediction`

```

evaluation(logreg)

```



```

=====
[[18239 1037]
 [ 617 18659]]
      precision    recall  f1-score   support

      0       0.97      0.95      0.96      19276
      1       0.95      0.97      0.96      19276

 accuracy          0.96      0.96      0.96      38552
 macro avg          0.96      0.96      0.96      38552
 weighted avg       0.96      0.96      0.96      38552

```

Accuracy of TRAIN data: 95.70969080722142

F1_Score of TRAIN data: 95.70918154097276

```

=====
[[7798 463]
 [ 19 720]]
      precision    recall  f1-score   support

      0       1.00      0.94      0.97      8261
      1       0.61      0.97      0.75       739

 accuracy          0.95      0.95      0.95      9000
 macro avg          0.80      0.96      0.86      9000
 weighted avg       0.97      0.95      0.95      9000

```

Accuracy of TEST data: 94.64444444444445

F1_Score of TEST data: 85.96203549320371

```

=====
RMSE: 0.23142073276946376
ROC AUC score: 0.9591215485188158

```

In []:

Decision Tree

```

In [99]: # initialize and fit the model

tree_model = DecisionTreeClassifier()
tree_model.fit(X_train, y_train)

```

Out[99]:

DecisionTreeClassifier

DecisionTreeClassifier()

In [100... # prediction

```
evaluation(tree_model)
```

```

=====
[[19276    0]
 [    0 19276]]
      precision    recall  f1-score   support

         0         1.00      1.00      1.00     19276
         1         1.00      1.00      1.00     19276

 accuracy                   1.00      38552
 macro avg           1.00      1.00      1.00      38552
 weighted avg        1.00      1.00      1.00      38552

```

Accuracy of TRAIN data: 100.0

F1_Score of TRAIN data: 100.0

```

=====
[[8050  211]
 [  89 650]]
      precision    recall  f1-score   support

         0         0.99      0.97      0.98      8261
         1         0.75      0.88      0.81       739

 accuracy                   0.97      9000
 macro avg           0.87      0.93      0.90      9000
 weighted avg        0.97      0.97      0.97      9000

```

Accuracy of TEST data: 96.66666666666667

F1_Score of TEST data: 89.71036585365853

```

=====
RMSE:  0.18257418583505536
ROC AUC score:  0.9270126402177667

```

In []:

Random Forest

```

In [101...  # initialize and fit the model

regr_rfr = RandomForestClassifier(random_state=42, oob_score=True)
regr_rfr.fit(X_train, y_train)

```

```

Out[101...  RandomForestClassifier
RandomForestClassifier(oob_score=True, random_state=42)

```

```

In [102...  # prediction

evaluation(regr_rfr)

```

```

=====
[[19276    0]
 [    0 19276]]
      precision    recall  f1-score   support

         0         1.00      1.00      1.00     19276
         1         1.00      1.00      1.00     19276

 accuracy                   1.00      38552
 macro avg              1.00      1.00      1.00      38552
 weighted avg          1.00      1.00      1.00      38552

```

Accuracy of TRAIN data: 100.0

F1_Score of TRAIN data: 100.0

```

=====
[[8009  252]
 [   73  666]]
      precision    recall  f1-score   support

         0         0.99      0.97      0.98     8261
         1         0.73      0.90      0.80      739

 accuracy                   0.96     9000
 macro avg              0.86      0.94      0.89     9000
 weighted avg          0.97      0.96      0.97     9000

```

Accuracy of TEST data: 96.38888888888889

F1_Score of TEST data: 89.19881060625838

```

=====
RMSE:  0.19002923751652298
ROC AUC score:  0.9353565402360965

```

In []:

CatBoost

In [103...

```

# Initialize and fit the model

reg_catb = CatBoostClassifier(random_seed=42)
reg_catb.fit(X_train, y_train)

```

Learning rate set to 0.048998

0:	learn: 0.5462118	total: 188ms	remaining: 3m 8s
1:	learn: 0.4354220	total: 204ms	remaining: 1m 41s
2:	learn: 0.3434804	total: 218ms	remaining: 1m 12s
3:	learn: 0.2738045	total: 233ms	remaining: 58s
4:	learn: 0.2250598	total: 248ms	remaining: 49.3s
5:	learn: 0.1913519	total: 262ms	remaining: 43.4s
6:	learn: 0.1674087	total: 277ms	remaining: 39.3s
7:	learn: 0.1490320	total: 292ms	remaining: 36.2s
8:	learn: 0.1340134	total: 307ms	remaining: 33.9s
9:	learn: 0.1213987	total: 328ms	remaining: 32.5s
10:	learn: 0.1123253	total: 345ms	remaining: 31s
11:	learn: 0.1057285	total: 364ms	remaining: 29.9s
12:	learn: 0.0998190	total: 382ms	remaining: 29s
13:	learn: 0.0953607	total: 402ms	remaining: 28.3s
14:	learn: 0.0914903	total: 422ms	remaining: 27.7s
15:	learn: 0.0886104	total: 440ms	remaining: 27.1s
16:	learn: 0.0854456	total: 460ms	remaining: 26.6s
17:	learn: 0.0837871	total: 477ms	remaining: 26s
18:	learn: 0.0816394	total: 495ms	remaining: 25.6s
19:	learn: 0.0797185	total: 514ms	remaining: 25.2s
20:	learn: 0.0782680	total: 535ms	remaining: 25s
21:	learn: 0.0772190	total: 551ms	remaining: 24.5s
22:	learn: 0.0756872	total: 570ms	remaining: 24.2s
23:	learn: 0.0749046	total: 588ms	remaining: 23.9s
24:	learn: 0.0741270	total: 607ms	remaining: 23.7s
25:	learn: 0.0735554	total: 630ms	remaining: 23.6s
26:	learn: 0.0729514	total: 646ms	remaining: 23.3s
27:	learn: 0.0724371	total: 660ms	remaining: 22.9s
28:	learn: 0.0720101	total: 675ms	remaining: 22.6s
29:	learn: 0.0716461	total: 690ms	remaining: 22.3s
30:	learn: 0.0712441	total: 704ms	remaining: 22s
31:	learn: 0.0709403	total: 719ms	remaining: 21.7s
32:	learn: 0.0706294	total: 733ms	remaining: 21.5s
33:	learn: 0.0703434	total: 748ms	remaining: 21.3s
34:	learn: 0.0700335	total: 763ms	remaining: 21s
35:	learn: 0.0697880	total: 777ms	remaining: 20.8s
36:	learn: 0.0694997	total: 792ms	remaining: 20.6s
37:	learn: 0.0693101	total: 806ms	remaining: 20.4s
38:	learn: 0.0691112	total: 821ms	remaining: 20.2s
39:	learn: 0.0685112	total: 835ms	remaining: 20s
40:	learn: 0.0681555	total: 851ms	remaining: 19.9s
41:	learn: 0.0679863	total: 866ms	remaining: 19.7s
42:	learn: 0.0678165	total: 880ms	remaining: 19.6s
43:	learn: 0.0676579	total: 895ms	remaining: 19.4s
44:	learn: 0.0675389	total: 909ms	remaining: 19.3s
45:	learn: 0.0674047	total: 923ms	remaining: 19.1s
46:	learn: 0.0670429	total: 937ms	remaining: 19s
47:	learn: 0.0668647	total: 952ms	remaining: 18.9s
48:	learn: 0.0665947	total: 966ms	remaining: 18.8s
49:	learn: 0.0662009	total: 980ms	remaining: 18.6s
50:	learn: 0.0660608	total: 995ms	remaining: 18.5s
51:	learn: 0.0659296	total: 1.01s	remaining: 18.4s
52:	learn: 0.0657935	total: 1.02s	remaining: 18.3s
53:	learn: 0.0656739	total: 1.04s	remaining: 18.2s
54:	learn: 0.0655293	total: 1.06s	remaining: 18.2s

55:	learn: 0.0653536	total: 1.07s	remaining: 18.1s
56:	learn: 0.0652142	total: 1.09s	remaining: 18s
57:	learn: 0.0651114	total: 1.1s	remaining: 17.9s
58:	learn: 0.0650127	total: 1.12s	remaining: 17.8s
59:	learn: 0.0649556	total: 1.13s	remaining: 17.7s
60:	learn: 0.0647125	total: 1.15s	remaining: 17.6s
61:	learn: 0.0645647	total: 1.16s	remaining: 17.6s
62:	learn: 0.0644152	total: 1.17s	remaining: 17.5s
63:	learn: 0.0643574	total: 1.19s	remaining: 17.4s
64:	learn: 0.0642940	total: 1.2s	remaining: 17.3s
65:	learn: 0.0640926	total: 1.22s	remaining: 17.2s
66:	learn: 0.0639457	total: 1.23s	remaining: 17.1s
67:	learn: 0.0639121	total: 1.24s	remaining: 17.1s
68:	learn: 0.0638354	total: 1.26s	remaining: 17s
69:	learn: 0.0637271	total: 1.27s	remaining: 16.9s
70:	learn: 0.0635721	total: 1.29s	remaining: 16.8s
71:	learn: 0.0634877	total: 1.3s	remaining: 16.8s
72:	learn: 0.0634037	total: 1.32s	remaining: 16.7s
73:	learn: 0.0633515	total: 1.33s	remaining: 16.7s
74:	learn: 0.0632165	total: 1.35s	remaining: 16.7s
75:	learn: 0.0631127	total: 1.37s	remaining: 16.7s
76:	learn: 0.0630802	total: 1.39s	remaining: 16.7s
77:	learn: 0.0629443	total: 1.41s	remaining: 16.6s
78:	learn: 0.0626772	total: 1.43s	remaining: 16.6s
79:	learn: 0.0625983	total: 1.44s	remaining: 16.6s
80:	learn: 0.0625393	total: 1.46s	remaining: 16.6s
81:	learn: 0.0624033	total: 1.48s	remaining: 16.6s
82:	learn: 0.0622143	total: 1.5s	remaining: 16.6s
83:	learn: 0.0621274	total: 1.52s	remaining: 16.6s
84:	learn: 0.0620489	total: 1.54s	remaining: 16.6s
85:	learn: 0.0617909	total: 1.56s	remaining: 16.6s
86:	learn: 0.0616755	total: 1.57s	remaining: 16.5s
87:	learn: 0.0615794	total: 1.59s	remaining: 16.5s
88:	learn: 0.0615431	total: 1.61s	remaining: 16.5s
89:	learn: 0.0614879	total: 1.63s	remaining: 16.5s
90:	learn: 0.0613137	total: 1.64s	remaining: 16.4s
91:	learn: 0.0612666	total: 1.66s	remaining: 16.4s
92:	learn: 0.0611125	total: 1.67s	remaining: 16.3s
93:	learn: 0.0610284	total: 1.69s	remaining: 16.3s
94:	learn: 0.0608569	total: 1.7s	remaining: 16.2s
95:	learn: 0.0606893	total: 1.72s	remaining: 16.2s
96:	learn: 0.0605932	total: 1.73s	remaining: 16.1s
97:	learn: 0.0605684	total: 1.75s	remaining: 16.1s
98:	learn: 0.0605481	total: 1.76s	remaining: 16s
99:	learn: 0.0603907	total: 1.77s	remaining: 16s
100:	learn: 0.0602574	total: 1.79s	remaining: 15.9s
101:	learn: 0.0601090	total: 1.8s	remaining: 15.9s
102:	learn: 0.0600691	total: 1.82s	remaining: 15.8s
103:	learn: 0.0599811	total: 1.83s	remaining: 15.8s
104:	learn: 0.0598365	total: 1.85s	remaining: 15.7s
105:	learn: 0.0597645	total: 1.86s	remaining: 15.7s
106:	learn: 0.0596609	total: 1.88s	remaining: 15.7s
107:	learn: 0.0595586	total: 1.89s	remaining: 15.6s
108:	learn: 0.0593743	total: 1.91s	remaining: 15.6s
109:	learn: 0.0592580	total: 1.93s	remaining: 15.6s
110:	learn: 0.0591407	total: 1.95s	remaining: 15.6s

111:	learn: 0.0589252	total: 1.97s	remaining: 15.6s
112:	learn: 0.0588242	total: 1.99s	remaining: 15.6s
113:	learn: 0.0587417	total: 2.01s	remaining: 15.6s
114:	learn: 0.0586399	total: 2.03s	remaining: 15.6s
115:	learn: 0.0586073	total: 2.05s	remaining: 15.6s
116:	learn: 0.0585484	total: 2.07s	remaining: 15.6s
117:	learn: 0.0585271	total: 2.09s	remaining: 15.6s
118:	learn: 0.0584422	total: 2.1s	remaining: 15.6s
119:	learn: 0.0583827	total: 2.12s	remaining: 15.5s
120:	learn: 0.0582842	total: 2.13s	remaining: 15.5s
121:	learn: 0.0581803	total: 2.15s	remaining: 15.5s
122:	learn: 0.0581190	total: 2.16s	remaining: 15.4s
123:	learn: 0.0580650	total: 2.18s	remaining: 15.4s
124:	learn: 0.0579704	total: 2.19s	remaining: 15.3s
125:	learn: 0.0578158	total: 2.21s	remaining: 15.3s
126:	learn: 0.0577203	total: 2.22s	remaining: 15.3s
127:	learn: 0.0576216	total: 2.23s	remaining: 15.2s
128:	learn: 0.0575758	total: 2.25s	remaining: 15.2s
129:	learn: 0.0575021	total: 2.26s	remaining: 15.1s
130:	learn: 0.0574892	total: 2.28s	remaining: 15.1s
131:	learn: 0.0573842	total: 2.29s	remaining: 15.1s
132:	learn: 0.0573022	total: 2.3s	remaining: 15s
133:	learn: 0.0571948	total: 2.32s	remaining: 15s
134:	learn: 0.0571245	total: 2.33s	remaining: 15s
135:	learn: 0.0570830	total: 2.35s	remaining: 15s
136:	learn: 0.0570461	total: 2.37s	remaining: 14.9s
137:	learn: 0.0569733	total: 2.39s	remaining: 14.9s
138:	learn: 0.0568250	total: 2.41s	remaining: 14.9s
139:	learn: 0.0567312	total: 2.42s	remaining: 14.9s
140:	learn: 0.0566018	total: 2.44s	remaining: 14.9s
141:	learn: 0.0565608	total: 2.46s	remaining: 14.9s
142:	learn: 0.0564728	total: 2.48s	remaining: 14.9s
143:	learn: 0.0563958	total: 2.5s	remaining: 14.8s
144:	learn: 0.0563352	total: 2.52s	remaining: 14.9s
145:	learn: 0.0562996	total: 2.54s	remaining: 14.8s
146:	learn: 0.0561952	total: 2.56s	remaining: 14.8s
147:	learn: 0.0560791	total: 2.58s	remaining: 14.8s
148:	learn: 0.0559565	total: 2.6s	remaining: 14.9s
149:	learn: 0.0558646	total: 2.62s	remaining: 14.9s
150:	learn: 0.0557865	total: 2.64s	remaining: 14.8s
151:	learn: 0.0557247	total: 2.66s	remaining: 14.8s
152:	learn: 0.0556313	total: 2.67s	remaining: 14.8s
153:	learn: 0.0555048	total: 2.69s	remaining: 14.8s
154:	learn: 0.0554447	total: 2.7s	remaining: 14.7s
155:	learn: 0.0553955	total: 2.72s	remaining: 14.7s
156:	learn: 0.0553372	total: 2.73s	remaining: 14.7s
157:	learn: 0.0552700	total: 2.75s	remaining: 14.6s
158:	learn: 0.0552198	total: 2.76s	remaining: 14.6s
159:	learn: 0.0551506	total: 2.78s	remaining: 14.6s
160:	learn: 0.0550795	total: 2.79s	remaining: 14.5s
161:	learn: 0.0549389	total: 2.81s	remaining: 14.5s
162:	learn: 0.0548794	total: 2.82s	remaining: 14.5s
163:	learn: 0.0547599	total: 2.83s	remaining: 14.4s
164:	learn: 0.0547229	total: 2.85s	remaining: 14.4s
165:	learn: 0.0546761	total: 2.86s	remaining: 14.4s
166:	learn: 0.0545636	total: 2.88s	remaining: 14.4s

167:	learn: 0.0544966	total: 2.89s	remaining: 14.3s
168:	learn: 0.0544049	total: 2.9s	remaining: 14.3s
169:	learn: 0.0543390	total: 2.92s	remaining: 14.3s
170:	learn: 0.0542431	total: 2.93s	remaining: 14.2s
171:	learn: 0.0542111	total: 2.95s	remaining: 14.2s
172:	learn: 0.0541022	total: 2.96s	remaining: 14.2s
173:	learn: 0.0540271	total: 2.98s	remaining: 14.2s
174:	learn: 0.0539841	total: 3s	remaining: 14.1s
175:	learn: 0.0538684	total: 3.01s	remaining: 14.1s
176:	learn: 0.0538068	total: 3.02s	remaining: 14.1s
177:	learn: 0.0537077	total: 3.04s	remaining: 14s
178:	learn: 0.0536488	total: 3.05s	remaining: 14s
179:	learn: 0.0536259	total: 3.07s	remaining: 14s
180:	learn: 0.0535358	total: 3.08s	remaining: 13.9s
181:	learn: 0.0534748	total: 3.1s	remaining: 13.9s
182:	learn: 0.0534102	total: 3.11s	remaining: 13.9s
183:	learn: 0.0533042	total: 3.12s	remaining: 13.9s
184:	learn: 0.0532251	total: 3.14s	remaining: 13.8s
185:	learn: 0.0531267	total: 3.15s	remaining: 13.8s
186:	learn: 0.0530737	total: 3.17s	remaining: 13.8s
187:	learn: 0.0529764	total: 3.18s	remaining: 13.8s
188:	learn: 0.0529101	total: 3.2s	remaining: 13.7s
189:	learn: 0.0528469	total: 3.21s	remaining: 13.7s
190:	learn: 0.0527804	total: 3.22s	remaining: 13.7s
191:	learn: 0.0526004	total: 3.24s	remaining: 13.6s
192:	learn: 0.0524539	total: 3.25s	remaining: 13.6s
193:	learn: 0.0523967	total: 3.27s	remaining: 13.6s
194:	learn: 0.0522781	total: 3.28s	remaining: 13.6s
195:	learn: 0.0522087	total: 3.3s	remaining: 13.5s
196:	learn: 0.0521293	total: 3.31s	remaining: 13.5s
197:	learn: 0.0520430	total: 3.33s	remaining: 13.5s
198:	learn: 0.0519907	total: 3.34s	remaining: 13.4s
199:	learn: 0.0519433	total: 3.36s	remaining: 13.4s
200:	learn: 0.0518833	total: 3.38s	remaining: 13.4s
201:	learn: 0.0517762	total: 3.39s	remaining: 13.4s
202:	learn: 0.0517068	total: 3.41s	remaining: 13.4s
203:	learn: 0.0516337	total: 3.43s	remaining: 13.4s
204:	learn: 0.0515414	total: 3.45s	remaining: 13.4s
205:	learn: 0.0515007	total: 3.47s	remaining: 13.4s
206:	learn: 0.0514486	total: 3.49s	remaining: 13.4s
207:	learn: 0.0514238	total: 3.5s	remaining: 13.3s
208:	learn: 0.0512572	total: 3.52s	remaining: 13.3s
209:	learn: 0.0510971	total: 3.54s	remaining: 13.3s
210:	learn: 0.0510463	total: 3.56s	remaining: 13.3s
211:	learn: 0.0509527	total: 3.57s	remaining: 13.3s
212:	learn: 0.0508795	total: 3.59s	remaining: 13.3s
213:	learn: 0.0508028	total: 3.61s	remaining: 13.3s
214:	learn: 0.0506726	total: 3.63s	remaining: 13.2s
215:	learn: 0.0505972	total: 3.64s	remaining: 13.2s
216:	learn: 0.0504652	total: 3.66s	remaining: 13.2s
217:	learn: 0.0504299	total: 3.68s	remaining: 13.2s
218:	learn: 0.0503936	total: 3.69s	remaining: 13.2s
219:	learn: 0.0503354	total: 3.71s	remaining: 13.1s
220:	learn: 0.0502725	total: 3.72s	remaining: 13.1s
221:	learn: 0.0501924	total: 3.73s	remaining: 13.1s
222:	learn: 0.0501312	total: 3.75s	remaining: 13.1s

223:	learn: 0.0500748	total: 3.76s	remaining: 13s
224:	learn: 0.0500452	total: 3.78s	remaining: 13s
225:	learn: 0.0499727	total: 3.79s	remaining: 13s
226:	learn: 0.0499226	total: 3.81s	remaining: 13s
227:	learn: 0.0498821	total: 3.82s	remaining: 12.9s
228:	learn: 0.0497963	total: 3.84s	remaining: 12.9s
229:	learn: 0.0496906	total: 3.85s	remaining: 12.9s
230:	learn: 0.0496174	total: 3.87s	remaining: 12.9s
231:	learn: 0.0495193	total: 3.88s	remaining: 12.8s
232:	learn: 0.0494633	total: 3.89s	remaining: 12.8s
233:	learn: 0.0493636	total: 3.91s	remaining: 12.8s
234:	learn: 0.0492568	total: 3.92s	remaining: 12.8s
235:	learn: 0.0491832	total: 3.94s	remaining: 12.8s
236:	learn: 0.0490654	total: 3.95s	remaining: 12.7s
237:	learn: 0.0489895	total: 3.97s	remaining: 12.7s
238:	learn: 0.0489526	total: 3.98s	remaining: 12.7s
239:	learn: 0.0489104	total: 4s	remaining: 12.7s
240:	learn: 0.0488603	total: 4.01s	remaining: 12.6s
241:	learn: 0.0488090	total: 4.03s	remaining: 12.6s
242:	learn: 0.0487543	total: 4.04s	remaining: 12.6s
243:	learn: 0.0486958	total: 4.05s	remaining: 12.6s
244:	learn: 0.0486319	total: 4.07s	remaining: 12.5s
245:	learn: 0.0485026	total: 4.08s	remaining: 12.5s
246:	learn: 0.0484040	total: 4.1s	remaining: 12.5s
247:	learn: 0.0483490	total: 4.11s	remaining: 12.5s
248:	learn: 0.0483061	total: 4.13s	remaining: 12.4s
249:	learn: 0.0482483	total: 4.14s	remaining: 12.4s
250:	learn: 0.0481585	total: 4.16s	remaining: 12.4s
251:	learn: 0.0480539	total: 4.17s	remaining: 12.4s
252:	learn: 0.0480174	total: 4.18s	remaining: 12.4s
253:	learn: 0.0479891	total: 4.2s	remaining: 12.3s
254:	learn: 0.0478873	total: 4.21s	remaining: 12.3s
255:	learn: 0.0477987	total: 4.23s	remaining: 12.3s
256:	learn: 0.0477541	total: 4.24s	remaining: 12.3s
257:	learn: 0.0476843	total: 4.26s	remaining: 12.2s
258:	learn: 0.0475040	total: 4.27s	remaining: 12.2s
259:	learn: 0.0473460	total: 4.29s	remaining: 12.2s
260:	learn: 0.0471811	total: 4.3s	remaining: 12.2s
261:	learn: 0.0470433	total: 4.31s	remaining: 12.2s
262:	learn: 0.0469049	total: 4.33s	remaining: 12.1s
263:	learn: 0.0467790	total: 4.34s	remaining: 12.1s
264:	learn: 0.0466538	total: 4.36s	remaining: 12.1s
265:	learn: 0.0465187	total: 4.37s	remaining: 12.1s
266:	learn: 0.0464084	total: 4.39s	remaining: 12.1s
267:	learn: 0.0462869	total: 4.41s	remaining: 12s
268:	learn: 0.0462268	total: 4.43s	remaining: 12s
269:	learn: 0.0460950	total: 4.45s	remaining: 12s
270:	learn: 0.0459928	total: 4.46s	remaining: 12s
271:	learn: 0.0458831	total: 4.48s	remaining: 12s
272:	learn: 0.0457729	total: 4.5s	remaining: 12s
273:	learn: 0.0457172	total: 4.52s	remaining: 12s
274:	learn: 0.0456265	total: 4.54s	remaining: 12s
275:	learn: 0.0455805	total: 4.55s	remaining: 11.9s
276:	learn: 0.0455013	total: 4.58s	remaining: 12s
277:	learn: 0.0454095	total: 4.6s	remaining: 11.9s
278:	learn: 0.0453210	total: 4.62s	remaining: 11.9s

279:	learn: 0.0452542	total: 4.64s	remaining: 11.9s
280:	learn: 0.0451632	total: 4.66s	remaining: 11.9s
281:	learn: 0.0450829	total: 4.67s	remaining: 11.9s
282:	learn: 0.0450362	total: 4.69s	remaining: 11.9s
283:	learn: 0.0449784	total: 4.7s	remaining: 11.9s
284:	learn: 0.0449375	total: 4.72s	remaining: 11.8s
285:	learn: 0.0448289	total: 4.73s	remaining: 11.8s
286:	learn: 0.0447868	total: 4.75s	remaining: 11.8s
287:	learn: 0.0447463	total: 4.76s	remaining: 11.8s
288:	learn: 0.0446802	total: 4.78s	remaining: 11.8s
289:	learn: 0.0446295	total: 4.79s	remaining: 11.7s
290:	learn: 0.0445466	total: 4.8s	remaining: 11.7s
291:	learn: 0.0445132	total: 4.82s	remaining: 11.7s
292:	learn: 0.0444427	total: 4.83s	remaining: 11.7s
293:	learn: 0.0443643	total: 4.85s	remaining: 11.6s
294:	learn: 0.0443066	total: 4.86s	remaining: 11.6s
295:	learn: 0.0442525	total: 4.88s	remaining: 11.6s
296:	learn: 0.0442072	total: 4.89s	remaining: 11.6s
297:	learn: 0.0441560	total: 4.91s	remaining: 11.6s
298:	learn: 0.0440958	total: 4.92s	remaining: 11.5s
299:	learn: 0.0440557	total: 4.94s	remaining: 11.5s
300:	learn: 0.0439556	total: 4.95s	remaining: 11.5s
301:	learn: 0.0438034	total: 4.96s	remaining: 11.5s
302:	learn: 0.0437706	total: 4.98s	remaining: 11.5s
303:	learn: 0.0435651	total: 4.99s	remaining: 11.4s
304:	learn: 0.0435063	total: 5.01s	remaining: 11.4s
305:	learn: 0.0434605	total: 5.02s	remaining: 11.4s
306:	learn: 0.0433816	total: 5.04s	remaining: 11.4s
307:	learn: 0.0433199	total: 5.05s	remaining: 11.4s
308:	learn: 0.0432104	total: 5.07s	remaining: 11.3s
309:	learn: 0.0431701	total: 5.08s	remaining: 11.3s
310:	learn: 0.0430694	total: 5.1s	remaining: 11.3s
311:	learn: 0.0430280	total: 5.11s	remaining: 11.3s
312:	learn: 0.0429634	total: 5.13s	remaining: 11.3s
313:	learn: 0.0429506	total: 5.14s	remaining: 11.2s
314:	learn: 0.0428849	total: 5.15s	remaining: 11.2s
315:	learn: 0.0427485	total: 5.17s	remaining: 11.2s
316:	learn: 0.0426164	total: 5.18s	remaining: 11.2s
317:	learn: 0.0425714	total: 5.2s	remaining: 11.1s
318:	learn: 0.0424076	total: 5.21s	remaining: 11.1s
319:	learn: 0.0422010	total: 5.22s	remaining: 11.1s
320:	learn: 0.0421069	total: 5.24s	remaining: 11.1s
321:	learn: 0.0420171	total: 5.25s	remaining: 11.1s
322:	learn: 0.0418966	total: 5.27s	remaining: 11s
323:	learn: 0.0418631	total: 5.28s	remaining: 11s
324:	learn: 0.0418165	total: 5.3s	remaining: 11s
325:	learn: 0.0417708	total: 5.31s	remaining: 11s
326:	learn: 0.0416009	total: 5.33s	remaining: 11s
327:	learn: 0.0415216	total: 5.34s	remaining: 10.9s
328:	learn: 0.0414477	total: 5.36s	remaining: 10.9s
329:	learn: 0.0413611	total: 5.37s	remaining: 10.9s
330:	learn: 0.0412406	total: 5.39s	remaining: 10.9s
331:	learn: 0.0411816	total: 5.41s	remaining: 10.9s
332:	learn: 0.0410432	total: 5.43s	remaining: 10.9s
333:	learn: 0.0410149	total: 5.45s	remaining: 10.9s
334:	learn: 0.0409616	total: 5.46s	remaining: 10.8s

335:	learn: 0.0408430	total: 5.48s	remaining: 10.8s
336:	learn: 0.0407199	total: 5.5s	remaining: 10.8s
337:	learn: 0.0406941	total: 5.52s	remaining: 10.8s
338:	learn: 0.0406185	total: 5.54s	remaining: 10.8s
339:	learn: 0.0405702	total: 5.56s	remaining: 10.8s
340:	learn: 0.0404947	total: 5.58s	remaining: 10.8s
341:	learn: 0.0403956	total: 5.6s	remaining: 10.8s
342:	learn: 0.0403386	total: 5.62s	remaining: 10.8s
343:	learn: 0.0403148	total: 5.63s	remaining: 10.7s
344:	learn: 0.0402718	total: 5.65s	remaining: 10.7s
345:	learn: 0.0402576	total: 5.67s	remaining: 10.7s
346:	learn: 0.0402106	total: 5.69s	remaining: 10.7s
347:	learn: 0.0401572	total: 5.7s	remaining: 10.7s
348:	learn: 0.0400954	total: 5.72s	remaining: 10.7s
349:	learn: 0.0400609	total: 5.73s	remaining: 10.6s
350:	learn: 0.0400268	total: 5.75s	remaining: 10.6s
351:	learn: 0.0399972	total: 5.76s	remaining: 10.6s
352:	learn: 0.0399180	total: 5.77s	remaining: 10.6s
353:	learn: 0.0398372	total: 5.79s	remaining: 10.6s
354:	learn: 0.0397442	total: 5.8s	remaining: 10.5s
355:	learn: 0.0397259	total: 5.82s	remaining: 10.5s
356:	learn: 0.0396885	total: 5.83s	remaining: 10.5s
357:	learn: 0.0396523	total: 5.84s	remaining: 10.5s
358:	learn: 0.0395928	total: 5.86s	remaining: 10.5s
359:	learn: 0.0395661	total: 5.87s	remaining: 10.4s
360:	learn: 0.0395327	total: 5.89s	remaining: 10.4s
361:	learn: 0.0395062	total: 5.9s	remaining: 10.4s
362:	learn: 0.0394530	total: 5.92s	remaining: 10.4s
363:	learn: 0.0393316	total: 5.94s	remaining: 10.4s
364:	learn: 0.0392276	total: 5.95s	remaining: 10.4s
365:	learn: 0.0391592	total: 5.97s	remaining: 10.3s
366:	learn: 0.0391090	total: 5.98s	remaining: 10.3s
367:	learn: 0.0390859	total: 6s	remaining: 10.3s
368:	learn: 0.0390104	total: 6.01s	remaining: 10.3s
369:	learn: 0.0389786	total: 6.02s	remaining: 10.3s
370:	learn: 0.0389383	total: 6.04s	remaining: 10.2s
371:	learn: 0.0389119	total: 6.05s	remaining: 10.2s
372:	learn: 0.0388669	total: 6.07s	remaining: 10.2s
373:	learn: 0.0388264	total: 6.08s	remaining: 10.2s
374:	learn: 0.0387973	total: 6.09s	remaining: 10.2s
375:	learn: 0.0387638	total: 6.11s	remaining: 10.1s
376:	learn: 0.0387486	total: 6.12s	remaining: 10.1s
377:	learn: 0.0387171	total: 6.14s	remaining: 10.1s
378:	learn: 0.0386918	total: 6.16s	remaining: 10.1s
379:	learn: 0.0386514	total: 6.18s	remaining: 10.1s
380:	learn: 0.0386290	total: 6.19s	remaining: 10.1s
381:	learn: 0.0385810	total: 6.21s	remaining: 10s
382:	learn: 0.0385326	total: 6.22s	remaining: 10s
383:	learn: 0.0385103	total: 6.24s	remaining: 10s
384:	learn: 0.0384701	total: 6.25s	remaining: 9.98s
385:	learn: 0.0384289	total: 6.26s	remaining: 9.96s
386:	learn: 0.0383839	total: 6.28s	remaining: 9.95s
387:	learn: 0.0383306	total: 6.29s	remaining: 9.93s
388:	learn: 0.0383014	total: 6.31s	remaining: 9.91s
389:	learn: 0.0381979	total: 6.32s	remaining: 9.89s
390:	learn: 0.0381524	total: 6.34s	remaining: 9.87s

391:	learn: 0.0381131	total: 6.35s	remaining: 9.85s
392:	learn: 0.0380939	total: 6.37s	remaining: 9.83s
393:	learn: 0.0380488	total: 6.38s	remaining: 9.82s
394:	learn: 0.0380093	total: 6.4s	remaining: 9.8s
395:	learn: 0.0379529	total: 6.42s	remaining: 9.79s
396:	learn: 0.0379259	total: 6.44s	remaining: 9.78s
397:	learn: 0.0378845	total: 6.46s	remaining: 9.77s
398:	learn: 0.0378543	total: 6.48s	remaining: 9.76s
399:	learn: 0.0378090	total: 6.5s	remaining: 9.74s
400:	learn: 0.0377847	total: 6.51s	remaining: 9.73s
401:	learn: 0.0377723	total: 6.54s	remaining: 9.72s
402:	learn: 0.0377519	total: 6.56s	remaining: 9.72s
403:	learn: 0.0377035	total: 6.58s	remaining: 9.71s
404:	learn: 0.0376761	total: 6.6s	remaining: 9.69s
405:	learn: 0.0376136	total: 6.62s	remaining: 9.68s
406:	learn: 0.0375898	total: 6.64s	remaining: 9.67s
407:	learn: 0.0375494	total: 6.66s	remaining: 9.66s
408:	learn: 0.0375105	total: 6.67s	remaining: 9.64s
409:	learn: 0.0374981	total: 6.69s	remaining: 9.62s
410:	learn: 0.0374694	total: 6.7s	remaining: 9.61s
411:	learn: 0.0373744	total: 6.72s	remaining: 9.59s
412:	learn: 0.0373356	total: 6.73s	remaining: 9.57s
413:	learn: 0.0372880	total: 6.75s	remaining: 9.55s
414:	learn: 0.0372376	total: 6.76s	remaining: 9.53s
415:	learn: 0.0371913	total: 6.78s	remaining: 9.51s
416:	learn: 0.0371276	total: 6.79s	remaining: 9.49s
417:	learn: 0.0370727	total: 6.81s	remaining: 9.48s
418:	learn: 0.0370170	total: 6.82s	remaining: 9.46s
419:	learn: 0.0369629	total: 6.83s	remaining: 9.44s
420:	learn: 0.0369032	total: 6.85s	remaining: 9.42s
421:	learn: 0.0368718	total: 6.86s	remaining: 9.4s
422:	learn: 0.0368472	total: 6.88s	remaining: 9.38s
423:	learn: 0.0368302	total: 6.89s	remaining: 9.37s
424:	learn: 0.0368124	total: 6.91s	remaining: 9.35s
425:	learn: 0.0367910	total: 6.92s	remaining: 9.33s
426:	learn: 0.0367532	total: 6.94s	remaining: 9.31s
427:	learn: 0.0367342	total: 6.95s	remaining: 9.29s
428:	learn: 0.0367059	total: 6.96s	remaining: 9.27s
429:	learn: 0.0366889	total: 6.98s	remaining: 9.25s
430:	learn: 0.0366334	total: 6.99s	remaining: 9.23s
431:	learn: 0.0365528	total: 7.01s	remaining: 9.22s
432:	learn: 0.0365154	total: 7.02s	remaining: 9.2s
433:	learn: 0.0364614	total: 7.04s	remaining: 9.18s
434:	learn: 0.0364261	total: 7.05s	remaining: 9.16s
435:	learn: 0.0364107	total: 7.07s	remaining: 9.14s
436:	learn: 0.0363281	total: 7.08s	remaining: 9.12s
437:	learn: 0.0362856	total: 7.09s	remaining: 9.1s
438:	learn: 0.0362211	total: 7.11s	remaining: 9.08s
439:	learn: 0.0361761	total: 7.12s	remaining: 9.07s
440:	learn: 0.0361235	total: 7.14s	remaining: 9.05s
441:	learn: 0.0360606	total: 7.15s	remaining: 9.03s
442:	learn: 0.0360122	total: 7.17s	remaining: 9.01s
443:	learn: 0.0359744	total: 7.18s	remaining: 8.99s
444:	learn: 0.0359304	total: 7.2s	remaining: 8.98s
445:	learn: 0.0359083	total: 7.21s	remaining: 8.96s
446:	learn: 0.0358914	total: 7.22s	remaining: 8.94s

447:	learn: 0.0358433	total: 7.24s	remaining: 8.92s
448:	learn: 0.0358337	total: 7.25s	remaining: 8.9s
449:	learn: 0.0358110	total: 7.27s	remaining: 8.88s
450:	learn: 0.0357656	total: 7.29s	remaining: 8.87s
451:	learn: 0.0357175	total: 7.3s	remaining: 8.85s
452:	learn: 0.0356212	total: 7.32s	remaining: 8.83s
453:	learn: 0.0355879	total: 7.33s	remaining: 8.81s
454:	learn: 0.0355049	total: 7.34s	remaining: 8.8s
455:	learn: 0.0354690	total: 7.36s	remaining: 8.78s
456:	learn: 0.0354353	total: 7.37s	remaining: 8.76s
457:	learn: 0.0354029	total: 7.39s	remaining: 8.75s
458:	learn: 0.0353689	total: 7.41s	remaining: 8.73s
459:	learn: 0.0353245	total: 7.43s	remaining: 8.72s
460:	learn: 0.0352759	total: 7.45s	remaining: 8.71s
461:	learn: 0.0352450	total: 7.47s	remaining: 8.7s
462:	learn: 0.0352266	total: 7.49s	remaining: 8.68s
463:	learn: 0.0351949	total: 7.5s	remaining: 8.67s
464:	learn: 0.0351594	total: 7.52s	remaining: 8.65s
465:	learn: 0.0351410	total: 7.54s	remaining: 8.64s
466:	learn: 0.0351006	total: 7.56s	remaining: 8.63s
467:	learn: 0.0350314	total: 7.58s	remaining: 8.61s
468:	learn: 0.0350224	total: 7.6s	remaining: 8.6s
469:	learn: 0.0349772	total: 7.61s	remaining: 8.58s
470:	learn: 0.0349399	total: 7.63s	remaining: 8.57s
471:	learn: 0.0348925	total: 7.64s	remaining: 8.55s
472:	learn: 0.0348774	total: 7.66s	remaining: 8.53s
473:	learn: 0.0348607	total: 7.67s	remaining: 8.51s
474:	learn: 0.0348401	total: 7.68s	remaining: 8.49s
475:	learn: 0.0348275	total: 7.7s	remaining: 8.47s
476:	learn: 0.0347818	total: 7.71s	remaining: 8.46s
477:	learn: 0.0347540	total: 7.73s	remaining: 8.44s
478:	learn: 0.0347119	total: 7.74s	remaining: 8.42s
479:	learn: 0.0346518	total: 7.75s	remaining: 8.4s
480:	learn: 0.0346343	total: 7.77s	remaining: 8.38s
481:	learn: 0.0346058	total: 7.78s	remaining: 8.36s
482:	learn: 0.0345823	total: 7.8s	remaining: 8.35s
483:	learn: 0.0345494	total: 7.81s	remaining: 8.33s
484:	learn: 0.0345397	total: 7.83s	remaining: 8.31s
485:	learn: 0.0345192	total: 7.84s	remaining: 8.29s
486:	learn: 0.0344646	total: 7.86s	remaining: 8.28s
487:	learn: 0.0344313	total: 7.87s	remaining: 8.26s
488:	learn: 0.0343750	total: 7.89s	remaining: 8.24s
489:	learn: 0.0343558	total: 7.9s	remaining: 8.22s
490:	learn: 0.0343170	total: 7.92s	remaining: 8.21s
491:	learn: 0.0342933	total: 7.93s	remaining: 8.19s
492:	learn: 0.0342723	total: 7.96s	remaining: 8.19s
493:	learn: 0.0342593	total: 7.97s	remaining: 8.16s
494:	learn: 0.0342173	total: 7.99s	remaining: 8.15s
495:	learn: 0.0341503	total: 8s	remaining: 8.13s
496:	learn: 0.0341144	total: 8.01s	remaining: 8.11s
497:	learn: 0.0340890	total: 8.03s	remaining: 8.09s
498:	learn: 0.0340751	total: 8.04s	remaining: 8.07s
499:	learn: 0.0340087	total: 8.06s	remaining: 8.06s
500:	learn: 0.0339915	total: 8.07s	remaining: 8.04s
501:	learn: 0.0339613	total: 8.09s	remaining: 8.02s
502:	learn: 0.0339287	total: 8.1s	remaining: 8.01s

503:	learn: 0.0338805	total: 8.12s	remaining: 7.99s
504:	learn: 0.0338351	total: 8.13s	remaining: 7.97s
505:	learn: 0.0338196	total: 8.14s	remaining: 7.95s
506:	learn: 0.0338090	total: 8.16s	remaining: 7.93s
507:	learn: 0.0338003	total: 8.17s	remaining: 7.92s
508:	learn: 0.0337853	total: 8.19s	remaining: 7.9s
509:	learn: 0.0337307	total: 8.2s	remaining: 7.88s
510:	learn: 0.0336804	total: 8.22s	remaining: 7.86s
511:	learn: 0.0336463	total: 8.23s	remaining: 7.84s
512:	learn: 0.0336177	total: 8.24s	remaining: 7.83s
513:	learn: 0.0336099	total: 8.26s	remaining: 7.81s
514:	learn: 0.0335799	total: 8.27s	remaining: 7.79s
515:	learn: 0.0335513	total: 8.29s	remaining: 7.78s
516:	learn: 0.0334813	total: 8.3s	remaining: 7.76s
517:	learn: 0.0334581	total: 8.32s	remaining: 7.74s
518:	learn: 0.0334060	total: 8.33s	remaining: 7.72s
519:	learn: 0.0333482	total: 8.35s	remaining: 7.71s
520:	learn: 0.0333275	total: 8.36s	remaining: 7.69s
521:	learn: 0.0332894	total: 8.38s	remaining: 7.67s
522:	learn: 0.0332711	total: 8.39s	remaining: 7.65s
523:	learn: 0.0332432	total: 8.4s	remaining: 7.63s
524:	learn: 0.0332176	total: 8.42s	remaining: 7.62s
525:	learn: 0.0331895	total: 8.43s	remaining: 7.6s
526:	learn: 0.0331019	total: 8.45s	remaining: 7.58s
527:	learn: 0.0330363	total: 8.46s	remaining: 7.57s
528:	learn: 0.0329949	total: 8.48s	remaining: 7.55s
529:	learn: 0.0329354	total: 8.5s	remaining: 7.54s
530:	learn: 0.0329101	total: 8.52s	remaining: 7.52s
531:	learn: 0.0328694	total: 8.53s	remaining: 7.5s
532:	learn: 0.0328382	total: 8.55s	remaining: 7.49s
533:	learn: 0.0327491	total: 8.56s	remaining: 7.47s
534:	learn: 0.0326682	total: 8.57s	remaining: 7.45s
535:	learn: 0.0326239	total: 8.59s	remaining: 7.44s
536:	learn: 0.0325917	total: 8.6s	remaining: 7.42s
537:	learn: 0.0325264	total: 8.62s	remaining: 7.4s
538:	learn: 0.0324641	total: 8.63s	remaining: 7.38s
539:	learn: 0.0324359	total: 8.65s	remaining: 7.37s
540:	learn: 0.0323790	total: 8.66s	remaining: 7.35s
541:	learn: 0.0323361	total: 8.68s	remaining: 7.33s
542:	learn: 0.0323102	total: 8.7s	remaining: 7.32s
543:	learn: 0.0322737	total: 8.71s	remaining: 7.3s
544:	learn: 0.0322493	total: 8.72s	remaining: 7.28s
545:	learn: 0.0322098	total: 8.74s	remaining: 7.27s
546:	learn: 0.0321732	total: 8.75s	remaining: 7.25s
547:	learn: 0.0321445	total: 8.77s	remaining: 7.23s
548:	learn: 0.0321000	total: 8.78s	remaining: 7.21s
549:	learn: 0.0320288	total: 8.8s	remaining: 7.2s
550:	learn: 0.0319822	total: 8.81s	remaining: 7.18s
551:	learn: 0.0319257	total: 8.83s	remaining: 7.16s
552:	learn: 0.0318559	total: 8.84s	remaining: 7.15s
553:	learn: 0.0318390	total: 8.86s	remaining: 7.13s
554:	learn: 0.0318180	total: 8.87s	remaining: 7.11s
555:	learn: 0.0317754	total: 8.88s	remaining: 7.09s
556:	learn: 0.0317377	total: 8.9s	remaining: 7.08s
557:	learn: 0.0317138	total: 8.91s	remaining: 7.06s
558:	learn: 0.0316801	total: 8.93s	remaining: 7.04s

559:	learn: 0.0316183	total: 8.94s	remaining: 7.03s
560:	learn: 0.0315743	total: 8.96s	remaining: 7.01s
561:	learn: 0.0315336	total: 8.97s	remaining: 6.99s
562:	learn: 0.0314708	total: 8.99s	remaining: 6.97s
563:	learn: 0.0314151	total: 9s	remaining: 6.96s
564:	learn: 0.0313777	total: 9.01s	remaining: 6.94s
565:	learn: 0.0313403	total: 9.03s	remaining: 6.92s
566:	learn: 0.0313116	total: 9.04s	remaining: 6.91s
567:	learn: 0.0312876	total: 9.06s	remaining: 6.89s
568:	learn: 0.0312461	total: 9.07s	remaining: 6.87s
569:	learn: 0.0312171	total: 9.09s	remaining: 6.86s
570:	learn: 0.0312034	total: 9.1s	remaining: 6.84s
571:	learn: 0.0311749	total: 9.12s	remaining: 6.82s
572:	learn: 0.0311172	total: 9.13s	remaining: 6.8s
573:	learn: 0.0310816	total: 9.15s	remaining: 6.79s
574:	learn: 0.0310473	total: 9.16s	remaining: 6.77s
575:	learn: 0.0310136	total: 9.18s	remaining: 6.75s
576:	learn: 0.0309771	total: 9.19s	remaining: 6.74s
577:	learn: 0.0309687	total: 9.2s	remaining: 6.72s
578:	learn: 0.0309406	total: 9.22s	remaining: 6.7s
579:	learn: 0.0309149	total: 9.23s	remaining: 6.68s
580:	learn: 0.0308791	total: 9.25s	remaining: 6.67s
581:	learn: 0.0308455	total: 9.26s	remaining: 6.65s
582:	learn: 0.0308153	total: 9.27s	remaining: 6.63s
583:	learn: 0.0307791	total: 9.29s	remaining: 6.62s
584:	learn: 0.0307312	total: 9.3s	remaining: 6.6s
585:	learn: 0.0306988	total: 9.32s	remaining: 6.58s
586:	learn: 0.0306833	total: 9.33s	remaining: 6.57s
587:	learn: 0.0306496	total: 9.35s	remaining: 6.55s
588:	learn: 0.0306206	total: 9.36s	remaining: 6.53s
589:	learn: 0.0306003	total: 9.38s	remaining: 6.52s
590:	learn: 0.0305653	total: 9.39s	remaining: 6.5s
591:	learn: 0.0305409	total: 9.41s	remaining: 6.48s
592:	learn: 0.0305148	total: 9.42s	remaining: 6.46s
593:	learn: 0.0304865	total: 9.44s	remaining: 6.45s
594:	learn: 0.0304403	total: 9.45s	remaining: 6.43s
595:	learn: 0.0304235	total: 9.46s	remaining: 6.41s
596:	learn: 0.0304107	total: 9.48s	remaining: 6.4s
597:	learn: 0.0303945	total: 9.49s	remaining: 6.38s
598:	learn: 0.0303269	total: 9.51s	remaining: 6.36s
599:	learn: 0.0303046	total: 9.52s	remaining: 6.35s
600:	learn: 0.0302540	total: 9.54s	remaining: 6.33s
601:	learn: 0.0301972	total: 9.55s	remaining: 6.31s
602:	learn: 0.0301676	total: 9.56s	remaining: 6.3s
603:	learn: 0.0301326	total: 9.58s	remaining: 6.28s
604:	learn: 0.0300920	total: 9.59s	remaining: 6.26s
605:	learn: 0.0300473	total: 9.61s	remaining: 6.25s
606:	learn: 0.0300172	total: 9.62s	remaining: 6.23s
607:	learn: 0.0300109	total: 9.64s	remaining: 6.21s
608:	learn: 0.0299970	total: 9.65s	remaining: 6.2s
609:	learn: 0.0299535	total: 9.67s	remaining: 6.18s
610:	learn: 0.0299417	total: 9.68s	remaining: 6.16s
611:	learn: 0.0299160	total: 9.69s	remaining: 6.15s
612:	learn: 0.0298710	total: 9.71s	remaining: 6.13s
613:	learn: 0.0298479	total: 9.72s	remaining: 6.11s
614:	learn: 0.0298251	total: 9.74s	remaining: 6.1s

615:	learn: 0.0297500	total: 9.75s	remaining: 6.08s
616:	learn: 0.0297008	total: 9.77s	remaining: 6.06s
617:	learn: 0.0296795	total: 9.78s	remaining: 6.05s
618:	learn: 0.0296666	total: 9.8s	remaining: 6.03s
619:	learn: 0.0296451	total: 9.81s	remaining: 6.01s
620:	learn: 0.0296130	total: 9.83s	remaining: 6s
621:	learn: 0.0295394	total: 9.84s	remaining: 5.98s
622:	learn: 0.0295049	total: 9.86s	remaining: 5.96s
623:	learn: 0.0294824	total: 9.87s	remaining: 5.95s
624:	learn: 0.0294665	total: 9.88s	remaining: 5.93s
625:	learn: 0.0294411	total: 9.9s	remaining: 5.91s
626:	learn: 0.0294338	total: 9.91s	remaining: 5.9s
627:	learn: 0.0294158	total: 9.93s	remaining: 5.88s
628:	learn: 0.0294087	total: 9.94s	remaining: 5.86s
629:	learn: 0.0294023	total: 9.96s	remaining: 5.85s
630:	learn: 0.0293905	total: 9.97s	remaining: 5.83s
631:	learn: 0.0293564	total: 9.99s	remaining: 5.81s
632:	learn: 0.0293055	total: 10s	remaining: 5.8s
633:	learn: 0.0292891	total: 10s	remaining: 5.78s
634:	learn: 0.0292578	total: 10s	remaining: 5.76s
635:	learn: 0.0292199	total: 10s	remaining: 5.75s
636:	learn: 0.0291905	total: 10.1s	remaining: 5.73s
637:	learn: 0.0291679	total: 10.1s	remaining: 5.71s
638:	learn: 0.0291462	total: 10.1s	remaining: 5.7s
639:	learn: 0.0291332	total: 10.1s	remaining: 5.68s
640:	learn: 0.0291179	total: 10.1s	remaining: 5.66s
641:	learn: 0.0290806	total: 10.1s	remaining: 5.65s
642:	learn: 0.0290259	total: 10.1s	remaining: 5.63s
643:	learn: 0.0290115	total: 10.2s	remaining: 5.61s
644:	learn: 0.0289834	total: 10.2s	remaining: 5.6s
645:	learn: 0.0289717	total: 10.2s	remaining: 5.58s
646:	learn: 0.0289171	total: 10.2s	remaining: 5.57s
647:	learn: 0.0288975	total: 10.2s	remaining: 5.55s
648:	learn: 0.0288600	total: 10.2s	remaining: 5.53s
649:	learn: 0.0288340	total: 10.2s	remaining: 5.52s
650:	learn: 0.0287977	total: 10.3s	remaining: 5.5s
651:	learn: 0.0287717	total: 10.3s	remaining: 5.48s
652:	learn: 0.0287293	total: 10.3s	remaining: 5.47s
653:	learn: 0.0287095	total: 10.3s	remaining: 5.45s
654:	learn: 0.0286872	total: 10.3s	remaining: 5.43s
655:	learn: 0.0286700	total: 10.3s	remaining: 5.42s
656:	learn: 0.0286464	total: 10.3s	remaining: 5.4s
657:	learn: 0.0286329	total: 10.4s	remaining: 5.38s
658:	learn: 0.0285927	total: 10.4s	remaining: 5.37s
659:	learn: 0.0285771	total: 10.4s	remaining: 5.35s
660:	learn: 0.0285583	total: 10.4s	remaining: 5.34s
661:	learn: 0.0285437	total: 10.4s	remaining: 5.32s
662:	learn: 0.0285285	total: 10.4s	remaining: 5.3s
663:	learn: 0.0284988	total: 10.4s	remaining: 5.29s
664:	learn: 0.0284630	total: 10.5s	remaining: 5.27s
665:	learn: 0.0284218	total: 10.5s	remaining: 5.26s
666:	learn: 0.0283702	total: 10.5s	remaining: 5.24s
667:	learn: 0.0283337	total: 10.5s	remaining: 5.22s
668:	learn: 0.0283257	total: 10.5s	remaining: 5.21s
669:	learn: 0.0282997	total: 10.5s	remaining: 5.19s
670:	learn: 0.0282862	total: 10.6s	remaining: 5.17s

671:	learn: 0.0282219	total: 10.6s	remaining: 5.16s
672:	learn: 0.0281859	total: 10.6s	remaining: 5.14s
673:	learn: 0.0281534	total: 10.6s	remaining: 5.13s
674:	learn: 0.0281175	total: 10.6s	remaining: 5.11s
675:	learn: 0.0280988	total: 10.6s	remaining: 5.09s
676:	learn: 0.0280664	total: 10.6s	remaining: 5.08s
677:	learn: 0.0280401	total: 10.7s	remaining: 5.06s
678:	learn: 0.0280289	total: 10.7s	remaining: 5.04s
679:	learn: 0.0279979	total: 10.7s	remaining: 5.03s
680:	learn: 0.0279762	total: 10.7s	remaining: 5.01s
681:	learn: 0.0279565	total: 10.7s	remaining: 5s
682:	learn: 0.0279460	total: 10.7s	remaining: 4.98s
683:	learn: 0.0279183	total: 10.7s	remaining: 4.96s
684:	learn: 0.0278975	total: 10.8s	remaining: 4.95s
685:	learn: 0.0278390	total: 10.8s	remaining: 4.93s
686:	learn: 0.0278327	total: 10.8s	remaining: 4.92s
687:	learn: 0.0278243	total: 10.8s	remaining: 4.9s
688:	learn: 0.0277932	total: 10.8s	remaining: 4.88s
689:	learn: 0.0277816	total: 10.8s	remaining: 4.87s
690:	learn: 0.0277628	total: 10.8s	remaining: 4.85s
691:	learn: 0.0277305	total: 10.9s	remaining: 4.83s
692:	learn: 0.0277169	total: 10.9s	remaining: 4.82s
693:	learn: 0.0276786	total: 10.9s	remaining: 4.8s
694:	learn: 0.0276375	total: 10.9s	remaining: 4.79s
695:	learn: 0.0276283	total: 10.9s	remaining: 4.77s
696:	learn: 0.0276160	total: 10.9s	remaining: 4.75s
697:	learn: 0.0276045	total: 10.9s	remaining: 4.74s
698:	learn: 0.0275735	total: 11s	remaining: 4.72s
699:	learn: 0.0275435	total: 11s	remaining: 4.7s
700:	learn: 0.0275275	total: 11s	remaining: 4.69s
701:	learn: 0.0275180	total: 11s	remaining: 4.67s
702:	learn: 0.0274995	total: 11s	remaining: 4.65s
703:	learn: 0.0274800	total: 11s	remaining: 4.64s
704:	learn: 0.0274560	total: 11s	remaining: 4.62s
705:	learn: 0.0274437	total: 11.1s	remaining: 4.61s
706:	learn: 0.0274020	total: 11.1s	remaining: 4.59s
707:	learn: 0.0273566	total: 11.1s	remaining: 4.57s
708:	learn: 0.0273418	total: 11.1s	remaining: 4.56s
709:	learn: 0.0273359	total: 11.1s	remaining: 4.54s
710:	learn: 0.0273246	total: 11.1s	remaining: 4.52s
711:	learn: 0.0272896	total: 11.1s	remaining: 4.51s
712:	learn: 0.0272722	total: 11.2s	remaining: 4.49s
713:	learn: 0.0272438	total: 11.2s	remaining: 4.48s
714:	learn: 0.0271922	total: 11.2s	remaining: 4.46s
715:	learn: 0.0271850	total: 11.2s	remaining: 4.44s
716:	learn: 0.0271748	total: 11.2s	remaining: 4.43s
717:	learn: 0.0271557	total: 11.2s	remaining: 4.41s
718:	learn: 0.0271469	total: 11.2s	remaining: 4.39s
719:	learn: 0.0271226	total: 11.3s	remaining: 4.38s
720:	learn: 0.0270648	total: 11.3s	remaining: 4.36s
721:	learn: 0.0270306	total: 11.3s	remaining: 4.35s
722:	learn: 0.0270068	total: 11.3s	remaining: 4.33s
723:	learn: 0.0269874	total: 11.3s	remaining: 4.32s
724:	learn: 0.0269724	total: 11.3s	remaining: 4.3s
725:	learn: 0.0269544	total: 11.3s	remaining: 4.28s
726:	learn: 0.0269288	total: 11.4s	remaining: 4.27s

727:	learn: 0.0269185	total: 11.4s	remaining: 4.25s
728:	learn: 0.0268691	total: 11.4s	remaining: 4.23s
729:	learn: 0.0268123	total: 11.4s	remaining: 4.22s
730:	learn: 0.0267700	total: 11.4s	remaining: 4.2s
731:	learn: 0.0267481	total: 11.4s	remaining: 4.19s
732:	learn: 0.0267278	total: 11.4s	remaining: 4.17s
733:	learn: 0.0267015	total: 11.5s	remaining: 4.16s
734:	learn: 0.0266859	total: 11.5s	remaining: 4.14s
735:	learn: 0.0266778	total: 11.5s	remaining: 4.12s
736:	learn: 0.0266677	total: 11.5s	remaining: 4.11s
737:	learn: 0.0266157	total: 11.5s	remaining: 4.09s
738:	learn: 0.0265842	total: 11.5s	remaining: 4.07s
739:	learn: 0.0265711	total: 11.6s	remaining: 4.06s
740:	learn: 0.0265446	total: 11.6s	remaining: 4.04s
741:	learn: 0.0265177	total: 11.6s	remaining: 4.03s
742:	learn: 0.0264904	total: 11.6s	remaining: 4.01s
743:	learn: 0.0264742	total: 11.6s	remaining: 3.99s
744:	learn: 0.0264600	total: 11.6s	remaining: 3.98s
745:	learn: 0.0264525	total: 11.6s	remaining: 3.96s
746:	learn: 0.0264423	total: 11.7s	remaining: 3.95s
747:	learn: 0.0264273	total: 11.7s	remaining: 3.93s
748:	learn: 0.0264006	total: 11.7s	remaining: 3.91s
749:	learn: 0.0263735	total: 11.7s	remaining: 3.9s
750:	learn: 0.0263394	total: 11.7s	remaining: 3.88s
751:	learn: 0.0263302	total: 11.7s	remaining: 3.87s
752:	learn: 0.0263243	total: 11.7s	remaining: 3.85s
753:	learn: 0.0263074	total: 11.8s	remaining: 3.83s
754:	learn: 0.0262848	total: 11.8s	remaining: 3.82s
755:	learn: 0.0262429	total: 11.8s	remaining: 3.8s
756:	learn: 0.0262216	total: 11.8s	remaining: 3.79s
757:	learn: 0.0262070	total: 11.8s	remaining: 3.77s
758:	learn: 0.0262028	total: 11.8s	remaining: 3.75s
759:	learn: 0.0261726	total: 11.9s	remaining: 3.74s
760:	learn: 0.0261518	total: 11.9s	remaining: 3.73s
761:	learn: 0.0261359	total: 11.9s	remaining: 3.71s
762:	learn: 0.0261129	total: 11.9s	remaining: 3.7s
763:	learn: 0.0261009	total: 11.9s	remaining: 3.69s
764:	learn: 0.0260836	total: 11.9s	remaining: 3.67s
765:	learn: 0.0260638	total: 12s	remaining: 3.66s
766:	learn: 0.0260349	total: 12s	remaining: 3.64s
767:	learn: 0.0260118	total: 12s	remaining: 3.63s
768:	learn: 0.0259859	total: 12s	remaining: 3.61s
769:	learn: 0.0259690	total: 12s	remaining: 3.6s
770:	learn: 0.0259395	total: 12.1s	remaining: 3.58s
771:	learn: 0.0258957	total: 12.1s	remaining: 3.56s
772:	learn: 0.0258698	total: 12.1s	remaining: 3.55s
773:	learn: 0.0258388	total: 12.1s	remaining: 3.53s
774:	learn: 0.0258285	total: 12.1s	remaining: 3.52s
775:	learn: 0.0257879	total: 12.1s	remaining: 3.5s
776:	learn: 0.0257717	total: 12.1s	remaining: 3.49s
777:	learn: 0.0257222	total: 12.2s	remaining: 3.47s
778:	learn: 0.0257137	total: 12.2s	remaining: 3.45s
779:	learn: 0.0257005	total: 12.2s	remaining: 3.44s
780:	learn: 0.0256894	total: 12.2s	remaining: 3.42s
781:	learn: 0.0256461	total: 12.2s	remaining: 3.41s
782:	learn: 0.0256409	total: 12.2s	remaining: 3.39s

783:	learn: 0.0255983	total: 12.2s	remaining: 3.37s
784:	learn: 0.0255600	total: 12.3s	remaining: 3.36s
785:	learn: 0.0255369	total: 12.3s	remaining: 3.34s
786:	learn: 0.0254945	total: 12.3s	remaining: 3.33s
787:	learn: 0.0254466	total: 12.3s	remaining: 3.31s
788:	learn: 0.0254270	total: 12.3s	remaining: 3.29s
789:	learn: 0.0253646	total: 12.3s	remaining: 3.28s
790:	learn: 0.0253301	total: 12.3s	remaining: 3.26s
791:	learn: 0.0252927	total: 12.4s	remaining: 3.25s
792:	learn: 0.0252578	total: 12.4s	remaining: 3.23s
793:	learn: 0.0252315	total: 12.4s	remaining: 3.21s
794:	learn: 0.0251878	total: 12.4s	remaining: 3.2s
795:	learn: 0.0251775	total: 12.4s	remaining: 3.18s
796:	learn: 0.0251726	total: 12.4s	remaining: 3.17s
797:	learn: 0.0251503	total: 12.5s	remaining: 3.15s
798:	learn: 0.0251289	total: 12.5s	remaining: 3.14s
799:	learn: 0.0250946	total: 12.5s	remaining: 3.12s
800:	learn: 0.0250822	total: 12.5s	remaining: 3.11s
801:	learn: 0.0250591	total: 12.5s	remaining: 3.09s
802:	learn: 0.0250149	total: 12.5s	remaining: 3.07s
803:	learn: 0.0249814	total: 12.5s	remaining: 3.06s
804:	learn: 0.0249580	total: 12.6s	remaining: 3.04s
805:	learn: 0.0249374	total: 12.6s	remaining: 3.03s
806:	learn: 0.0249179	total: 12.6s	remaining: 3.01s
807:	learn: 0.0249037	total: 12.6s	remaining: 3s
808:	learn: 0.0248950	total: 12.6s	remaining: 2.98s
809:	learn: 0.0248847	total: 12.6s	remaining: 2.96s
810:	learn: 0.0248436	total: 12.6s	remaining: 2.95s
811:	learn: 0.0248192	total: 12.7s	remaining: 2.93s
812:	learn: 0.0248065	total: 12.7s	remaining: 2.92s
813:	learn: 0.0247796	total: 12.7s	remaining: 2.9s
814:	learn: 0.0247570	total: 12.7s	remaining: 2.88s
815:	learn: 0.0247421	total: 12.7s	remaining: 2.87s
816:	learn: 0.0247232	total: 12.7s	remaining: 2.85s
817:	learn: 0.0246967	total: 12.8s	remaining: 2.84s
818:	learn: 0.0246717	total: 12.8s	remaining: 2.82s
819:	learn: 0.0246635	total: 12.8s	remaining: 2.81s
820:	learn: 0.0246564	total: 12.8s	remaining: 2.79s
821:	learn: 0.0246355	total: 12.8s	remaining: 2.77s
822:	learn: 0.0246114	total: 12.8s	remaining: 2.76s
823:	learn: 0.0245950	total: 12.8s	remaining: 2.74s
824:	learn: 0.0245668	total: 12.9s	remaining: 2.73s
825:	learn: 0.0245548	total: 12.9s	remaining: 2.71s
826:	learn: 0.0245419	total: 12.9s	remaining: 2.69s
827:	learn: 0.0245333	total: 12.9s	remaining: 2.68s
828:	learn: 0.0245133	total: 12.9s	remaining: 2.66s
829:	learn: 0.0244954	total: 12.9s	remaining: 2.65s
830:	learn: 0.0244717	total: 12.9s	remaining: 2.63s
831:	learn: 0.0244448	total: 13s	remaining: 2.62s
832:	learn: 0.0244086	total: 13s	remaining: 2.6s
833:	learn: 0.0243887	total: 13s	remaining: 2.58s
834:	learn: 0.0243630	total: 13s	remaining: 2.57s
835:	learn: 0.0243543	total: 13s	remaining: 2.55s
836:	learn: 0.0243499	total: 13s	remaining: 2.54s
837:	learn: 0.0243386	total: 13s	remaining: 2.52s
838:	learn: 0.0243104	total: 13.1s	remaining: 2.5s

839:	learn: 0.0243013	total: 13.1s	remaining: 2.49s
840:	learn: 0.0242625	total: 13.1s	remaining: 2.47s
841:	learn: 0.0242527	total: 13.1s	remaining: 2.46s
842:	learn: 0.0242259	total: 13.1s	remaining: 2.44s
843:	learn: 0.0241946	total: 13.1s	remaining: 2.43s
844:	learn: 0.0241890	total: 13.1s	remaining: 2.41s
845:	learn: 0.0241651	total: 13.2s	remaining: 2.4s
846:	learn: 0.0241571	total: 13.2s	remaining: 2.38s
847:	learn: 0.0241297	total: 13.2s	remaining: 2.36s
848:	learn: 0.0241042	total: 13.2s	remaining: 2.35s
849:	learn: 0.0240888	total: 13.2s	remaining: 2.33s
850:	learn: 0.0240814	total: 13.2s	remaining: 2.32s
851:	learn: 0.0240595	total: 13.2s	remaining: 2.3s
852:	learn: 0.0240430	total: 13.3s	remaining: 2.29s
853:	learn: 0.0240093	total: 13.3s	remaining: 2.27s
854:	learn: 0.0239701	total: 13.3s	remaining: 2.25s
855:	learn: 0.0239623	total: 13.3s	remaining: 2.24s
856:	learn: 0.0239372	total: 13.3s	remaining: 2.22s
857:	learn: 0.0239018	total: 13.3s	remaining: 2.21s
858:	learn: 0.0238734	total: 13.3s	remaining: 2.19s
859:	learn: 0.0238734	total: 13.4s	remaining: 2.17s
860:	learn: 0.0238656	total: 13.4s	remaining: 2.16s
861:	learn: 0.0238517	total: 13.4s	remaining: 2.14s
862:	learn: 0.0238476	total: 13.4s	remaining: 2.13s
863:	learn: 0.0238368	total: 13.4s	remaining: 2.11s
864:	learn: 0.0238237	total: 13.4s	remaining: 2.1s
865:	learn: 0.0237930	total: 13.4s	remaining: 2.08s
866:	learn: 0.0237604	total: 13.5s	remaining: 2.06s
867:	learn: 0.0237383	total: 13.5s	remaining: 2.05s
868:	learn: 0.0237153	total: 13.5s	remaining: 2.03s
869:	learn: 0.0237050	total: 13.5s	remaining: 2.02s
870:	learn: 0.0236857	total: 13.5s	remaining: 2s
871:	learn: 0.0236730	total: 13.5s	remaining: 1.99s
872:	learn: 0.0236633	total: 13.5s	remaining: 1.97s
873:	learn: 0.0236484	total: 13.6s	remaining: 1.96s
874:	learn: 0.0236307	total: 13.6s	remaining: 1.94s
875:	learn: 0.0236013	total: 13.6s	remaining: 1.92s
876:	learn: 0.0235796	total: 13.6s	remaining: 1.91s
877:	learn: 0.0235660	total: 13.6s	remaining: 1.89s
878:	learn: 0.0235614	total: 13.6s	remaining: 1.88s
879:	learn: 0.0235455	total: 13.7s	remaining: 1.86s
880:	learn: 0.0235375	total: 13.7s	remaining: 1.84s
881:	learn: 0.0235276	total: 13.7s	remaining: 1.83s
882:	learn: 0.0235058	total: 13.7s	remaining: 1.81s
883:	learn: 0.0234964	total: 13.7s	remaining: 1.8s
884:	learn: 0.0234760	total: 13.7s	remaining: 1.78s
885:	learn: 0.0234582	total: 13.7s	remaining: 1.77s
886:	learn: 0.0234321	total: 13.8s	remaining: 1.75s
887:	learn: 0.0234061	total: 13.8s	remaining: 1.74s
888:	learn: 0.0233871	total: 13.8s	remaining: 1.72s
889:	learn: 0.0233765	total: 13.8s	remaining: 1.71s
890:	learn: 0.0233416	total: 13.8s	remaining: 1.69s
891:	learn: 0.0233214	total: 13.8s	remaining: 1.67s
892:	learn: 0.0232989	total: 13.8s	remaining: 1.66s
893:	learn: 0.0232867	total: 13.9s	remaining: 1.64s
894:	learn: 0.0232749	total: 13.9s	remaining: 1.63s

895:	learn: 0.0232602	total: 13.9s	remaining: 1.61s
896:	learn: 0.0232481	total: 13.9s	remaining: 1.59s
897:	learn: 0.0232307	total: 13.9s	remaining: 1.58s
898:	learn: 0.0231974	total: 13.9s	remaining: 1.56s
899:	learn: 0.0231754	total: 13.9s	remaining: 1.55s
900:	learn: 0.0231631	total: 14s	remaining: 1.53s
901:	learn: 0.0231552	total: 14s	remaining: 1.52s
902:	learn: 0.0231307	total: 14s	remaining: 1.5s
903:	learn: 0.0231080	total: 14s	remaining: 1.49s
904:	learn: 0.0230799	total: 14s	remaining: 1.47s
905:	learn: 0.0230654	total: 14s	remaining: 1.46s
906:	learn: 0.0230458	total: 14s	remaining: 1.44s
907:	learn: 0.0230194	total: 14.1s	remaining: 1.42s
908:	learn: 0.0230090	total: 14.1s	remaining: 1.41s
909:	learn: 0.0230020	total: 14.1s	remaining: 1.39s
910:	learn: 0.0229763	total: 14.1s	remaining: 1.38s
911:	learn: 0.0229594	total: 14.1s	remaining: 1.36s
912:	learn: 0.0229522	total: 14.1s	remaining: 1.35s
913:	learn: 0.0229432	total: 14.1s	remaining: 1.33s
914:	learn: 0.0229103	total: 14.2s	remaining: 1.31s
915:	learn: 0.0229001	total: 14.2s	remaining: 1.3s
916:	learn: 0.0228892	total: 14.2s	remaining: 1.28s
917:	learn: 0.0228513	total: 14.2s	remaining: 1.27s
918:	learn: 0.0228316	total: 14.2s	remaining: 1.25s
919:	learn: 0.0228120	total: 14.2s	remaining: 1.24s
920:	learn: 0.0227783	total: 14.2s	remaining: 1.22s
921:	learn: 0.0227487	total: 14.3s	remaining: 1.21s
922:	learn: 0.0227203	total: 14.3s	remaining: 1.19s
923:	learn: 0.0227045	total: 14.3s	remaining: 1.18s
924:	learn: 0.0226866	total: 14.3s	remaining: 1.16s
925:	learn: 0.0226794	total: 14.3s	remaining: 1.14s
926:	learn: 0.0226558	total: 14.3s	remaining: 1.13s
927:	learn: 0.0226392	total: 14.3s	remaining: 1.11s
928:	learn: 0.0226254	total: 14.4s	remaining: 1.1s
929:	learn: 0.0226157	total: 14.4s	remaining: 1.08s
930:	learn: 0.0226025	total: 14.4s	remaining: 1.07s
931:	learn: 0.0225924	total: 14.4s	remaining: 1.05s
932:	learn: 0.0225865	total: 14.4s	remaining: 1.03s
933:	learn: 0.0225688	total: 14.4s	remaining: 1.02s
934:	learn: 0.0225557	total: 14.5s	remaining: 1s
935:	learn: 0.0225482	total: 14.5s	remaining: 989ms
936:	learn: 0.0225326	total: 14.5s	remaining: 974ms
937:	learn: 0.0225140	total: 14.5s	remaining: 958ms
938:	learn: 0.0225035	total: 14.5s	remaining: 943ms
939:	learn: 0.0224840	total: 14.5s	remaining: 927ms
940:	learn: 0.0224646	total: 14.5s	remaining: 912ms
941:	learn: 0.0224507	total: 14.6s	remaining: 896ms
942:	learn: 0.0224341	total: 14.6s	remaining: 881ms
943:	learn: 0.0224232	total: 14.6s	remaining: 865ms
944:	learn: 0.0224057	total: 14.6s	remaining: 850ms
945:	learn: 0.0223888	total: 14.6s	remaining: 834ms
946:	learn: 0.0223713	total: 14.6s	remaining: 819ms
947:	learn: 0.0223664	total: 14.6s	remaining: 803ms
948:	learn: 0.0223529	total: 14.7s	remaining: 788ms
949:	learn: 0.0223331	total: 14.7s	remaining: 772ms
950:	learn: 0.0223218	total: 14.7s	remaining: 757ms

951:	learn: 0.0222965	total: 14.7s	remaining: 742ms
952:	learn: 0.0222895	total: 14.7s	remaining: 726ms
953:	learn: 0.0222686	total: 14.7s	remaining: 711ms
954:	learn: 0.0222594	total: 14.7s	remaining: 695ms
955:	learn: 0.0222466	total: 14.8s	remaining: 680ms
956:	learn: 0.0222356	total: 14.8s	remaining: 664ms
957:	learn: 0.0222313	total: 14.8s	remaining: 649ms
958:	learn: 0.0221961	total: 14.8s	remaining: 633ms
959:	learn: 0.0221751	total: 14.8s	remaining: 618ms
960:	learn: 0.0221494	total: 14.8s	remaining: 602ms
961:	learn: 0.0221320	total: 14.9s	remaining: 587ms
962:	learn: 0.0221236	total: 14.9s	remaining: 571ms
963:	learn: 0.0221093	total: 14.9s	remaining: 556ms
964:	learn: 0.0220972	total: 14.9s	remaining: 540ms
965:	learn: 0.0220878	total: 14.9s	remaining: 525ms
966:	learn: 0.0220564	total: 14.9s	remaining: 509ms
967:	learn: 0.0220351	total: 14.9s	remaining: 494ms
968:	learn: 0.0220189	total: 15s	remaining: 478ms
969:	learn: 0.0219986	total: 15s	remaining: 463ms
970:	learn: 0.0219769	total: 15s	remaining: 447ms
971:	learn: 0.0219647	total: 15s	remaining: 432ms
972:	learn: 0.0219373	total: 15s	remaining: 417ms
973:	learn: 0.0219325	total: 15s	remaining: 401ms
974:	learn: 0.0219219	total: 15s	remaining: 386ms
975:	learn: 0.0218934	total: 15.1s	remaining: 370ms
976:	learn: 0.0218859	total: 15.1s	remaining: 355ms
977:	learn: 0.0218732	total: 15.1s	remaining: 339ms
978:	learn: 0.0218467	total: 15.1s	remaining: 324ms
979:	learn: 0.0218310	total: 15.1s	remaining: 308ms
980:	learn: 0.0218091	total: 15.1s	remaining: 293ms
981:	learn: 0.0217957	total: 15.1s	remaining: 278ms
982:	learn: 0.0217867	total: 15.2s	remaining: 262ms
983:	learn: 0.0217634	total: 15.2s	remaining: 247ms
984:	learn: 0.0217453	total: 15.2s	remaining: 231ms
985:	learn: 0.0217221	total: 15.2s	remaining: 216ms
986:	learn: 0.0217068	total: 15.2s	remaining: 200ms
987:	learn: 0.0216920	total: 15.2s	remaining: 185ms
988:	learn: 0.0216804	total: 15.2s	remaining: 170ms
989:	learn: 0.0216725	total: 15.3s	remaining: 154ms
990:	learn: 0.0216652	total: 15.3s	remaining: 139ms
991:	learn: 0.0216551	total: 15.3s	remaining: 123ms
992:	learn: 0.0216167	total: 15.3s	remaining: 108ms
993:	learn: 0.0215961	total: 15.3s	remaining: 92.5ms
994:	learn: 0.0215696	total: 15.3s	remaining: 77ms
995:	learn: 0.0215566	total: 15.3s	remaining: 61.6ms
996:	learn: 0.0215247	total: 15.4s	remaining: 46.2ms
997:	learn: 0.0215071	total: 15.4s	remaining: 30.8ms
998:	learn: 0.0214967	total: 15.4s	remaining: 15.4ms
999:	learn: 0.0214788	total: 15.4s	remaining: 0us

Out[103... CatBoostClassifier(random_seed=42)

In [104... `# prediction`
`evaluation(reg_catb)`

```

=====
[[19071  205]
 [   26 19250]]
      precision    recall  f1-score   support

         0         1.00      0.99      0.99     19276
         1         0.99      1.00      0.99     19276

 accuracy          0.99          0.99          0.99     38552
 macro avg          0.99          0.99          0.99     38552
 weighted avg       0.99          0.99          0.99     38552

```

Accuracy of TRAIN data: 99.40080929653455

F1_Score of TRAIN data: 99.40079637879083

```

=====
[[8057  204]
 [   74  665]]
      precision    recall  f1-score   support

         0         0.99      0.98      0.98      8261
         1         0.77      0.90      0.83       739

 accuracy          0.97          0.97          0.97     9000
 macro avg          0.88          0.94          0.91     9000
 weighted avg       0.97          0.97          0.97     9000

```

Accuracy of TEST data: 96.91111111111111

F1_Score of TEST data: 90.50774677126812

```

=====
RMSE:  0.1757523510195209
ROC AUC score:  0.937585167535671

```

In []:

XGBoost

In [105...

```
# Initialize and fit the model
```

```
xgb_model = XGBClassifier(random_seed=42)
xgb_model.fit(X_train, y_train)
```

C:\Users\Home\anaconda3\Lib\site-packages\xgboost\training.py:200: UserWarning: [17:40:16] WARNING: C:\actions-runner_work\xgboost\xgboost\src\learner.cc:782: Parameters: { "random_seed" } are not used.

```
bst.update(dtrain, iteration=i, fobj=obj)
```

Out[105...

```

XGBClassifier
XGBClassifier(base_score=None, booster=None, callbacks=None,
               colsample_bylevel=None, colsample_bynode=None,
               colsample_bytree=None, device=None, early_stopping_rounds=
None,
               enable_categorical=False, eval_metric=None, feature_types=
None,
               feature_weights=None, gamma=None, grow_policy=None,
               importance_type=None, interaction_constraints=None,
               learning_rate=None, max_bin=None, max_cat_threshold=None,

```

In [106...

```

# prediction
evaluation(xgb_model)

```

```

=====
[[19044  232]
 [   23 19253]]
      precision    recall  f1-score   support

     0       1.00      0.99      0.99     19276
     1       0.99      1.00      0.99     19276

 accuracy          0.99      38552
 macro avg         0.99      0.99      0.99     38552
weighted avg         0.99      0.99      0.99     38552

Accuracy of TRAIN data: 99.33855571695372
F1_Score of TRAIN data: 99.33853627657253
=====
[[8044  217]
 [   70  669]]
      precision    recall  f1-score   support

     0       0.99      0.97      0.98      8261
     1       0.76      0.91      0.82       739

 accuracy          0.97      9000
 macro avg         0.87      0.94      0.90      9000
weighted avg         0.97      0.97      0.97      9000

Accuracy of TEST data: 96.81111111111112
F1_Score of TEST data: 90.29289489136818
=====
RMSE:  0.17857460314638499
ROC AUC score:  0.9395046977999072

```

In []:

In [107...

```

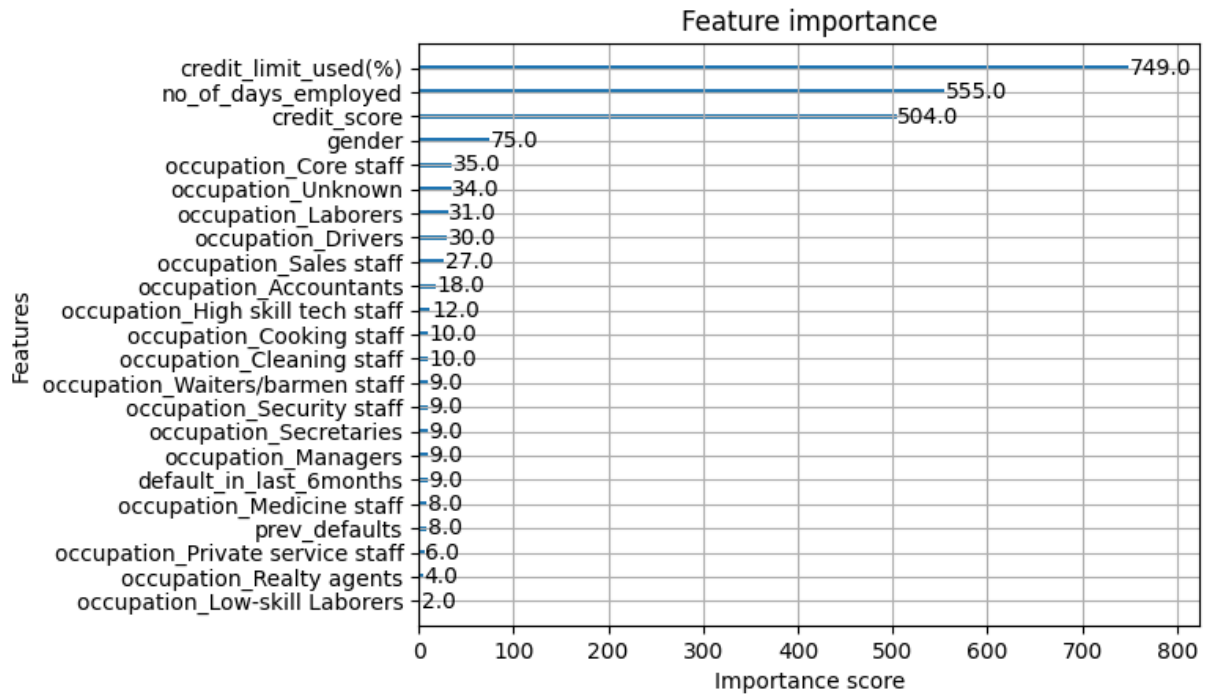
xgb_model.feature_importances_

```

```
Out[107...] array([0.00931961, 0.01161044, 0.11123751, 0.62002844, 0.03398515,
      0.02012188, 0.00746186, 0.01355023, 0.00649294, 0.01391891,
      0.01388959, 0. , 0.02209179, 0. , 0.01000313,
      0.00386124, 0.0166196 , 0.00754175, 0.01885029, 0.00214677,
      0.01209913, 0.01452901, 0.00553574, 0.01804676, 0.00705822],
      dtype=float32)
```

```
In [108...] from xgboost import plot_importance
```

```
In [109...] plot_importance(xgb_model)
plt.show()
plt.savefig("Feature importance.png")
```



<Figure size 640x480 with 0 Axes>

```
In [ ]:
```

LightGBM

```
In [110...] # Initialize and fit the model

lgbm_model = LGBMClassifier(random_state=42)
lgbm_model.fit(X_train, y_train)
```



```
[LightGBM] [Warning] Found whitespace in feature_names, replace with underlines
[LightGBM] [Info] Number of positive: 19276, number of negative: 19276
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.004466 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 1199
[LightGBM] [Info] Number of data points in the train set: 38552, number of used features: 25
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.500000 -> initscore=0.000000
```

Out[110]...

```
LGBMClassifier
LGBMClassifier(random_state=42)
```

In [111]...

```
# prediction
evaluation(lgbm_model)
```

```
=====
[[19004  272]
 [   46 19230]]
      precision    recall  f1-score   support

      0       1.00      0.99      0.99      19276
      1       0.99      1.00      0.99      19276

   accuracy                0.99      38552
  macro avg              0.99      0.99      0.99      38552
 weighted avg              0.99      0.99      0.99      38552
```

Accuracy of TRAIN data: 99.17514007055405

F1_Score of TRAIN data: 99.17511172283086

```
=====
[[8048  213]
 [   75  664]]
      precision    recall  f1-score   support

      0       0.99      0.97      0.98      8261
      1       0.76      0.90      0.82       739

   accuracy                0.97      9000
  macro avg              0.87      0.94      0.90      9000
 weighted avg              0.97      0.97      0.97      9000
```

Accuracy of TEST data: 96.8

F1_Score of TEST data: 90.21020266089108

```
=====
RMSE: 0.17888543819998318
ROC AUC score: 0.9363638493080697
```

In []:

KNN

In [112... *## Iterating knn model for all values of k from 1-39*

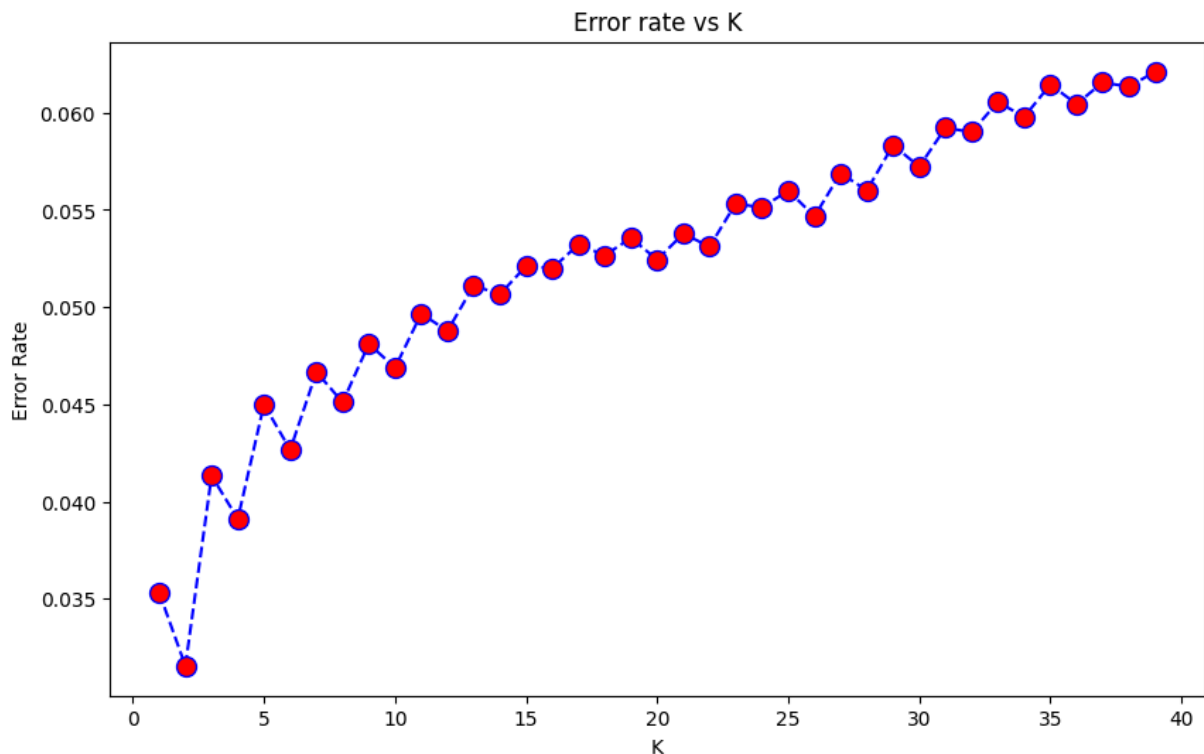
```
error_rate = []

for i in range(1,40):
    knn = KNeighborsClassifier(n_neighbors=i)
    knn.fit(X_train, y_train)
    pred_i = knn.predict(X_test)
    error_rate.append(np.mean(pred_i != y_test))
```

In [113... *# plotting the error_rate*

```
plt.figure(figsize=[10,6])
plt.plot(range(1,40), error_rate, color='blue', linestyle='dashed',
        marker='o', markerfacecolor='red', markersize=10)
plt.title('Error rate vs K')
plt.xlabel('K')
plt.ylabel('Error Rate')
```

Out[113... Text(0, 0.5, 'Error Rate')



In [114... *# # building knn and fit the model*

```
knn = KNeighborsClassifier(n_neighbors = 2)
knn.fit(X_train, y_train)
```

Out[114...

▼ KNeighborsClassifier ⓘ ?
KNeighborsClassifier(n_neighbors=2)

In [115...]

Evaluation

evaluation(knn)

```

=====
[[19276    0]
 [  249 19027]]
      precision    recall  f1-score   support

         0         0.99      1.00      0.99      19276
         1         1.00      0.99      0.99      19276

 accuracy          0.99      38552
 macro avg         0.99      0.99      0.99      38552
 weighted avg      0.99      0.99      0.99      38552

```

Accuracy of TRAIN data: 99.35411911184893

F1_Score of TRAIN data: 99.35409216702074

```

=====
[[8067  194]
 [  90  649]]
      precision    recall  f1-score   support

         0         0.99      0.98      0.98      8261
         1         0.77      0.88      0.82      739

 accuracy          0.97      9000
 macro avg         0.88      0.93      0.90      9000
 weighted avg      0.97      0.97      0.97      9000

```

Accuracy of TEST data: 96.84444444444445

F1_Score of TEST data: 90.15911585431118

```

=====
RMSE:  0.1776388345929897
ROC AUC score:  0.9273649813534388

```

In []:

Conclusion: - Random Forest, Decision Tree, LightGBM, KNN, CatBoost and XGBoost models performed well with almost the same accuracy of 0.96 However minor difference exists in F1_Score, precision and recall. - The most significant variables in predicting credit card defaults are credit score, credit limit utilization and number of days employed. - Age of the borrower is not a significant factor in predicting credit card defaults highlighting the importance of considering other variables when analyzing credit risk. - By using machine learning techniques such as XGBoost, Random Forest, Decision Tree, LightGBM, KNN, CatBoost financial institutions can better identify and manage credit risk, thereby reducing their losses due to defaults.

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []:

In []: